TEAM SEVEN

Airoplane Ticket Service

Flight Reservation & E-Ticket System · Complete Source Code

Project Title	Airoplane Ticket Service — Flight Reservation & E-Ticket System
Team Name	Team Seven
Team Members	1. _____ 2. _____ 3. _____
Guide / Faculty	_____
Department	Computer Science & Engineering
College / Institution	_____
Academic Year	2025 – 2026
Date of Submission	_____
Repository	github.com/Mrkarthik123/airoplane-ticket-service

58
source files ·
5,204
lines of code
Java
3,163
lines · JavaScript
757
lines · HTML
398
lines · XML
348
lines · CSS
167
lines · Markdown
139
lines · Gradle
104
lines · Shell
52
lines · txt

32
lines · JSON
29
lines · Shell
15
lines

Table of contents

Part 1 — Desktop Application and HTTP Server (Java 17)

1. src/main/java/com/airline/App.java — 358 lines
2. src/main/java/com/airline/model/Flight.java — 65 lines
3. src/main/java/com/airline/model/User.java — 32 lines
4. src/main/java/com/airline/model/Booking.java — 48 lines
5. src/main/java/com/airline/service/AuthService.java — 58 lines
6. src/main/java/com/airline/service/FlightService.java — 58 lines
7. src/main/java/com/airline/service/BookingService.java — 118 lines
8. src/main/java/com/airline/store/DataStore.java — 173 lines
9. src/main/java/com/airline/util/Json.java — 181 lines
0. src/main/java/com/airline/util/HttpUtil.java — 57 lines
1. src/main/java/com/airline/util/Passwords.java — 25 lines
2. src/main/java/com/airline/ui/DesktopLauncher.java — 27 lines
3. src/main/java/com/airline/ui/MainFrame.java — 133 lines
4. src/main/java/com/airline/ui/LoginDialog.java — 126 lines
5. src/main/java/com/airline/ui/Theme.java — 176 lines
6. src/main/java/com/airline/ui/PlaceholderField.java — 43 lines
7. src/main/java/com/airline/ui/PlaceholderPasswordField.java — 43 lines
8. src/main/java/com/airline/ui/SkyPanel.java — 101 lines
9. src/main/java/com/airline/ui/SearchPanel.java — 190 lines
0. src/main/java/com/airline/ui/AirportsPanel.java — 172 lines
1. src/main/java/com/airline/ui/BookingDialog.java — 313 lines
2. src/main/java/com/airline/ui/BookingsPanel.java — 137 lines
3. src/main/java/com/airline/ui/TicketDialog.java — 117 lines
4. src/main/java/com/airline/ui/AdminPanel.java — 342 lines

Part 2 — Web and Progressive Web App Front End

1. src/main/resources/static/index.html — 199 lines
2. src/main/resources/static/styles.css — 85 lines
3. src/main/resources/static/app.js — 364 lines
4. src/main/resources/static/manifest.webmanifest — 16 lines
5. src/main/resources/static/sw.js — 29 lines

Part 3 — Build, Packaging and Run Scripts

1. pom.xml — 46 lines
2. run.sh — 24 lines
3. AiroplaneTicketService.command — 15 lines
4. mobile/package.json — 19 lines
5. mobile/capacitor.config.js — 11 lines
6. mobile/build-apk.sh — 28 lines
7. mobile/tools/IconGen.java — 65 lines

Part 4 — Android Client (Capacitor + Web Assets)

1. mobile/www/index.html — 199 lines
2. mobile/www/styles.css — 82 lines
3. mobile/www/app.js — 353 lines
4. mobile/www/manifest.webmanifest — 16 lines
5. mobile/android/app/src/main/java/in/skyjet/app/MainActivity.java — 5 lines
6. mobile/android/app/src/main/AndroidManifest.xml — 41 lines
7. mobile/android/app/src/main/assets/capacitor.config.json — 8 lines
8. mobile/android/app/src/main/res/layout/activity_main.xml — 12 lines
9. mobile/android/app/src/main/res/values/strings.xml — 7 lines
0. mobile/android/app/src/main/res/values/styles.xml — 22 lines
1. mobile/android/app/src/main/res/xml/config.xml — 6 lines
2. mobile/android/app/build.gradle — 54 lines
3. mobile/android/build.gradle — 29 lines
4. mobile/android/variables.gradle — 16 lines
5. mobile/android/settings.gradle — 5 lines
6. mobile/android/app/src/main/res/drawable/ic_launcher_background.xml — 170 lines

7. mobile/android/app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml — 5 lines
8. mobile/android/app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml — 5 lines
9. mobile/android/app/src/main/res/drawable-v24/ic_launcher_foreground.xml — 34 lines

Part 5 — Seed Data and Project Documentation

1. data/flights.json — 1 lines
2. data/users.json — 1 lines
3. README.md — 139 lines

Part 1 — Desktop Application and HTTP Server (Java 17)

#	src/main/java/com/airline/App.java · Java · 359 lines

```java
package com.airline;

import com.airline.model.*;
import com.airline.service.*;
import com.airline.store.DataStore;
import com.airline.util.*;
import com.sun.net.httpserver.*;
import java.io.IOException;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.*;
import java.util.concurrent.Executors;

/**
 * Airoplane Ticket Service — zero-dependency Java backend + static UI.
 * Run:  ./run.sh   (or)  java -cp target/classes com.airline.App
 * App:  http://localhost:8080
 */
public class App {
    private final DataStore db;
    private final AuthService auth;
    private final FlightService flights;
    private final BookingService bookings;

    public App(DataStore db) {
        this.db = db;
        this.auth = new AuthService(db);
        this.flights = new FlightService(db);
        this.bookings = new BookingService(db);
    }

    public static void main(String[] args) throws Exception {
        int port = 8080;
        boolean openBrowser = false;
        boolean ui = false;
        for (int i = 0; i < args.length; i++) {
            if (args[i].equals("--port") && i + 1 < args.length) port = Integer.parseInt(args[i + 1]);
            if (args[i].equals("--open-browser")) openBrowser = true;
            if (args[i].equals("--ui")) ui = true;
        }
        String portEnv = System.getenv("PORT");
        if (portEnv != null && !portEnv.isBlank()) { try { port = Integer.parseInt(portEnv.trim()); } catch (Exception ignored) {} }

        Path dataDir = resolveDataDir();
        // when running from jar/classic layout, keep data next to working dir
        DataStore db = new DataStore(dataDir);
        db.load();
        App app = new App(db);

        HttpServer server;
        try {
            server = HttpServer.create(new InetSocketAddress(port), 0);
        } catch (java.net.BindException e) {
            String msg = "Port " + port + " is already in use.\n\n"
                    + "Another copy of Airoplane Ticket Service may already be running.\n"
                    + "Stop it with:  lsof -ti tcp:" + port + " | xargs kill\n\n"
                    + "Then open:  http://localhost:" + port;
            System.err.println(msg);
            showError("Cannot start server", msg);
            return;
        }
        server.createContext("/api/", app::handleApi);
        server.createContext("/", app::handleStatic);
```

src/main/java/com/airline/App.java · Java · 359 lines

```
        server.setExecutor(Executors.newFixedThreadPool(8));
        server.start();
        System.out.println("✈  Airoplane Ticket Service running at http://localhost:" + port);
        System.out.println("   Demo login: demo@skyjet.in / demo123   |   Admin: admin@skyjet.in / admin123");
        String url = "http://localhost:" + port;

        boolean gui = !java.awt.GraphicsEnvironment.isHeadless();
        if (gui && (isBundled() || ui)) {
            // native desktop window (this is the app — no browser needed)
            com.airline.ui.DesktopLauncher.launch(db, app.auth, app.flights, app.bookings, url);
        } else if (openBrowser) {
            openInBrowser(port);
        }
    }

    /** Shows a blocking error dialog (desktop app) or prints to stderr. */
    private static void showError(String title, String message) {
        if (java.awt.GraphicsEnvironment.isHeadless()) return;
        try {
            javax.swing.SwingUtilities.invokeLater(() -> javax.swing.JOptionPane.showMessageDialog(null, message, title, javax.swing.JOptionPane.ERROR_MESSAGE));
            Thread.sleep(60000);
        } catch (Exception ignored) {}
    }

    /** True when running from a jpackage .app bundle (bundles open the browser automatically). */
    private static boolean isBundled() {
        Path p = Paths.get(System.getProperty("java.home", ""));
        for (int i = 0; i < 6 && p != null; i++) {
            if (p.getFileName() != null && "Contents".equals(p.getFileName().toString())) {
                if (Files.isDirectory(p.resolve("app"))) return true;
            }
            p = p.getParent();
        }
        return false;
    }

    /** Data dir: SKYJET_DATA env, else inside a .app bundle, else ./data */
    private static Path resolveDataDir() {
        String env = System.getenv("SKYJET_DATA");
        if (env != null && !env.isBlank()) return Paths.get(env);
        Path p = Paths.get(System.getProperty("java.home", ""));
        for (int i = 0; i < 6 && p != null; i++) {
            if (p.getFileName() != null && "Contents".equals(p.getFileName().toString())) {
                Path appDir = p.resolve("app");
                if (Files.isDirectory(appDir)) return appDir.resolve("data");
            }
            p = p.getParent();
        }
        return Paths.get("data");
    }

    /** Opens the UI in the default browser (used by the desktop app). */
    private static void openInBrowser(int port) {
        Thread t = new Thread(() -> {
            try { Thread.sleep(1200); } catch (InterruptedException ignored) { return; }
            String url = "http://localhost:" + port;
            String os = System.getProperty("os.name", "").toLowerCase();
            try {
                if (os.contains("mac")) new ProcessBuilder("open", url).start();
                else if (os.contains("win")) new ProcessBuilder("rundll32", "url.dll,FileProtocolHandler", url).start();
                else new ProcessBuilder("xdg-open", url).start();
            } catch (Exception ignored) {}
        });
        t.setDaemon(true);
        t.start();
    }

```

src/main/java/com/airline/App.java · Java · 359 lines

```java
    // ---------------- static files ----------------
    private void handleStatic(HttpExchange ex) throws IOException {
        try {
            if (!ex.getRequestMethod().equalsIgnoreCase("GET")) { send(ex, 405, "Method not allowed", "text/plain"); return; }
            String path = ex.getRequestURI().getPath();
            if (path.equals("/")) path = "/index.html";
            String res = "static" + path;
            byte[] bytes = loadStatic(res);
            if (bytes == null) { // SPA fallback
                bytes = loadStatic("static/index.html");
                if (bytes == null) { send(ex, 404, "Not found", "text/plain"); return; }
                path = "/index.html";
            }
            String ct = "application/octet-stream";
            if (path.endsWith(".html")) ct = "text/html";
            else if (path.endsWith(".css")) ct = "text/css";
            else if (path.endsWith(".js")) ct = "text/javascript";
            else if (path.endsWith(".json")) ct = "application/json";
            else if (path.endsWith(".webmanifest")) ct = "application/manifest+json";
            else if (path.endsWith(".png")) ct = "image/png";
            else if (path.endsWith(".svg")) ct = "image/svg+xml";
            ex.getResponseHeaders().set("Content-Type", ct + "; charset=utf-8");
            ex.sendResponseHeaders(200, bytes.length);
            try (var os = ex.getResponseBody()) { os.write(bytes); }
        } catch (Exception e) {
            send(ex, 500, "Server error: " + e.getMessage(), "text/plain");
        }
    }

    private byte[] loadStatic(String res) {
        try {
            // 1) classpath (mvn / jar)
            try (var in = App.class.getClassLoader().getResourceAsStream(res)) {
                if (in != null) return in.readAllBytes();
            }
            // 2) file system (javac run.sh layout)
            Path p = Paths.get("src/main/resources/" + res);
            if (Files.exists(p)) return Files.readAllBytes(p);
        } catch (Exception ignored) {}
        return null;
    }

    // ---------------- API ----------------
    private void handleApi(HttpExchange ex) throws IOException {
        try {
            String method = ex.getRequestMethod().toUpperCase();
            String path = ex.getRequestURI().getPath();
            Map<String, String> q = HttpUtil.query(ex.getRequestURI().getRawQuery());

            // CORS preflight
            if (method.equals("OPTIONS")) {
                ex.getResponseHeaders().set("Access-Control-Allow-Origin", "*");
                ex.getResponseHeaders().set("Access-Control-Allow-Methods", "GET,POST,PUT,DELETE,OPTIONS");
                ex.getResponseHeaders().set("Access-Control-Allow-Headers", "Content-Type, Authorization");
                ex.sendResponseHeaders(204, -1);
                return;
            }

            // auth
            if (path.equals("/api/auth/register") && method.equals("POST")) {
                Map<String, Object> b = Json.parseObject(HttpUtil.readBody(ex));
                AuthService.AuthResult r = auth.register(Json.str(b, "name", ""), Json.str(b, "email", ""), Json.str(b, "password", ""));
                if (r.error != null) { json(ex, 400, err(r.error)); return; }
                json(ex, 200, ok(Map.of("user", r.user.publicJson(), "token", r.token))); return;
            }
            if (path.equals("/api/auth/login") && method.equals("POST")) {
```

#	src/main/java/com/airline/App.java · Java · 359 lines

```
198                 Map<String, Object> b = Json.parseObject(HttpUtil.readBody(ex));
199                 AuthService.AuthResult r = auth.login(Json.str(b, "email", ""), Json.str(b, "passwor
    d", ""));
200                 if (r.error != null) { json(ex, 401, err(r.error)); return; }
201                 json(ex, 200, ok(Map.of("user", r.user.publicJson(), "token", r.token))); return;
202             }
203             if (path.equals("/api/auth/me") && method.equals("GET")) {
204                 User u = auth.me(HttpUtil.bearer(ex));
205                 if (u == null) { json(ex, 401, err("Not logged in.")); return; }
206                 json(ex, 200, ok(Map.of("user", u.publicJson()))); return;
207             }
208 
209             // flights search
210             if (path.equals("/api/flights") && method.equals("GET")) {
211                 List<Object> list = new ArrayList<>();
212                 for (Flight f : flights.search(q.get("from"), q.get("to"), q.get("date"))) list.add
    (f.toJson());
213                 Map<String, Object> airports = new LinkedHashMap<>();
214                 airports.put("DEL", "New Delhi (DEL)"); airports.put("BOM", "Mumbai (BOM)");
215                 airports.put("BLR", "Bengaluru (BLR)"); airports.put("HYD", "Hyderabad (HYD)");
216                 airports.put("MAA", "Chennai (MAA)");
217                 json(ex, 200, ok(Map.of("flights", list, "airports", airports))); return;
218             }
219             if (path.equals("/api/flights") && method.equals("POST")) {
220                 User u = requireAdmin(ex); if (u == null) return;
221                 Flight f = flights.create(Json.parseObject(HttpUtil.readBody(ex)));
222                 json(ex, 201, ok(Map.of("flight", f.toJson()))); return;
223             }
224             if (path.startsWith("/api/flights/")) {
225                 String id = path.substring("/api/flights/".length());
226                 if (method.equals("GET")) {
227                     Flight f = flights.get(id);
228                     if (f == null) { json(ex, 404, err("Flight not found.")); return; }
229                     json(ex, 200, ok(Map.of("flight", f.toJson()))); return;
230                 }
231                 if (method.equals("PUT")) {
232                     User u = requireAdmin(ex); if (u == null) return;
233                     Flight f = flights.update(id, Json.parseObject(HttpUtil.readBody(ex)));
234                     if (f == null) { json(ex, 404, err("Flight not found.")); return; }
235                     json(ex, 200, ok(Map.of("flight", f.toJson()))); return;
236                 }
237                 if (method.equals("DELETE")) {
238                     User u = requireAdmin(ex); if (u == null) return;
239                     boolean okDel = flights.delete(id);
240                     if (!okDel) { json(ex, 400, err("Cannot delete: flight has active bookings or do
    es not exist.")); return; }
241                     json(ex, 200, ok(Map.of("deleted", true))); return;
242                 }
243             }
244 
245             // mock payment
246             if (path.equals("/api/payments/mock") && method.equals("POST")) {
247                 Map<String, Object> b = Json.parseObject(HttpUtil.readBody(ex));
248                 String card = Json.str(b, "cardNumber", "").replaceAll("\\s", "");
249                 if (!card.matches("\\d{12,19}")) { json(ex, 400, err("Enter a 12–19 digit test card
    number (e.g. 4111 1111 1111 1111).")); return; }
250                 double amount = Json.dbl(b, "amount", 0);
251                 Map<String, Object> pay = bookings.mockPay(amount, card.substring(card.length() -
    4));
252                 json(ex, 200, ok(Map.of("payment", pay))); return;
253             }
254 
255             // bookings
256             if (path.equals("/api/bookings") && method.equals("POST")) {
257                 User u = requireLogin(ex); if (u == null) return;
258                 Map<String, Object> b = Json.parseObject(HttpUtil.readBody(ex));
259                 String flightId = Json.str(b, "flightId", "");
260                 List<Map<String, Object>> pax = new ArrayList<>();
261                 Object po = b.get("passengers");
```

src/main/java/com/airline/App.java · Java · 359 lines

```
                if (po instanceof List) for (Object o : (List<Object>) po) if (o instanceof Map) pa
x.add((Map<String, Object>) o);
                List<String> seats = new ArrayList<>();
                Object so = b.get("seats");
                if (so instanceof List) for (Object o : (List<Object>) so) seats.add(String.valueOf
(o));
                BookingService.Result r = bookings.create(u.id, flightId, pax, seats, Json.str(b, "p
aymentId", ""), Json.dbl(b, "paidAmount", 0));
                if (r.error != null) { json(ex, 400, err(r.error)); return; }
                json(ex, 201, ok(Map.of("booking", enrich(r.booking)))); return;
            }
            if (path.equals("/api/bookings") && method.equals("GET")) {
                User u = requireLogin(ex); if (u == null) return;
                List<Object> list = new ArrayList<>();
                List<Booking> src = u.role.equals("admin") ? bookings.all() : bookings.forUser(u.i
d);
                for (Booking bl : src) list.add(enrich(bl));
                json(ex, 200, ok(Map.of("bookings", list))); return;
            }
            if (path.startsWith("/api/bookings/") && path.endsWith("/confirm") && method.equals("POS
T")) {
                User u = requireAdmin(ex); if (u == null) return;
                String pnr = path.substring("/api/bookings/".length(), path.length() - "/confirm".le
ngth());
                BookingService.Result r = bookings.confirm(pnr, true);
                if (r.error != null) { json(ex, 400, err(r.error)); return; }
                json(ex, 200, ok(Map.of("booking", enrich(r.booking)))); return;
            }
            if (path.startsWith("/api/bookings/") && path.endsWith("/cancel") && method.equals("POS
T")) {
                User u = requireLogin(ex); if (u == null) return;
                String pnr = path.substring("/api/bookings/".length(), path.length() - "/cancel".len
gth());
                BookingService.Result r = bookings.cancel(pnr, u.id, u.role.equals("admin"));
                if (r.error != null) { json(ex, 400, err(r.error)); return; }
                json(ex, 200, ok(Map.of("booking", enrich(r.booking)))); return;
            }
            if (path.startsWith("/api/bookings/") && method.equals("GET")) {
                User u = requireLogin(ex); if (u == null) return;
                String pnr = path.substring("/api/bookings/".length());
                Booking bl = db.bookings.get(pnr);
                if (bl == null) { json(ex, 404, err("Booking not found.")); return; }
                if (!u.role.equals("admin") && !bl.userId.equals(u.id)) { json(ex, 403, err("Not you
r booking.")); return; }
                json(ex, 200, ok(Map.of("booking", enrich(bl)))); return;
            }

            // admin stats
            if (path.equals("/api/admin/stats") && method.equals("GET")) {
                User u = requireAdmin(ex); if (u == null) return;
                double revenue = 0; int active = 0;
                for (Booking bl : db.bookings.values()) if (bl.status.equals("CONFIRMED")) { revenue
+= bl.totalPrice; active++; }
                Map<String, Object> s = new LinkedHashMap<>();
                s.put("flights", db.flights.size()); s.put("users", db.users.size());
                s.put("bookings", db.bookings.size()); s.put("activeBookings", active);
                s.put("pendingBookings", bookings.pendingCount());
                s.put("revenue", revenue);
                json(ex, 200, ok(Map.of("stats", s))); return;
            }

            json(ex, 404, err("Unknown endpoint: " + path));
        } catch (Exception e) {
            json(ex, 500, err("Server error: " + e.getMessage()));
        }
    }

    private Map<String, Object> enrich(Booking b) {
        Map<String, Object> m = new LinkedHashMap<>(b.toJson());
        Flight f = db.flights.get(b.flightId);
```

src/main/java/com/airline/App.java · Java · 359 lines

```
322         if (f != null) m.put("flight", f.toJson());
323         User u = db.users.get(b.userId);
324         if (u != null) m.put("customer", u.publicJson());
325         return m;
326     }
327 
328     private User requireLogin(HttpExchange ex) throws IOException {
329         User u = auth.me(HttpUtil.bearer(ex));
330         if (u == null) json(ex, 401, err("Login required."));
331         return u;
332     }
333 
334     private User requireAdmin(HttpExchange ex) throws IOException {
335         User u = auth.me(HttpUtil.bearer(ex));
336         if (u == null) { json(ex, 401, err("Login required.")); return null; }
337         if (!u.role.equals("admin")) { json(ex, 403, err("Admin only.")); return null; }
338         return u;
339     }
340 
341     private static Map<String, Object> ok(Map<String, Object> d) {
342         Map<String, Object> m = new LinkedHashMap<>();
343         m.put("ok", true); m.putAll(d);
344         return m;
345     }
346 
347     private static Map<String, Object> err(String msg) {
348         return Map.of("ok", false, "error", msg);
349     }
350 
351     private void json(HttpExchange ex, int status, Map<String, Object> m) throws IOException {
352         HttpUtil.sendJson(ex, status, Json.stringify(m));
353     }
354 
355     private void send(HttpExchange ex, int status, String text, String ct) throws IOException {
356         HttpUtil.sendText(ex, status, text, ct);
357     }
358 }
359 
```

src/main/java/com/airline/model/Flight.java · Java · 66 lines

```
1  package com.airline.model;
2  
3  import java.util.*;
4  
5  /** Flight entity. */
6  public class Flight {
7      public String id;
8      public String airline;
9      public String flightNumber;
10     public String from;
11     public String to;
12     public String date;        // yyyy-MM-dd
13     public String departure;   // HH:mm
14     public String arrival;     // HH:mm
15     public int durationMin;
16     public double price;       // INR per adult
17     public int totalSeats;
18     public Set<String> bookedSeats = new LinkedHashSet<>();
19 
20     public Flight() {}
21 
22     public Flight(String id, String airline, String flightNumber, String from, String to,
23                   String date, String departure, String arrival, int durationMin,
24                   double price, int totalSeats) {
25         this.id = id; this.airline = airline; this.flightNumber = flightNumber;
26         this.from = from; this.to = to; this.date = date;
27         this.departure = departure; this.arrival = arrival;
28         this.durationMin = durationMin; this.price = price; this.totalSeats = totalSeats;
29     }
```

src/main/java/com/airline/model/Flight.java · Java · 66 lines

```
30
31      public int availableSeats() { return Math.max(0, totalSeats - bookedSeats.size()); }
32
33      public Map<String, Object> toJson() {
34          Map<String, Object> m = new LinkedHashMap<>();
35          m.put("id", id); m.put("airline", airline); m.put("flightNumber", flightNumber);
36          m.put("from", from); m.put("to", to); m.put("date", date);
37          m.put("departure", departure); m.put("arrival", arrival);
38          m.put("durationMin", durationMin); m.put("price", price);
39          m.put("totalSeats", totalSeats); m.put("availableSeats", availableSeats());
40          m.put("bookedSeats", new ArrayList<>(bookedSeats));
41          return m;
42      }
43
44      @SuppressWarnings("unchecked")
45      public static Flight fromJson(Map<String, Object> m) {
46          Flight f = new Flight();
47          f.id = String.valueOf(m.getOrDefault("id", UUID.randomUUID().toString()));
48          f.airline = String.valueOf(m.getOrDefault("airline", ""));
49          f.flightNumber = String.valueOf(m.getOrDefault("flightNumber", ""));
50          f.from = String.valueOf(m.getOrDefault("from", "")).toUpperCase();
51          f.to = String.valueOf(m.getOrDefault("to", "")).toUpperCase();
52          f.date = String.valueOf(m.getOrDefault("date", ""));
53          f.departure = String.valueOf(m.getOrDefault("departure", ""));
54          f.arrival = String.valueOf(m.getOrDefault("arrival", ""));
55          Object d = m.get("durationMin");
56          f.durationMin = d instanceof Number ? ((Number) d).intValue() : 0;
57          Object p = m.get("price");
58          f.price = p instanceof Number ? ((Number) p).doubleValue() : 0;
59          Object t = m.get("totalSeats");
60          f.totalSeats = t instanceof Number ? ((Number) t).intValue() : 180;
61          Object b = m.get("bookedSeats");
62          if (b instanceof List) for (Object s : (List<Object>) b) f.bookedSeats.add(String.valueOf
    (s));
63          return f;
64      }
65  }
66
```

src/main/java/com/airline/model/User.java · Java · 33 lines

```
1   package com.airline.model;
2
3   import java.util.*;
4
5   /** User entity (role = admin | customer). */
6   public class User {
7       public String id;
8       public String name;
9       public String email;
10      public String passwordHash;
11      public String role = "customer";
12
13      public Map<String, Object> toJson(boolean includeHash) {
14          Map<String, Object> m = new LinkedHashMap<>();
15          m.put("id", id); m.put("name", name); m.put("email", email); m.put("role", role);
16          if (includeHash) m.put("passwordHash", passwordHash);
17          return m;
18      }
19
20      public Map<String, Object> publicJson() { return toJson(false); }
21
22      @SuppressWarnings("unchecked")
23      public static User fromJson(Map<String, Object> m) {
24          User u = new User();
25          u.id = String.valueOf(m.getOrDefault("id", UUID.randomUUID().toString()));
26          u.name = String.valueOf(m.getOrDefault("name", ""));
27          u.email = String.valueOf(m.getOrDefault("email", "")).toLowerCase().trim();
28          u.passwordHash = String.valueOf(m.getOrDefault("passwordHash", ""));
29          u.role = String.valueOf(m.getOrDefault("role", "customer"));
```

src/main/java/com/airline/model/User.java · Java · 33 lines

```
        return u;
    }
}

```

src/main/java/com/airline/model/Booking.java · Java · 49 lines

```java
package com.airline.model;

import java.util.*;

/** Booking / e-ticket entity. PNR is the public ticket id. */
public class Booking {
    public String pnr;
    public String userId;
    public String flightId;
    public List<Map<String, Object>> passengers = new ArrayList<>();
    public List<String> seats = new ArrayList<>();
    public double totalPrice;
    public String status = "PENDING"; // PENDING | CONFIRMED | CANCELLED
    public String paymentId;
    public String createdAt;
    public String confirmedAt = "";

    public Map<String, Object> toJson() {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("pnr", pnr); m.put("userId", userId); m.put("flightId", flightId);
        m.put("passengers", passengers); m.put("seats", seats);
        m.put("totalPrice", totalPrice); m.put("status", status);
        m.put("paymentId", paymentId); m.put("createdAt", createdAt);
        m.put("confirmedAt", confirmedAt);
        return m;
    }

    @SuppressWarnings("unchecked")
    public static Booking fromJson(Map<String, Object> m) {
        Booking b = new Booking();
        b.pnr = String.valueOf(m.getOrDefault("pnr", ""));
        b.userId = String.valueOf(m.getOrDefault("userId", ""));
        b.flightId = String.valueOf(m.getOrDefault("flightId", ""));
        Object p = m.get("passengers");
        if (p instanceof List) for (Object o : (List<Object>) p) {
            if (o instanceof Map) b.passengers.add((Map<String, Object>) o);
        }
        Object s = m.get("seats");
        if (s instanceof List) for (Object o : (List<Object>) s) b.seats.add(String.valueOf(o));
        Object t = m.get("totalPrice");
        b.totalPrice = t instanceof Number ? ((Number) t).doubleValue() : 0;
        b.status = String.valueOf(m.getOrDefault("status", "CONFIRMED"));
        b.paymentId = String.valueOf(m.getOrDefault("paymentId", ""));
        b.createdAt = String.valueOf(m.getOrDefault("createdAt", ""));
        b.confirmedAt = String.valueOf(m.getOrDefault("confirmedAt", ""));
        return b;
    }
}

```

src/main/java/com/airline/service/AuthService.java · Java · 59 lines

```java
package com.airline.service;

import com.airline.model.User;
import com.airline.store.DataStore;
import com.airline.util.Passwords;
import java.util.UUID;

/** Register / login / session lookup. */
public class AuthService {
    private final DataStore db;
    public AuthService(DataStore db) { this.db = db; }
```

src/main/java/com/airline/service/AuthService.java · Java · 59 lines

```java

    public static class AuthResult {
        public User user; public String token; public String error;
    }

    public AuthResult register(String name, String email, String password) {
        AuthResult r = new AuthResult();
        if (name == null || name.isBlank() || email == null || !email.contains("@") || password == null || password.length() < 4) {
            r.error = "Name, valid email and 4+ char password required."; return r;
        }
        synchronized (db) {
            if (db.findUserByEmail(email) != null) { r.error = "Email already registered."; return r; }
            User u = new User();
            u.id = "u-" + UUID.randomUUID().toString().substring(0, 8);
            u.name = name.trim(); u.email = email.toLowerCase().trim();
            u.passwordHash = Passwords.hash(password); u.role = "customer";
            db.users.put(u.id, u);
            db.saveQuietly();
            r.user = u; r.token = issueToken(u);
        }
        return r;
    }

    public AuthResult login(String email, String password) {
        AuthResult r = new AuthResult();
        User u = db.findUserByEmail(email == null ? "" : email.trim());
        if (u == null || !Passwords.verify(password == null ? "" : password, u.passwordHash)) {
            r.error = "Invalid email or password. Try demo@skyjet.in / demo123"; return r;
        }
        r.user = u; r.token = issueToken(u);
        return r;
    }

    private String issueToken(User u) {
        String t = UUID.randomUUID().toString();
        db.sessions.put(t, u.id);
        return t;
    }

    public User me(String token) {
        if (token == null || token.isEmpty()) return null;
        String uid = db.sessions.get(token);
        return uid == null ? null : db.users.get(uid);
    }

    public void logout(String token) { if (token != null) db.sessions.remove(token); }
}

```

src/main/java/com/airline/service/FlightService.java · Java · 59 lines

```java
package com.airline.service;

import com.airline.model.*;
import com.airline.store.DataStore;
import java.time.LocalDateTime;
import java.util.*;

/** Flight search + admin CRUD. */
public class FlightService {
    private final DataStore db;
    public FlightService(DataStore db) { this.db = db; }

    public List<Flight> search(String from, String to, String date) {
        List<Flight> out = new ArrayList<>();
        for (Flight f : db.flights.values()) {
            if (from != null && !from.isBlank() && !f.from.equalsIgnoreCase(from.trim())) continue;
            if (to != null && !to.isBlank() && !f.to.equalsIgnoreCase(to.trim())) continue;
```

src/main/java/com/airline/service/FlightService.java · Java · 59 lines

```
            if (date != null && !date.isBlank() && !f.date.equals(date.trim())) continue;
            out.add(f);
        }
        out.sort(Comparator.comparing((Flight f) -> f.date).thenComparing(f -> f.departure));
        return out;
    }

    public Flight get(String id) { return db.flights.get(id); }

    public Flight create(Map<String, Object> m) {
        Flight f = Flight.fromJson(m);
        if (f.id == null || f.id.isBlank()) f.id = "f-" + UUID.randomUUID().toString().substring(0,
8);
        synchronized (db) { db.flights.put(f.id, f); db.saveQuietly(); }
        return f;
    }

    public Flight update(String id, Map<String, Object> m) {
        synchronized (db) {
            Flight f = db.flights.get(id);
            if (f == null) return null;
            Set<String> keepSeats = f.bookedSeats;
            Flight n = Flight.fromJson(m);
            n.id = id; n.bookedSeats = keepSeats;
            db.flights.put(id, n); db.saveQuietly();
            return n;
        }
    }

    public boolean delete(String id) {
        synchronized (db) {
            // refuse delete if active bookings exist
            for (Booking b : db.bookings.values()) {
                if (b.flightId.equals(id) && b.status.equals("CONFIRMED")) return false;
            }
            db.saveQuietly();
            return db.flights.remove(id) != null;
        }
    }

    public static String now() { return LocalDateTime.now().toString(); }
}

```

src/main/java/com/airline/service/BookingService.java · Java · 119 lines

```
package com.airline.service;

import com.airline.model.*;
import com.airline.store.DataStore;
import java.time.LocalDateTime;
import java.util.*;

/** Booking creation / cancellation / queries + mock payments. */
public class BookingService {
    private final DataStore db;
    public BookingService(DataStore db) { this.db = db; }

    public static class Result {
        public Booking booking; public String error;
    }

    /** Mock payment: accepts any 12-19 digit card, always approves. */
    public Map<String, Object> mockPay(double amount, String cardLast4) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("paymentId", "PAY-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase());
        m.put("amount", amount);
        m.put("status", "SUCCESS");
        m.put("cardLast4", cardLast4);
        m.put("at", LocalDateTime.now().toString());
```

```java
        return m;
    }

    public Result create(String userId, String flightId, List<Map<String, Object>> passengers,
                         List<String> seats, String paymentId, double paidAmount) {
        Result r = new Result();
        synchronized (db) {
            Flight f = db.flights.get(flightId);
            if (f == null) { r.error = "Flight not found."; return r; }
            if (passengers == null || passengers.isEmpty()) { r.error = "Add at least one passenger."; return r; }
            if (seats == null || seats.size() != passengers.size()) { r.error = "Seats must match passengers."; return r; }
            for (String s : seats) {
                if (f.bookedSeats.contains(s)) { r.error = "Seat " + s + " just got booked. Pick another."; return r; }
            }
            double total = f.price * passengers.size();
            Booking b = new Booking();
            b.pnr = genPnr();
            b.userId = userId; b.flightId = flightId;
            b.passengers = passengers; b.seats = new ArrayList<>(seats);
            b.totalPrice = total; b.status = "PENDING"; // admin must confirm before the ticket is issued
            b.paymentId = paymentId == null ? "" : paymentId;
            b.createdAt = LocalDateTime.now().toString();
            f.bookedSeats.addAll(seats);
            db.bookings.put(b.pnr, b);
            db.saveQuietly();
            r.booking = b;
        }
        return r;
    }

    /** Admin approves a pending booking and issues the ticket. */
    public Result confirm(String pnr, boolean isAdmin) {
        Result r = new Result();
        synchronized (db) {
            Booking b = db.bookings.get(pnr);
            if (b == null) { r.error = "Booking not found."; return r; }
            if (!isAdmin) { r.error = "Only an admin can confirm a booking."; return r; }
            if (b.status.equals("CONFIRMED")) { r.error = "Booking " + pnr + " is already confirmed."; return r; }
            if (b.status.equals("CANCELLED")) { r.error = "Booking " + pnr + " was cancelled."; return r; }
            b.status = "CONFIRMED";
            b.confirmedAt = LocalDateTime.now().toString();
            db.saveQuietly();
            r.booking = b;
        }
        return r;
    }

    public int pendingCount() {
        int n = 0;
        for (Booking b : db.bookings.values()) if (b.status.equals("PENDING")) n++;
        return n;
    }

    public Result cancel(String pnr, String requesterId, boolean isAdmin) {
        Result r = new Result();
        synchronized (db) {
            Booking b = db.bookings.get(pnr);
            if (b == null) { r.error = "Booking not found."; return r; }
            if (!isAdmin && !b.userId.equals(requesterId)) { r.error = "Not your booking."; return r; }
            if (b.status.equals("CANCELLED")) { r.error = "Already cancelled."; return r; }
            b.status = "CANCELLED";
            Flight f = db.flights.get(b.flightId);
            if (f != null) f.bookedSeats.removeAll(b.seats);
```

src/main/java/com/airline/service/BookingService.java · Java · 119 lines

```
            db.saveQuietly();
            r.booking = b;
        }
        return r;
    }

    public List<Booking> forUser(String userId) {
        List<Booking> out = new ArrayList<>();
        for (Booking b : db.bookings.values()) if (b.userId.equals(userId)) out.add(b);
        out.sort(Comparator.comparing((Booking b) -> b.createdAt).reversed());
        return out;
    }

    public List<Booking> all() {
        List<Booking> out = new ArrayList<>(db.bookings.values());
        out.sort(Comparator.comparing((Booking b) -> b.createdAt).reversed());
        return out;
    }

    private String genPnr() {
        String chars = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
        Random r = new Random();
        String pnr;
        do {
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < 6; i++) sb.append(chars.charAt(r.nextInt(chars.length())));
            pnr = sb.toString();
        } while (db.bookings.containsKey(pnr));
        return pnr;
    }
}

```

src/main/java/com/airline/store/DataStore.java · Java · 174 lines

```
package com.airline.store;

import com.airline.model.*;
import com.airline.util.*;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

/**
 * File-backed data store (data/*.json). Thread-safe via synchronized blocks.
 * Seeds demo Indian domestic flights + admin/customer users on first run.
 */
public class DataStore {
    private final Path dataDir;
    public final Map<String, Flight> flights = new ConcurrentHashMap<>();
    public final Map<String, User> users = new ConcurrentHashMap<>();
    public final Map<String, Booking> bookings = new ConcurrentHashMap<>();
    /** token -> userId */
    public final Map<String, String> sessions = new ConcurrentHashMap<>();

    public DataStore(Path dataDir) {
        this.dataDir = dataDir;
    }

    public synchronized void load() throws IOException {
        Files.createDirectories(dataDir);
        loadFlights(); loadUsers(); loadBookings();
        if (flights.isEmpty()) seedFlights();
        if (users.isEmpty()) seedUsers();
        rollFlightDatesForward();
        saveAll();
    }

```

src/main/java/com/airline/store/DataStore.java · Java · 174 lines

```
36      // ---------- flights ----------
37      private void loadFlights() throws IOException {
38          Path f = dataDir.resolve("flights.json");
39          if (!Files.exists(f)) return;
40          String s = Files.readString(f, StandardCharsets.UTF_8).trim();
41          if (s.isEmpty()) return;
42          for (Object o : Json.parseArray(s)) {
43              if (o instanceof Map) {
44                  Flight fl = Flight.fromJson((Map<String, Object>) o);
45                  flights.put(fl.id, fl);
46              }
47          }
48      }
49
50      // ---------- users ----------
51      private void loadUsers() throws IOException {
52          Path f = dataDir.resolve("users.json");
53          if (!Files.exists(f)) return;
54          String s = Files.readString(f, StandardCharsets.UTF_8).trim();
55          if (s.isEmpty()) return;
56          for (Object o : Json.parseArray(s)) {
57              if (o instanceof Map) {
58                  User u = User.fromJson((Map<String, Object>) o);
59                  users.put(u.id, u);
60              }
61          }
62      }
63
64      // ---------- bookings ----------
65      private void loadBookings() throws IOException {
66          Path f = dataDir.resolve("bookings.json");
67          if (!Files.exists(f)) return;
68          String s = Files.readString(f, StandardCharsets.UTF_8).trim();
69          if (s.isEmpty()) return;
70          for (Object o : Json.parseArray(s)) {
71              if (o instanceof Map) {
72                  Booking b = Booking.fromJson((Map<String, Object>) o);
73                  bookings.put(b.pnr, b);
74              }
75          }
76      }
77
78      public synchronized void saveAll() throws IOException {
79          List<Object> fl = new ArrayList<>();
80          for (Flight x : flights.values()) fl.add(x.toJson());
81          Files.writeString(dataDir.resolve("flights.json"), Json.stringify(fl), StandardCharsets.UTF_
    8);
82          List<Object> ul = new ArrayList<>();
83          for (User u : users.values()) ul.add(u.toJson(true));
84          Files.writeString(dataDir.resolve("users.json"), Json.stringify(ul), StandardCharsets.UTF_
    8);
85          List<Object> bl = new ArrayList<>();
86          for (Booking b : bookings.values()) bl.add(b.toJson());
87          Files.writeString(dataDir.resolve("bookings.json"), Json.stringify(bl), StandardCharsets.UTF
    _8);
88      }
89
90      public void saveQuietly() {
91          try { saveAll(); } catch (Exception ignored) {}
92      }
93
94      // ---------- seed ----------
95      private void seedUsers() {
96          User admin = new User();
97          admin.id = "u-admin"; admin.name = "Admin";
98          admin.email = "admin@skyjet.in";
99          admin.passwordHash = Passwords.hash("admin123");
100         admin.role = "admin";
101         users.put(admin.id, admin);
102
```

src/main/java/com/airline/store/DataStore.java · Java · 174 lines

```
103         User demo = new User();
104         demo.id = "u-demo"; demo.name = "Demo Flyer";
105         demo.email = "demo@skyjet.in";
106         demo.passwordHash = Passwords.hash("demo123");
107         demo.role = "customer";
108         users.put(demo.id, demo);
109     }
110
111     private void seedFlights() {
112         // date rolling: use a fixed demo date + today-ish dates so search always hits
113         String d1 = demoDate(1), d2 = demoDate(2), d3 = demoDate(3);
114         add(new Flight("f1", "SkyJet Airways", "SJ-101", "DEL", "BOM", d1, "06:00", "08:10", 130, 5499, 180));
115         add(new Flight("f2", "SkyJet Airways", "SJ-204", "DEL", "BOM", d1, "14:30", "16:45", 135, 6299, 180));
116         add(new Flight("f3", "Air Himalaya", "AH-312", "DEL", "BLR", d1, "09:15", "12:00", 165, 6999, 180));
117         add(new Flight("f4", "Deccan Wings", "DW-418", "BOM", "BLR", d2, "07:45", "09:20", 95, 4999, 150));
118         add(new Flight("f5", "SkyJet Airways", "SJ-520", "DEL", "HYD", d2, "11:00", "13:10", 130, 5799, 180));
119         add(new Flight("f6", "Air Himalaya", "AH-611", "BLR", "DEL", d2, "18:00", "20:45", 165, 7199, 180));
120         add(new Flight("f7", "Deccan Wings", "DW-702", "DEL", "MAA", d3, "08:30", "11:15", 165, 6499, 150));
121         add(new Flight("f8", "SkyJet Airways", "SJ-808", "BOM", "DEL", d3, "19:20", "21:30", 130, 5899, 180));
122         add(new Flight("f9", "Air Himalaya", "AH-909", "HYD", "BOM", d3, "10:05", "11:25", 80, 4299, 150));
123         add(new Flight("f10", "Deccan Wings", "DW-515", "BLR", "HYD", d1, "16:40", "17:50", 70, 3599, 120));
124         // pre-book a few random seats so seat maps look realistic
125         Random r = new Random(42);
126         for (Flight f : flights.values()) {
127             int n = 8 + r.nextInt(20);
128             for (int k = 0; k < n; k++) f.bookedSeats.add(randomSeat(r));
129         }
130     }
131
132     private void add(Flight f) { flights.put(f.id, f); }
133
134     /**
135      * Keep the demo searchable: shift all flight dates forward so the earliest
136      * flight is tomorrow. Flight IDs, seats and bookings stay intact.
137      */
138     private void rollFlightDatesForward() {
139         try {
140             java.time.LocalDate tomorrow = java.time.LocalDate.now().plusDays(1);
141             java.time.LocalDate earliest = null;
142             for (Flight f : flights.values()) {
143                 try {
144                     java.time.LocalDate d = java.time.LocalDate.parse(f.date);
145                     if (earliest == null || d.isBefore(earliest)) earliest = d;
146                 } catch (Exception ignored) {}
147             }
148             if (earliest == null || !earliest.isBefore(tomorrow)) return;
149             long shift = java.time.temporal.ChronoUnit.DAYS.between(earliest, tomorrow);
150             for (Flight f : flights.values()) {
151                 try {
152                     f.date = java.time.LocalDate.parse(f.date).plusDays(shift).toString();
153                 } catch (Exception ignored) {}
154             }
155         } catch (Exception ignored) {}
156     }
157
158     private static String demoDate(int plusDays) {
159         java.time.LocalDate d = java.time.LocalDate.now().plusDays(plusDays);
160         return d.toString();
161     }
162
```

src/main/java/com/airline/store/DataStore.java · Java · 174 lines

```
163     private static String randomSeat(Random r) {
164         int row = 1 + r.nextInt(30);
165         char col = "ABCDEF".charAt(r.nextInt(6));
166         return row + String.valueOf(col);
167     }
168 
169     public User findUserByEmail(String email) {
170         for (User u : users.values()) if (u.email.equalsIgnoreCase(email)) return u;
171         return null;
172     }
173 }
174 
```

src/main/java/com/airline/util/Json.java · Java · 182 lines

```
1   package com.airline.util;
2   
3   import java.util.*;
4   
5   /**
6    * Minimal JSON parser + serializer (no external deps).
7    * Supports objects, arrays, strings, numbers, booleans, null.
8    */
9   public final class Json {
10      private Json() {}
11  
12      @SuppressWarnings("unchecked")
13      public static Map<String, Object> parseObject(String s) {
14          Object v = new Parser(s).parseValue();
15          if (v instanceof Map) return (Map<String, Object>) v;
16          return new LinkedHashMap<>();
17      }
18  
19      public static List<Object> parseArray(String s) {
20          Object v = new Parser(s).parseValue();
21          if (v instanceof List) return (List<Object>) v;
22          return new ArrayList<>();
23      }
24  
25      public static Object parse(String s) {
26          return new Parser(s).parseValue();
27      }
28  
29      public static String stringify(Object v) {
30          if (v == null) return "null";
31          if (v instanceof String) return quote((String) v);
32          if (v instanceof Number || v instanceof Boolean) return v.toString();
33          if (v instanceof Map) {
34              StringBuilder sb = new StringBuilder("{");
35              boolean first = true;
36              for (Map.Entry<?, ?> e : ((Map<?, ?>) v).entrySet()) {
37                  if (!first) sb.append(",");
38                  sb.append(quote(String.valueOf(e.getKey()))).append(":").append(stringify(e.getValue
    ()));
39                  first = false;
40              }
41              return sb.append("}").toString();
42          }
43          if (v instanceof Collection) {
44              StringBuilder sb = new StringBuilder("[");
45              boolean first = true;
46              for (Object o : (Collection<?>) v) {
47                  if (!first) sb.append(",");
48                  sb.append(stringify(o));
49                  first = false;
50              }
51              return sb.append("]").toString();
52          }
53          return quote(String.valueOf(v));
54      }
```

src/main/java/com/airline/util/Json.java · Java · 182 lines

```java

    public static String quote(String s) {
        StringBuilder sb = new StringBuilder("\"");
        for (char c : s.toCharArray()) {
            switch (c) {
                case '"': sb.append("\\\""); break;
                case '\\': sb.append("\\\\"); break;
                case '\n': sb.append("\\n"); break;
                case '\r': sb.append("\\r"); break;
                case '\t': sb.append("\\t"); break;
                default:
                    if (c < 0x20) sb.append(String.format("\\u%04x", (int) c));
                    else sb.append(c);
            }
        }
        return sb.append("\"").toString();
    }

    public static String str(Map<String, Object> m, String key, String def) {
        Object v = m.get(key);
        return v == null ? def : String.valueOf(v);
    }

    public static int num(Map<String, Object> m, String key, int def) {
        try {
            Object v = m.get(key);
            if (v instanceof Number) return ((Number) v).intValue();
            if (v != null) return Integer.parseInt(String.valueOf(v));
        } catch (Exception ignored) {}
        return def;
    }

    public static double dbl(Map<String, Object> m, String key, double def) {
        try {
            Object v = m.get(key);
            if (v instanceof Number) return ((Number) v).doubleValue();
            if (v != null) return Double.parseDouble(String.valueOf(v));
        } catch (Exception ignored) {}
        return def;
    }

    private static final class Parser {
        private final String s;
        private int i;
        Parser(String s) { this.s = s == null ? "" : s.trim(); }
        private void ws() { while (i < s.length() && Character.isWhitespace(s.charAt(i))) i++; }
        Object parseValue() {
            ws();
            if (i >= s.length()) return null;
            char c = s.charAt(i);
            if (c == '{') return parseObj();
            if (c == '[') return parseArr();
            if (c == '"') return parseStr();
            if (c == 't' && s.startsWith("true", i)) { i += 4; return Boolean.TRUE; }
            if (c == 'f' && s.startsWith("false", i)) { i += 5; return Boolean.FALSE; }
            if (c == 'n' && s.startsWith("null", i)) { i += 4; return null; }
            return parseNum();
        }
        private Map<String, Object> parseObj() {
            Map<String, Object> m = new LinkedHashMap<>();
            i++; ws();
            if (i < s.length() && s.charAt(i) == '}') { i++; return m; }
            while (i < s.length()) {
                ws();
                String k = parseStr();
                ws();
                if (i < s.length() && s.charAt(i) == ':') i++;
                Object v = parseValue();
                m.put(k, v);
                ws();
```

src/main/java/com/airline/util/Json.java · Java · 182 lines

```java
                if (i < s.length() && s.charAt(i) == ',') { i++; continue; }
                if (i < s.length() && s.charAt(i) == '}') { i++; break; }
                break;
            }
            return m;
        }
        private List<Object> parseArr() {
            List<Object> l = new ArrayList<>();
            i++; ws();
            if (i < s.length() && s.charAt(i) == ']') { i++; return l; }
            while (i < s.length()) {
                l.add(parseValue());
                ws();
                if (i < s.length() && s.charAt(i) == ',') { i++; continue; }
                if (i < s.length() && s.charAt(i) == ']') { i++; break; }
                break;
            }
            return l;
        }
        private String parseStr() {
            if (i < s.length() && s.charAt(i) == '"') i++;
            StringBuilder sb = new StringBuilder();
            while (i < s.length()) {
                char c = s.charAt(i++);
                if (c == '"') break;
                if (c == '\\' && i < s.length()) {
                    char e = s.charAt(i++);
                    switch (e) {
                        case '"': sb.append('"'); break;
                        case '\\': sb.append('\\'); break;
                        case 'n': sb.append('\n'); break;
                        case 'r': sb.append('\r'); break;
                        case 't': sb.append('\t'); break;
                        case 'u':
                            if (i + 4 <= s.length()) {
                                sb.append((char) Integer.parseInt(s.substring(i, i + 4), 16));
                                i += 4;
                            }
                            break;
                        default: sb.append(e);
                    }
                } else sb.append(c);
            }
            return sb.toString();
        }
        private Number parseNum() {
            int j = i;
            while (j < s.length() && "-+0123456789.eE".indexOf(s.charAt(j)) >= 0) j++;
            String t = s.substring(i, j);
            i = j;
            try {
                if (t.contains(".") || t.contains("e") || t.contains("E")) return Double.parseDouble
(t);
                return Long.parseLong(t);
            } catch (Exception e) { return 0; }
        }
    }
}

```

src/main/java/com/airline/util/HttpUtil.java · Java · 58 lines

```java
package com.airline.util;

import com.sun.net.httpserver.HttpExchange;
import java.io.*;
import java.net.URLDecoder;
import java.nio.charset.StandardCharsets;
import java.util.*;

```

src/main/java/com/airline/util/HttpUtil.java · Java · 58 lines

```java
/** HTTP helpers: read body, query params, send JSON/text. */
public final class HttpUtil {
    private HttpUtil() {}

    public static String readBody(HttpExchange ex) throws IOException {
        try (InputStream in = ex.getRequestBody();
             ByteArrayOutputStream out = new ByteArrayOutputStream()) {
            byte[] buf = new byte[8192];
            int n;
            while ((n = in.read(buf)) != -1) out.write(buf, 0, n);
            return out.toString(StandardCharsets.UTF_8);
        }
    }

    public static Map<String, String> query(String raw) {
        Map<String, String> m = new HashMap<>();
        if (raw == null || raw.isEmpty()) return m;
        for (String part : raw.split("&")) {
            int eq = part.indexOf('=');
            try {
                if (eq > 0) m.put(URLDecoder.decode(part.substring(0, eq), "UTF-8"),
                        URLDecoder.decode(part.substring(eq + 1), "UTF-8"));
                else if (!part.isEmpty()) m.put(URLDecoder.decode(part, "UTF-8"), "");
            } catch (Exception ignored) {}
        }
        return m;
    }

    public static void sendJson(HttpExchange ex, int status, String json) throws IOException {
        byte[] b = json.getBytes(StandardCharsets.UTF_8);
        ex.getResponseHeaders().set("Content-Type", "application/json; charset=utf-8");
        ex.getResponseHeaders().set("Access-Control-Allow-Origin", "*");
        ex.sendResponseHeaders(status, b.length);
        try (OutputStream os = ex.getResponseBody()) { os.write(b); }
    }

    public static void sendText(HttpExchange ex, int status, String text, String contentType) throws
IOException {
        byte[] b = text.getBytes(StandardCharsets.UTF_8);
        ex.getResponseHeaders().set("Content-Type", contentType + "; charset=utf-8");
        ex.sendResponseHeaders(status, b.length);
        try (OutputStream os = ex.getResponseBody()) { os.write(b); }
    }

    public static String bearer(HttpExchange ex) {
        String h = ex.getRequestHeaders().getFirst("Authorization");
        if (h != null && h.startsWith("Bearer ")) return h.substring(7).trim();
        return "";
    }
}

```

src/main/java/com/airline/util/Passwords.java · Java · 26 lines

```java
package com.airline.util;

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;

/** SHA-256 password hashing (demo-grade; use bcrypt in production). */
public final class Passwords {
    private Passwords() {}

    public static String hash(String plain) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            byte[] d = md.digest(plain.getBytes(StandardCharsets.UTF_8));
            StringBuilder sb = new StringBuilder();
            for (byte b : d) sb.append(String.format("%02x", b));
            return sb.toString();
```

src/main/java/com/airline/util/Passwords.java · Java · 26 lines

```
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }

    public static boolean verify(String plain, String hash) {
        return hash(plain).equals(hash);
    }
}

```

src/main/java/com/airline/ui/DesktopLauncher.java · Java · 28 lines

```
package com.airline.ui;

import com.airline.model.User;
import com.airline.service.AuthService;
import com.airline.service.BookingService;
import com.airline.service.FlightService;
import com.airline.store.DataStore;
import javax.swing.*;

/** Starts the native desktop UI: login → main window → (logout) → login … */
public final class DesktopLauncher {
    private DesktopLauncher() {}

    public static void launch(DataStore db, AuthService auth, FlightService flights,
                              BookingService bookings, String serverUrl) {
        SwingUtilities.invokeLater(() -> {
            LoginDialog login = new LoginDialog(null, auth);
            login.setVisible(true);
            User user = login.getUser();
            if (user == null) { System.exit(0); return; }
            MainFrame frame = new MainFrame(db, auth, flights, bookings, user, login.getToken(), ser
verUrl);
            frame.setOnLogout(() -> SwingUtilities.invokeLater(
                    () -> launch(db, auth, flights, bookings, serverUrl)));
            frame.setVisible(true);
        });
    }
}

```

src/main/java/com/airline/ui/MainFrame.java · Java · 134 lines

```
package com.airline.ui;

import com.airline.model.Flight;
import com.airline.model.User;
import com.airline.service.AuthService;
import com.airline.service.BookingService;
import com.airline.service.FlightService;
import com.airline.store.DataStore;
import javax.swing.*;
import java.awt.*;
import java.net.URL;

/**
 * Main desktop window of Airoplane Ticket Service.
 * Tabs: Search · Airports · My bookings · Admin.
 */
public class MainFrame extends JFrame {
    private final DataStore db;
    private final AuthService auth;
    private final FlightService flights;
    private final BookingService bookings;
    private final User user;
    private final String token;
    private Runnable onLogout;

```

src/main/java/com/airline/ui/MainFrame.java · Java · 134 lines

```java
    private final JTabbedPane tabs = new JTabbedPane();
    private final JLabel userChip = new JLabel();
    private SearchPanel searchPanel;
    private BookingsPanel bookingsPanel;
    private AdminPanel adminPanel;

    public MainFrame(DataStore db, AuthService auth, FlightService flights, BookingService bookings,
                     User user, String token, String serverUrl) {
        super("Airoplane Ticket Service");
        this.db = db;
        this.auth = auth;
        this.flights = flights;
        this.bookings = bookings;
        this.user = user;
        this.token = token;

        Theme.style(this);
        setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        setSize(1120, 740);
        setLocationRelativeTo(null);
        URL icon = MainFrame.class.getResource("/icons/icon-512.png");
        if (icon != null) setIconImage(Toolkit.getDefaultToolkit().getImage(icon));

        setLayout(new BorderLayout());
        add(header(serverUrl), BorderLayout.NORTH);

        searchPanel = new SearchPanel(flights, this::openBooking);
        AirportsPanel airportsPanel = new AirportsPanel(this::pickAirport);
        bookingsPanel = new BookingsPanel(bookings, flights, user.id, user.role.equals("admin"), this::refreshAll);

        tabs.setFont(Theme.bold(13));
        tabs.addTab("🔍 Search", searchPanel);
        tabs.addTab("🗺 Airports", airportsPanel);
        tabs.addTab("🎫 My bookings", bookingsPanel);
        if (user.role.equals("admin")) {
            adminPanel = new AdminPanel(db, flights, bookings, this::refreshAll);
            tabs.addTab("⚙ Admin", adminPanel);
        }
        add(tabs, BorderLayout.CENTER);
        add(statusBar(serverUrl), BorderLayout.SOUTH);

        refreshAll();
        searchPanel.doSearch();
    }

    private JComponent header(String serverUrl) {
        JPanel p = new JPanel(new BorderLayout());
        p.setBackground(Theme.NAVY);
        p.setBorder(Theme.pad(12, 18, 12, 18));

        JPanel left = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
        left.setOpaque(false);
        JLabel plane = new JLabel("✈");
        plane.setFont(Theme.bold(24));
        plane.setForeground(new Color(0x38BDF8));
        JLabel brand = Theme.title("Airoplane Ticket Service", 19);
        brand.setForeground(Color.WHITE);
        left.add(plane);
        left.add(brand);

        JPanel right = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 0));
        right.setOpaque(false);
        userChip.setFont(Theme.font(13));
        userChip.setForeground(Color.WHITE);
        userChip.setOpaque(true);
        userChip.setBackground(Theme.SLATE);
        userChip.setBorder(Theme.pad(5, 12, 5, 12));
        userChip.setText(user.name + (user.role.equals("admin") ? "  (admin)" : ""));
        JButton logout = Theme.button("Logout");
```

src/main/java/com/airline/ui/MainFrame.java · Java · 134 lines

```
        logout.addActionListener(e -> {
            auth.logout(token);
            if (onLogout != null) onLogout.run();
            dispose();
        });
        right.add(userChip);
        right.add(logout);

        p.add(left, BorderLayout.WEST);
        p.add(right, BorderLayout.EAST);
        return p;
    }

    private JComponent statusBar(String serverUrl) {
        JPanel p = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));
        p.setBackground(Theme.CARD);
        p.setBorder(BorderFactory.createMatteBorder(1, 0, 0, 0, Theme.LINE));
        p.add(Theme.muted("Server: " + serverUrl + "  ·  Phone access: same Wi-Fi + this address", 11));
        return p;
    }

    private void pickAirport(String code) {        searchPanel.setFrom(code);
        tabs.setSelectedIndex(0);
        searchPanel.doSearch();
    }

    private void openBooking(Flight flight, int travellers) {
        new BookingDialog(this, flight, travellers, bookings, user.id).setVisible(true);
        refreshAll();
    }

    private void refreshAll() {
        if (bookingsPanel != null) bookingsPanel.refresh();
        if (adminPanel != null) adminPanel.refresh();
    }

    /** Called when the user logs out (set by the launcher to show the login screen again). */
    public void setOnLogout(Runnable r) { this.onLogout = r; }
}

```

src/main/java/com/airline/ui/LoginDialog.java · Java · 127 lines

```
package com.airline.ui;

import com.airline.model.User;
import com.airline.service.AuthService;
import javax.swing.*;
import java.awt.*;

/** Login / register dialog shown before the main window. */
public class LoginDialog extends JDialog {
    private final AuthService auth;
    private User user;
    private String token;
    private boolean registerMode = false;

    private final JTextField nameField = new PlaceholderField("e.g. Karthik V", 20);
    private final JTextField emailField = new PlaceholderField("you@example.com", 20);
    private final JPasswordField passField = new PlaceholderPasswordField("Enter your password", 20);
    private final JButton tabLogin;
    private final JButton tabRegister;
    private final JLabel title;
    private final JLabel error;
    private final JButton submit;

    public LoginDialog(Window owner, AuthService auth) {
        super(owner, "Sign in — Airoplane Ticket Service", ModalityType.APPLICATION_MODAL);
```

src/main/java/com/airline/ui/LoginDialog.java · Java · 127 lines

```
        this.auth = auth;
        setLayout(new BorderLayout(0, 10));
        getContentPane().setBackground(Theme.BG);
        setSize(430, 420);
        setLocationRelativeTo(owner);

        // sky strip on top
        SkyPanel sky = new SkyPanel();
        sky.setPreferredSize(new Dimension(430, 120));
        add(sky, BorderLayout.NORTH);

        JPanel card = Theme.card(new GridBagLayout());
        GridBagConstraints g = new GridBagConstraints();
        g.insets = new Insets(6, 6, 6, 6);
        g.fill = GridBagConstraints.HORIZONTAL;
        g.anchor = GridBagConstraints.WEST;

        title = Theme.title("Login", 19);
        g.gridx = 0; g.gridy = 0; g.gridwidth = 2;
        card.add(title, g);

        JPanel tabs = new JPanel(new GridLayout(1, 2, 6, 0));
        tabs.setOpaque(false);
        tabLogin = Theme.button("Login");
        tabRegister = Theme.button("Register");
        tabLogin.addActionListener(e -> setMode(false));
        tabRegister.addActionListener(e -> setMode(true));
        tabs.add(tabLogin);
        tabs.add(tabRegister);
        g.gridy = 1; g.gridwidth = 2;
        card.add(tabs, g);

        nameField.setVisible(false);
        g.gridy = 2; g.gridwidth = 1; g.weightx = 0.3;
        card.add(Theme.label("Full name"), g);
        g.gridx = 1; g.weightx = 0.7;
        card.add(nameField, g);

        g.gridx = 0; g.gridy = 3; g.weightx = 0.3;
        card.add(Theme.label("Email"), g);
        g.gridx = 1; g.weightx = 0.7;
        card.add(emailField, g);

        g.gridx = 0; g.gridy = 4; g.weightx = 0.3;
        card.add(Theme.label("Password"), g);
        g.gridx = 1; g.weightx = 0.7;
        card.add(passField, g);

        error = Theme.muted("", 12);
        error.setForeground(Theme.BAD);
        g.gridx = 0; g.gridy = 5; g.gridwidth = 2; g.weightx = 0;
        card.add(error, g);

        g.gridy = 6; g.gridwidth = 2;
        submit = Theme.primary("Continue");
        submit.addActionListener(e -> submit());
        card.add(submit, g);

        add(card, BorderLayout.CENTER);

        JPanel foot = new JPanel(new GridLayout(1, 1));
        foot.setOpaque(false);
        foot.setBorder(Theme.pad(0, 20, 12, 20));
        foot.add(Theme.muted("Demo customer login: demo@skyjet.in / demo123", 11));
        add(foot, BorderLayout.SOUTH);

        getRootPane().setDefaultButton(submit);
        setMode(false);
    }

```

src/main/java/com/airline/ui/LoginDialog.java · Java · 127 lines

```java
    private void setMode(boolean reg) {
        registerMode = reg;
        title.setText(reg ? "Create account" : "Login");
        nameField.setVisible(reg);
        tabLogin.setBackground(reg ? Theme.CARD : Theme.NAVY);
        tabLogin.setForeground(reg ? Theme.INK : Color.WHITE);
        tabRegister.setBackground(reg ? Theme.NAVY : Theme.CARD);
        tabRegister.setForeground(reg ? Color.WHITE : Theme.INK);
        error.setText("");
        pack();
        setLocationRelativeTo(getOwner());
    }

    private void submit() {
        String email = emailField.getText().trim();
        String pass = new String(passField.getPassword());
        AuthService.AuthResult r = registerMode
                ? auth.register(nameField.getText().trim(), email, pass)
                : auth.login(email, pass);
        if (r.error != null) {
            error.setText(r.error);
            return;
        }
        user = r.user;
        token = r.token;
        dispose();
    }

    public User getUser() { return user; }
    public String getToken() { return token; }
}

```

src/main/java/com/airline/ui/Theme.java · Java · 177 lines

```java
package com.airline.ui;

import javax.swing.*;
import javax.swing.border.Border;
import javax.swing.border.CompoundBorder;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;

/** Shared colors, fonts and small widget factories for the desktop UI. */
public final class Theme {
    public static final Color BG = new Color(0xEEF2F7);
    public static final Color CARD = Color.WHITE;
    public static final Color INK = new Color(0x0F172A);
    public static final Color MUTED = new Color(0x64748B);
    public static final Color PRIMARY = new Color(0x0EA5E9);
    public static final Color PRIMARY_DARK = new Color(0x0369A1);
    public static final Color OK = new Color(0x16A34A);
    public static final Color BAD = new Color(0xDC2626);
    public static final Color LINE = new Color(0xE2E8F0);
    public static final Color INPUT_BG = new Color(0xF8FAFC);
    public static final Color NAVY = new Color(0x0F172A);
    public static final Color SLATE = new Color(0x1E293B);

    private Theme() {}

    public static Font font(int size) { return new Font("SansSerif", Font.PLAIN, size); }
    public static Font bold(int size) { return new Font("SansSerif", Font.BOLD, size); }

    public static JButton button(String text) {
        JButton b = new JButton(text);
        b.setFont(bold(13));
        b.setBackground(CARD);
        b.setForeground(INK);
        b.setFocusPainted(false);
```

src/main/java/com/airline/ui/Theme.java · Java · 177 lines

```
        b.setOpaque(true);
        b.setContentAreaFilled(true);
        b.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        b.setBorder(new CompoundBorder(new LineBorder(LINE), new EmptyBorder(8, 14, 8, 14)));
        return b;
    }

    public static JButton primary(String text) {
        JButton b = button(text);
        b.setBackground(PRIMARY);
        b.setForeground(Color.WHITE);
        b.setBorder(new CompoundBorder(new LineBorder(PRIMARY_DARK), new EmptyBorder(9, 18, 9, 1
8)));
        return b;
    }

    public static JButton danger(String text) {
        JButton b = button(text);
        b.setForeground(BAD);
        return b;
    }

    /** Visible outline + padding, so inputs never blend into the white card. */
    public static Border inputBorder() {
        return new CompoundBorder(new LineBorder(LINE), new EmptyBorder(8, 10, 8, 10));
    }

    public static JTextField field(int cols) {
        JTextField f = new JTextField(cols);
        f.setFont(font(14));
        f.setForeground(INK);
        f.setBackground(INPUT_BG);
        f.setCaretColor(INK);
        f.setOpaque(true);
        f.setBorder(inputBorder());
        return f;
    }

    public static JPasswordField passwordField(int cols) {
        JPasswordField f = new JPasswordField(cols);
        f.setFont(font(14));
        f.setForeground(INK);
        f.setBackground(INPUT_BG);
        f.setCaretColor(INK);
        f.setOpaque(true);
        f.setBorder(inputBorder());
        return f;
    }

    public static <T> JComboBox<T> combo(T[] items) {
        JComboBox<T> c = new JComboBox<>(items);
        c.setFont(font(14));
        c.setBackground(CARD);
        c.setForeground(INK);
        c.setBorder(new CompoundBorder(new LineBorder(LINE), new EmptyBorder(6, 8, 6, 8)));
        return c;
    }

    public static JLabel label(String text) {
        JLabel l = new JLabel(text);
        l.setFont(bold(12));
        l.setForeground(new Color(0x334155));
        return l;
    }

    public static JLabel title(String text, int size) {
        JLabel l = new JLabel(text);
        l.setFont(bold(size));
        l.setForeground(INK);
        return l;
```

src/main/java/com/airline/ui/Theme.java · Java · 177 lines

```
    }

    public static JLabel muted(String text, int size) {
        JLabel l = new JLabel(text);
        l.setFont(font(size));
        l.setForeground(MUTED);
        return l;
    }

    public static JPanel card(LayoutManager layout) {
        JPanel p = new JPanel(layout);
        p.setBackground(CARD);
        p.setBorder(new EmptyBorder(16, 16, 16, 16));
        p.setOpaque(true);
        return p;
    }

    public static Border pad(int top, int left, int bottom, int right) {
        return new EmptyBorder(top, left, bottom, right);
    }

    /** Foreground colour for a booking status badge. */
    public static Color statusFg(String status) {
        if (status == null) return MUTED;
        switch (status) {
            case "CONFIRMED": return new Color(0x166534);
            case "PENDING": return new Color(0x92400E);
            default: return new Color(0x991B1B);
        }
    }

    /** Background colour for a booking status badge. */
    public static Color statusBg(String status) {
        if (status == null) return new Color(0xF1F5F9);
        switch (status) {
            case "CONFIRMED": return new Color(0xDCFCE7);
            case "PENDING": return new Color(0xFEF3C7);
            default: return new Color(0xFEE2E2);
        }
    }

    /** Status badge label. */
    public static JLabel statusPill(String status) {
        String text = status == null ? "PENDING" : status;
        if (text.equals("PENDING")) text = "PENDING · awaiting admin";
        return pill(text, statusFg(status), statusBg(status));
    }

    public static Border round() {
        return new LineBorder(LINE);
    }

    /** Small rounded pill used for status/seat counts. */
    public static JLabel pill(String text, Color fg, Color bg) {
        JLabel l = new JLabel(text);
        l.setFont(bold(12));
        l.setForeground(fg);
        l.setOpaque(true);
        l.setBackground(bg);
        l.setBorder(new EmptyBorder(4, 10, 4, 10));
        l.setHorizontalAlignment(SwingConstants.CENTER);
        return l;
    }

    /** Applies the app look and feel to a window. */
    public static void style(JFrame f) {
        try {
            UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());
        } catch (Exception ignored) {}
        f.setBackground(BG);
```

src/main/java/com/airline/ui/Theme.java · Java · 177 lines

```
    }
}

```

src/main/java/com/airline/ui/PlaceholderField.java · Java · 44 lines

```
package com.airline.ui;

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;
import java.awt.event.FocusAdapter;
import java.awt.event.FocusEvent;

/** Text field with a visible grey hint that disappears as soon as the user types. */
public class PlaceholderField extends JTextField {
    private final String hint;

    public PlaceholderField(String hint, int cols) {
        super(cols);
        this.hint = hint;
        setFont(Theme.font(14));
        setForeground(Theme.INK);
        setBackground(Theme.INPUT_BG);
        setCaretColor(Theme.INK);
        setOpaque(true);
        setBorder(new javax.swing.border.CompoundBorder(new LineBorder(Theme.LINE), new EmptyBorder
(8, 10, 8, 10)));
        addFocusListener(new FocusAdapter() {
            @Override public void focusGained(FocusEvent e) { repaint(); }
            @Override public void focusLost(FocusEvent e) { repaint(); }
        });
    }

    public String getHint() { return hint; }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        if (!getText().isEmpty()) return;
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIA
S_ON);
        g2.setColor(isFocusOwner() ? Theme.PRIMARY : Theme.MUTED);
        g2.setFont(Theme.font(14));
        Insets in = getInsets();
        g2.drawString(hint, in.left, getHeight() - in.bottom - 9);
        g2.dispose();
    }
}

```

src/main/java/com/airline/ui/PlaceholderPasswordField.java · Java · 44 lines

```
package com.airline.ui;

import javax.swing.*;
import javax.swing.border.CompoundBorder;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import java.awt.*;
import java.awt.event.FocusAdapter;
import java.awt.event.FocusEvent;

/** Password field with a visible grey hint while it is empty. */
public class PlaceholderPasswordField extends JPasswordField {
    private final String hint;

    public PlaceholderPasswordField(String hint, int cols) {
```

src/main/java/com/airline/ui/PlaceholderPasswordField.java · Java · 44 lines

```
        super(cols);
        this.hint = hint;
        setFont(Theme.font(14));
        setForeground(Theme.INK);
        setBackground(Theme.INPUT_BG);
        setCaretColor(Theme.INK);
        setOpaque(true);
        setEchoChar('•');
        setBorder(new CompoundBorder(new LineBorder(Theme.LINE), new EmptyBorder(8, 10, 8, 10)));
        addFocusListener(new FocusAdapter() {
            @Override public void focusGained(FocusEvent e) { repaint(); }
            @Override public void focusLost(FocusEvent e) { repaint(); }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        if (getPassword().length > 0) return;
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
        g2.setColor(isFocusOwner() ? Theme.PRIMARY : Theme.MUTED);
        g2.setFont(Theme.font(14));
        Insets in = getInsets();
        g2.drawString(hint, in.left, getHeight() - in.bottom - 9);
        g2.dispose();
    }
}

```

src/main/java/com/airline/ui/SkyPanel.java · Java · 102 lines

```
package com.airline.ui;

import javax.swing.*;
import java.awt.*;
import java.awt.geom.*;
import java.util.ArrayList;
import java.util.List;

/**
 * Animated sky background: gradient sky, drifting clouds and a plane flying across.
 * Used as the window backdrop of the desktop app.
 */
public class SkyPanel extends JPanel {
    private final List<Cloud> clouds = new ArrayList<>();
    private double planeX = -0.15;
    private final Timer timer;

    private static class Cloud {
        double x, y, w, h, speed, alpha;
        Cloud(double x, double y, double w, double h, double speed, double alpha) {
            this.x = x; this.y = y; this.w = w; this.h = h; this.speed = speed; this.alpha = alpha;
        }
    }

    public SkyPanel() {
        setOpaque(true);
        clouds.add(new Cloud(0.10, 0.62, 0.20, 0.055, 0.0045, 0.85));
        clouds.add(new Cloud(0.45, 0.30, 0.14, 0.040, 0.0028, 0.60));
        clouds.add(new Cloud(0.75, 0.72, 0.26, 0.070, 0.0060, 0.45));
        timer = new Timer(33, e -> {
            planeX += 0.0022;
            if (planeX > 1.25) planeX = -0.25;
            for (Cloud c : clouds) {
                c.x += c.speed;
                if (c.x > 1.2) c.x = -c.w - 0.1;
            }
            repaint();
```

src/main/java/com/airline/ui/SkyPanel.java · Java · 102 lines

```
        });
        timer.start();
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        int w = getWidth(), h = getHeight();

        g2.setPaint(new GradientPaint(0, 0, new Color(0x0C4A6E), 0, h, new Color(0xBAE6FD)));
        g2.fillRect(0, 0, w, h);

        // sun
        g2.setPaint(new Color(0xFE, 0xF9, 0xC3, 210));
        g2.fillOval((int) (w * 0.82), (int) (h * 0.08), (int) (h * 0.16), (int) (h * 0.16));
        g2.setPaint(new Color(0xFE, 0xF9, 0xC3, 60));
        g2.fillOval((int) (w * 0.82) - 30, (int) (h * 0.08) - 30, (int) (h * 0.16) + 60, (int) (h * 0.16) + 60);

        // clouds
        for (Cloud c : clouds) drawCloud(g2, c, w, h);

        // flying plane
        drawPlane(g2, w, h);
        g2.dispose();
    }

    private void drawCloud(Graphics2D g, Cloud c, int w, int h) {
        double cw = c.w * w, ch = c.h * h, cx = c.x * w, cy = c.y * h;
        g.setPaint(new Color(255, 255, 255, (int) (255 * c.alpha)));
        g.fillOval((int) cx, (int) cy, (int) cw, (int) ch);
        g.fillOval((int) (cx + cw * 0.18), (int) (cy - ch * 0.55), (int) (cw * 0.5), (int) (ch * 1.1));
        g.fillOval((int) (cx + cw * 0.55), (int) (cy - ch * 0.30), (int) (cw * 0.42), (int) (ch * 0.95));
    }

    private void drawPlane(Graphics2D g, int w, int h) {
        double px = planeX * w, py = h * 0.30 + Math.sin(planeX * 6) * h * 0.045;
        double size = Math.max(26.0, h * 0.055);
        g.setPaint(new Color(0, 0, 0, 45));
        g.fillOval((int) (px - size * 0.35), (int) (py + size * 0.55), (int) (size * 1.5), (int) (size * 0.22));

        AffineTransform old = g.getTransform();
        g.translate(px, py);
        g.rotate(-0.07);
        Path2D plane = new Path2D.Double();
        plane.moveTo(-size, 0);
        plane.lineTo(size * 0.15, -size * 0.16);
        plane.lineTo(size * 0.9, 0);
        plane.lineTo(size * 0.15, size * 0.16);
        plane.closePath();
        g.setPaint(Color.WHITE);
        g.fill(plane);
        Path2D wing = new Path2D.Double();
        wing.moveTo(-size * 0.45, 0);
        wing.lineTo(size * 0.2, -size * 0.55);
        wing.lineTo(size * 0.55, -size * 0.5);
        wing.lineTo(size * 0.05, size * 0.02);
        wing.closePath();
        g.setPaint(new Color(0xE0F2FE));
        g.fill(wing);
        g.setTransform(old);
    }
}

```

src/main/java/com/airline/ui/SearchPanel.java · Java · 191 lines

```java
package com.airline.ui;

import com.airline.model.Flight;
import com.airline.service.FlightService;
import javax.swing.*;
import java.awt.*;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.List;

/** Search tab: from/to/date/travellers + flight result cards. */
public class SearchPanel extends JPanel {
    public interface OnBook { void book(Flight flight, int travellers); }

    private final FlightService flights;
    private final OnBook onBook;
    private final JComboBox<String> fromSel = Theme.combo(new String[0]);
    private final JComboBox<String> toSel = Theme.combo(new String[0]);
    private final JComboBox<String> dateSel = Theme.combo(new String[0]);
    private final JComboBox<Integer> paxSel = Theme.combo(new Integer[]{1, 2, 3, 4, 5, 6});
    private final JPanel results = new JPanel();
    private final JLabel count = Theme.muted("", 12);

    public SearchPanel(FlightService flights, OnBook onBook) {
        this.flights = flights;
        this.onBook = onBook;
        setLayout(new BorderLayout(0, 12));
        setBackground(Theme.BG);
        setBorder(Theme.pad(14, 14, 14, 14));

        // ---- search form ----
        JPanel form = Theme.card(new GridBagLayout());
        GridBagConstraints g = new GridBagConstraints();
        g.insets = new Insets(6, 6, 6, 6);
        g.anchor = GridBagConstraints.WEST;

        g.gridx = 0; g.gridy = 0; form.add(Theme.label("From"), g);
        g.gridx = 1; g.gridy = 0; g.fill = GridBagConstraints.HORIZONTAL; g.weightx = 0.18;
        fromSel.setPreferredSize(new Dimension(150, 38));
        form.add(fromSel, g);
        g.gridx = 2; g.gridy = 0; g.weightx = 0; g.fill = GridBagConstraints.NONE;
        JButton swap = Theme.button("⇄");
        swap.setToolTipText("Swap");
        swap.addActionListener(e -> {
            String a = (String) fromSel.getSelectedItem(), b = (String) toSel.getSelectedItem();
            fromSel.setSelectedItem(b);
            toSel.setSelectedItem(a);
        });
        form.add(swap, g);

        g.gridx = 3; g.gridy = 0; form.add(Theme.label("To"), g);
        g.gridx = 4; g.gridy = 0; g.fill = GridBagConstraints.HORIZONTAL; g.weightx = 0.18;
        toSel.setPreferredSize(new Dimension(150, 38));
        form.add(toSel, g);

        g.gridx = 5; g.gridy = 0; g.fill = GridBagConstraints.NONE; g.weightx = 0;
        form.add(Theme.label("Date"), g);
        g.gridx = 6; g.gridy = 0; g.fill = GridBagConstraints.HORIZONTAL; g.weightx = 0.24;
        dateSel.setPreferredSize(new Dimension(170, 38));
        form.add(dateSel, g);

        g.gridx = 7; g.gridy = 0; g.fill = GridBagConstraints.NONE; g.weightx = 0;
        form.add(Theme.label("Travellers"), g);
        g.gridx = 8; g.gridy = 0; g.fill = GridBagConstraints.HORIZONTAL; g.weightx = 0.1;
        paxSel.setPreferredSize(new Dimension(80, 38));
        form.add(paxSel, g);

        g.gridx = 9; g.gridy = 0; g.fill = GridBagConstraints.NONE; g.weightx = 0;
        JButton search = Theme.primary("🔍 Search flights");
        search.addActionListener(e -> doSearch());
```

src/main/java/com/airline/ui/SearchPanel.java · Java · 191 lines

```
71          form.add(search, g);
72
73          // ---- results ----
74          results.setLayout(new BoxLayout(results, BoxLayout.Y_AXIS));
75          results.setOpaque(false);
76
77          JPanel head = new JPanel(new BorderLayout());
78          head.setOpaque(false);
79          head.add(Theme.title("Available flights", 17), BorderLayout.WEST);
80          head.add(count, BorderLayout.EAST);
81
82          JScrollPane scroll = new JScrollPane(results);
83          scroll.setBorder(BorderFactory.createEmptyBorder());
84          scroll.setOpaque(false);
85          scroll.getViewport().setOpaque(false);
86          scroll.getVerticalScrollBar().setUnitIncrement(16);
87
88          add(form, BorderLayout.NORTH);
89          add(head, BorderLayout.CENTER);
90          JPanel center = new JPanel(new BorderLayout(0, 8));
91          center.setOpaque(false);
92          center.add(head, BorderLayout.NORTH);
93          center.add(scroll, BorderLayout.CENTER);
94          remove(head);
95          add(center, BorderLayout.CENTER);
96
97          loadAirports();
98      }
99
100     private void loadAirports() {
101         String[] codes = AirportsPanel.DATA.keySet().toArray(new String[0]);
102         fromSel.setModel(new DefaultComboBoxModel<>(codes));
103         toSel.setModel(new DefaultComboBoxModel<>(codes));
104         fromSel.setSelectedItem("DEL");
105         toSel.setSelectedItem("BOM");
106
107         DateTimeFormatter fmt = DateTimeFormatter.ofPattern("EEE, dd MMM yyyy");
108         String[] dates = new String[14];
109         for (int i = 0; i < dates.length; i++) dates[i] = LocalDate.now().plusDays(i + 1).format(fm
    t);
110         dateSel.setModel(new DefaultComboBoxModel<>(dates));
111     }
112
113     /** Selects a departure airport (used by the Airports tab). */
114     public void setFrom(String code) {
115         fromSel.setSelectedItem(code);
116     }
117
118     public void doSearch() {
119         String from = (String) fromSel.getSelectedItem();
120         String to = (String) toSel.getSelectedItem();
121         String dateText = (String) dateSel.getSelectedItem();
122         if (from == null || to == null || dateText == null) return;
123         if (from.equals(to)) { JOptionPane.showMessageDialog(this, "From and To cannot be the sam
    e."); return; }
124         String date = LocalDate.parse(dateText, DateTimeFormatter.ofPattern("EEE, dd MMM yyyy")).toS
    tring();
125
126         List<Flight> list = flights.search(from, to, date);
127         results.removeAll();
128         count.setText(list.size() + " flight" + (list.size() == 1 ? "" : "s") + " found");
129         if (list.isEmpty()) {
130             JPanel empty = Theme.card(new BorderLayout());
131             empty.add(Theme.muted("No flights on this route and date. Try another day or city.", 1
    3), BorderLayout.CENTER);
132             empty.setAlignmentX(Component.LEFT_ALIGNMENT);
133             results.add(empty);
134         } else {
135             for (Flight f : list) results.add(flightCard(f));
136         }
```

src/main/java/com/airline/ui/SearchPanel.java · Java · 191 lines

```
        results.revalidate();
        results.repaint();
    }

    private JPanel flightCard(Flight f) {
        JPanel p = new JPanel(new BorderLayout(14, 6));
        p.setBackground(Theme.CARD);
        p.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createMatteBorder(0, 3, 0, 0, Theme.PRIMARY),
                Theme.pad(12, 14, 12, 14)));
        p.setAlignmentX(Component.LEFT_ALIGNMENT);
        p.setMaximumSize(new Dimension(Integer.MAX_VALUE, 92));

        JPanel left = new JPanel(new GridLayout(0, 1));
        left.setOpaque(false);
        JLabel route = Theme.title(f.from + "  →  " + f.to, 18);
        JLabel meta = Theme.muted(f.airline + " · " + f.flightNumber + " · " + f.date + " · " +
                f.departure + "–" + f.arrival + " · " + (f.durationMin / 60) + "h " + (f.durationMin
% 60) + "m", 12);
        left.add(route);
        left.add(meta);

        JPanel right = new JPanel(new GridBagLayout());
        right.setOpaque(false);
        GridBagConstraints g = new GridBagConstraints();
        g.insets = new Insets(2, 6, 2, 6);
        g.anchor = GridBagConstraints.EAST;

        g.gridx = 0; g.gridy = 0;
        right.add(Theme.pill(f.availableSeats() + " seats left",
                f.availableSeats() > 20 ? new Color(0x166534) : new Color(0x991B1B),
                f.availableSeats() > 20 ? new Color(0xDCFCE7) : new Color(0xFEE2E2)), g);

        g.gridx = 1; g.gridy = 0;
        JPanel fare = new JPanel(new GridLayout(0, 1));
        fare.setOpaque(false);
        JLabel price = Theme.title("₹" + String.format("%,.0f", f.price), 18);
        price.setForeground(Theme.PRIMARY_DARK);
        price.setHorizontalAlignment(SwingConstants.RIGHT);
        JLabel per = Theme.muted("per adult", 11);
        per.setHorizontalAlignment(SwingConstants.RIGHT);
        fare.add(price);
        fare.add(per);
        right.add(fare, g);

        g.gridx = 2; g.gridy = 0;
        JButton book = Theme.primary("Book now");
        book.addActionListener(e -> onBook.book(f, (Integer) paxSel.getSelectedItem()));
        right.add(book, g);

        p.add(left, BorderLayout.CENTER);
        p.add(right, BorderLayout.EAST);
        return p;
    }
}
```

src/main/java/com/airline/ui/AirportsPanel.java · Java · 173 lines

```
package com.airline.ui;

import javax.swing.*;
import java.awt.*;
import java.awt.geom.*;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** Airport locations, map painting and the airport list panel. */
```

#	src/main/java/com/airline/ui/AirportsPanel.java · Java · 173 lines

```java
public class AirportsPanel extends JPanel {
    /** code -> [city, airport name, lat, lng] */
    public static final Map<String, String[]> DATA = new LinkedHashMap<>();
    static {
        DATA.put("DEL", new String[]{"New Delhi", "Indira Gandhi International Airport", "28.556", "77.100"});
        DATA.put("BOM", new String[]{"Mumbai", "Chhatrapati Shivaji Maharaj International", "19.090", "72.865"});
        DATA.put("BLR", new String[]{"Bengaluru", "Kempegowda International Airport", "13.198", "77.706"});
        DATA.put("HYD", new String[]{"Hyderabad", "Rajiv Gandhi International Airport", "17.231", "78.429"});
        DATA.put("MAA", new String[]{"Chennai", "Chennai International Airport", "12.994", "80.171"});
    }

    private static final String OUTLINE =
            "109,4 138,18 168,14 196,30 228,32 252,52 262,80 288,96 296,124 282,150 292,178 276,204 282,232 " +
            "258,258 240,292 222,330 206,368 188,410 172,438 158,410 146,372 128,338 104,308 78,282 52,258 " +
            "30,232 14,206 22,182 12,158 28,132 24,104 48,88 60,60 84,44";

    public interface OnPick { void pick(String code); }

    private final OnPick onPick;
    private final MapPanel map;

    public AirportsPanel(OnPick onPick) {
        this.onPick = onPick;
        setLayout(new BorderLayout(12, 12));
        setBackground(Theme.BG);
        setBorder(Theme.pad(14, 14, 14, 14));

        map = new MapPanel();
        JPanel mapCard = Theme.card(new BorderLayout());
        mapCard.setBorder(Theme.pad(10, 10, 10, 10));
        mapCard.add(map, BorderLayout.CENTER);

        JPanel list = new JPanel();
        list.setLayout(new BoxLayout(list, BoxLayout.Y_AXIS));
        list.setOpaque(false);
        list.add(Theme.title("Served airports", 17));
        list.add(Box.createVerticalStrut(4));
        list.add(Theme.muted("Click a marker on the map or pick a city to search flights from it.", 12));
        list.add(Box.createVerticalStrut(10));
        for (Map.Entry<String, String[]> e : DATA.entrySet()) list.add(airportCard(e.getKey(), e.getValue()));

        JScrollPane scroll = new JScrollPane(list);
        scroll.setBorder(BorderFactory.createEmptyBorder());
        scroll.setOpaque(false);
        scroll.getViewport().setOpaque(false);

        add(mapCard, BorderLayout.CENTER);
        add(scroll, BorderLayout.EAST);
        mapCard.setPreferredSize(new Dimension(460, 520));
        scroll.setPreferredSize(new Dimension(360, 520));
    }

    private JPanel airportCard(String code, String[] d) {
        JPanel p = new JPanel(new BorderLayout(10, 4));
        p.setBackground(Theme.CARD);
        p.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createMatteBorder(0, 3, 0, 0, Theme.PRIMARY),
                Theme.pad(10, 12, 10, 12)));
        p.setAlignmentX(Component.LEFT_ALIGNMENT);
        p.setMaximumSize(new Dimension(Integer.MAX_VALUE, 74));

```

src/main/java/com/airline/ui/AirportsPanel.java · Java · 173 lines

```
        JPanel left = new JPanel(new GridLayout(0, 1));
        left.setOpaque(false);
        JLabel codeL = Theme.title(code + "  ·  " + d[0], 15);
        JLabel meta = Theme.muted(d[1] + "  ·  " + d[2] + "°N, " + d[3] + "°E", 11);
        left.add(codeL);
        left.add(meta);

        JButton go = Theme.button("Fly from here");
        go.addActionListener(e -> onPick.pick(code));

        p.add(left, BorderLayout.CENTER);
        p.add(go, BorderLayout.EAST);
        return p;
    }

    /** Painted map of India with airport markers and route network. */
    static class MapPanel extends JPanel {
        private String selected = null;
        private final Timer timer;
        private double pulse = 0;

        MapPanel() {
            setPreferredSize(new Dimension(420, 480));
            setBackground(new Color(0xE0F2FE));
            timer = new Timer(40, e -> { pulse = (pulse + 0.05) % 1.0; repaint(); });
            timer.start();
        }

        void select(String code) { selected = code; repaint(); }

        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
            g2.setRenderingHint(RenderingHints.KEY_STROKE_CONTROL, RenderingHints.VALUE_STROKE_PUR
E);
            int w = getWidth(), h = getHeight();
            double sx = w / 400.0, sy = h / 440.0;

            // sea
            g2.setPaint(new GradientPaint(0, 0, new Color(0xBAE6FD), 0, h, new Color(0xE0F2FE)));
            g2.fillRect(0, 0, w, h);

            // land
            Path2D land = new Path2D.Double();
            String[] pts = OUTLINE.split(" ");
            for (int i = 0; i < pts.length; i++) {
                String[] xy = pts[i].split(",");
                double x = Double.parseDouble(xy[0]) * sx, y = Double.parseDouble(xy[1]) * sy;
                if (i == 0) land.moveTo(x, y); else land.lineTo(x, y);
            }
            land.closePath();
            g2.setPaint(Color.WHITE);
            g2.fill(land);
            g2.setPaint(new Color(0x0369A1));
            g2.setStroke(new BasicStroke(2f));
            g2.draw(land);

            // routes
            List<String> codes = new ArrayList<>(DATA.keySet());
            g2.setColor(new Color(0x0E, 0xA5, 0xE9, 140));
            g2.setStroke(new BasicStroke(1.4f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10f, ne
w float[]{5, 4}, 0));
            for (int i = 0; i < codes.size(); i++) {
                for (int j = i + 1; j < codes.size(); j++) {
                    Point2D a = project(codes.get(i), sx, sy), b = project(codes.get(j), sx, sy);
                    g2.draw(new Line2D.Double(a, b));
                }
            }
```

src/main/java/com/airline/ui/AirportsPanel.java · Java · 173 lines

```
141
142             // markers
143             for (String code : codes) {
144                 Point2D p = project(code, sx, sy);
145                 boolean sel = code.equals(selected);
146                 double r = sel ? 9 : 6;
147                 g2.setPaint(new Color(0xDC, 0x26, 0x26, 60));
148                 double pr = 10 + pulse * 12;
149                 g2.fillOval((int) (p.getX() - pr), (int) (p.getY() - pr), (int) (pr * 2), (int) (pr
    * 2));
150                 g2.setPaint(sel ? new Color(0xB91C1C) : new Color(0xDC2626));
151                 g2.fillOval((int) (p.getX() - r), (int) (p.getY() - r), (int) (r * 2), (int) (r *
    2));
152                 g2.setPaint(Color.WHITE);
153                 g2.setStroke(new BasicStroke(2f));
154                 g2.drawOval((int) (p.getX() - r), (int) (p.getY() - r), (int) (r * 2), (int) (r *
    2));
155                 g2.setFont(Theme.bold(12));
156                 g2.setColor(Theme.INK);
157                 String label = code;
158                 int tw = g2.getFontMetrics().stringWidth(label);
159                 g2.drawString(label, (int) (p.getX() - tw / 2), (int) (p.getY() - r - 7));
160             }
161             g2.dispose();
162         }
163
164         private Point2D project(String code, double sx, double sy) {
165             String[] d = DATA.get(code);
166             double lat = Double.parseDouble(d[2]), lng = Double.parseDouble(d[3]);
167             double x = (lng - 66) / (92 - 66) * 400 * sx;
168             double y = (36 - lat) / (36 - 7) * 440 * sy;
169             return new Point2D.Double(x, y);
170         }
171     }
172 }
173
```

src/main/java/com/airline/ui/BookingDialog.java · Java · 314 lines

```
1   package com.airline.ui;
2
3   import com.airline.model.Booking;
4   import com.airline.model.Flight;
5   import com.airline.service.AuthService;
6   import com.airline.service.BookingService;
7   import javax.swing.*;
8   import java.awt.*;
9   import java.util.ArrayList;
10  import java.util.LinkedHashMap;
11  import java.util.List;
12  import java.util.Map;
13
14  /** Booking flow: seat map, passenger details, mock payment. */
15  public class BookingDialog extends JDialog {
16      private final Flight flight;
17      private final int travellers;
18      private final BookingService bookings;
19      private final String userId;
20
21      private final JPanel seatPanel = new JPanel();
22      private final List<String> selectedSeats = new ArrayList<>();
23      private final Map<Integer, JTextField> nameFields = new LinkedHashMap<>();
24      private final Map<Integer, JTextField> ageFields = new LinkedHashMap<>();
25      private final Map<Integer, JComboBox<String>> genderFields = new LinkedHashMap<>();
26      private final JTextField cardField = Theme.field(18);
27      private final JLabel totalLabel = Theme.title("", 18);
28      private final JLabel seatInfo = Theme.muted("", 12);
29
30      public BookingDialog(Window owner, Flight flight, int travellers, BookingService bookings, Strin
    g userId) {
```

#	src/main/java/com/airline/ui/BookingDialog.java · Java · 314 lines

```
        super(owner, "Complete booking — " + flight.airline + " " + flight.flightNumber, ModalityTyp
e.APPLICATION_MODAL);
        this.flight = flight;
        this.travellers = travellers;
        this.bookings = bookings;
        this.userId = userId;

        setLayout(new BorderLayout(14, 14));
        getContentPane().setBackground(Theme.BG);
        setSize(920, 620);
        setLocationRelativeTo(owner);

        add(header(), BorderLayout.NORTH);

        JPanel center = new JPanel(new GridLayout(1, 2, 14, 0));
        center.setOpaque(false);
        center.add(seatsCard());
        center.add(detailsCard());
        add(center, BorderLayout.CENTER);
        add(footer(), BorderLayout.SOUTH);

        renderSeats();
        updateTotal();
    }

    private JPanel header() {
        JPanel p = new JPanel(new BorderLayout());
        p.setBackground(Theme.NAVY);
        p.setBorder(Theme.pad(14, 18, 14, 18));
        JLabel t = Theme.title(flight.from + "  →  " + flight.to, 20);
        t.setForeground(Color.WHITE);
        JLabel m = Theme.muted(flight.airline + " · " + flight.flightNumber + " · " + flight.date +
" · " +
                flight.departure + "–" + flight.arrival, 12);
        m.setForeground(new Color(0xBAE6FD));
        p.add(t, BorderLayout.NORTH);
        p.add(m, BorderLayout.SOUTH);
        return p;
    }

    private JPanel seatsCard() {
        JPanel card = Theme.card(new BorderLayout(0, 8));
        JLabel h = Theme.title("1. Select seats (" + travellers + (travellers == 1 ? " person" : " p
eople") + ")", 15);
        card.add(h, BorderLayout.NORTH);

        seatPanel.setLayout(new GridLayout(30, 7, 4, 4));
        seatPanel.setBackground(new Color(0xF8FAFC));
        seatPanel.setBorder(Theme.pad(8, 8, 8, 8));
        JScrollPane sc = new JScrollPane(seatPanel);
        sc.setBorder(BorderFactory.createEmptyBorder());
        sc.getViewport().setBackground(new Color(0xF8FAFC));
        sc.getVerticalScrollBar().setUnitIncrement(16);
        card.add(sc, BorderLayout.CENTER);

        JPanel legend = new JPanel(new FlowLayout(FlowLayout.LEFT, 12, 0));
        legend.setOpaque(false);
        legend.add(seatLegend("Free", Color.WHITE, Theme.LINE));
        legend.add(seatLegend("Taken", new Color(0xE2E8F0), new Color(0xCBD5E1)));
        legend.add(seatLegend("Selected", Theme.PRIMARY, Theme.PRIMARY));
        card.add(legend, BorderLayout.SOUTH);
        return card;
    }

    private JComponent seatLegend(String text, Color bg, Color border) {
        JPanel row = new JPanel(new FlowLayout(FlowLayout.LEFT, 4, 0));
        row.setOpaque(false);
        JButton sq = new JButton();
        sq.setPreferredSize(new Dimension(20, 20));
        sq.setBackground(bg);
```

src/main/java/com/airline/ui/BookingDialog.java · Java · 314 lines

```
98          sq.setBorder(BorderFactory.createLineBorder(border));
99          sq.setEnabled(false);
100         row.add(sq);
101         row.add(Theme.muted(text, 11));
102         return row;
103     }
104
105     private JPanel detailsCard() {
106         JPanel card = Theme.card(new GridBagLayout());
107         GridBagConstraints g = new GridBagConstraints();
108         g.insets = new Insets(5, 4, 5, 4);
109         g.fill = GridBagConstraints.HORIZONTAL;
110         g.anchor = GridBagConstraints.WEST;
111
112         g.gridx = 0; g.gridy = 0; g.gridwidth = 4;
113         card.add(Theme.title("2. Passenger details (" + travellers + (travellers == 1 ? " person" :
" people") + ")", 15), g);
114         g.gridy = 1; g.gridwidth = 4; g.weightx = 1;
115         card.add(buildPassengerRows(), g);
116
117         g.gridy = 2; g.gridwidth = 4;
118         card.add(Theme.title("3. Payment (mock)", 15), g);
119         g.gridx = 0; g.gridy = 3; g.gridwidth = 1; g.weightx = 0.2;
120         card.add(Theme.label("Card number"), g);
121         g.gridx = 1; g.gridy = 3; g.gridwidth = 3; g.weightx = 0.8;
122         card.setBackground(Theme.CARD);
123         cardField.setText("4111 1111 1111 1111");
124         card.add(cardField, g);
125
126         g.gridx = 0; g.gridy = 4; g.gridwidth = 4; g.weightx = 0;
127         totalLabel.setForeground(Theme.PRIMARY_DARK);
128         card.add(totalLabel, g);
129         g.gridy = 5;
130         seatInfo.setText("Pick " + travellers + " seat(s) for " + travellers + (travellers == 1 ? "
person" : " people") + " and fill each person's name and age.");
131         card.add(seatInfo, g);
132         g.gridy = 6;
133         card.add(Theme.muted("Test card only — no real money is charged.", 11), g);
134         return card;
135     }
136
137     private JPanel footer() {
138         JPanel p = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
139         p.setOpaque(false);
140         JButton cancel = Theme.button("Cancel");
141         cancel.addActionListener(e -> dispose());
142         JButton pay = Theme.primary("Pay & confirm booking");
143         pay.addActionListener(e -> confirm(pay));
144         p.add(cancel);
145         p.add(pay);
146         return p;
147     }
148
149     private void renderSeats() {
150         seatPanel.removeAll();
151         for (int row = 1; row <= 30; row++) {
152             for (String col : new String[]{"A", "B", "C", "", "D", "E", "F"}) {
153                 if (col.isEmpty()) {
154                     JLabel l = new JLabel(String.valueOf(row), SwingConstants.CENTER);
155                     l.setFont(Theme.font(10));
156                     l.setForeground(Theme.MUTED);
157                     seatPanel.add(l);
158                     continue;
159                 }
160                 String seat = row + col;
161                 boolean taken = flight.bookedSeats.contains(seat);
162                 JButton b = new JButton(seat);
163                 b.setFont(Theme.font(10));
164                 b.setMargin(new Insets(1, 1, 1, 1));
165                 b.setFocusPainted(false);
```

src/main/java/com/airline/ui/BookingDialog.java · Java · 314 lines

```
                b.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
                b.setBorder(BorderFactory.createLineBorder(taken ? new Color(0xCBD5E1) : Theme.LINE));
                b.setBackground(taken ? new Color(0xE2E8F0) : Color.WHITE);
                b.setForeground(taken ? new Color(0x94A3B8) : Theme.INK);
                b.setEnabled(!taken);
                b.addActionListener(e -> {
                    if (selectedSeats.contains(seat)) {
                        selectedSeats.remove(seat);
                        b.setBackground(Color.WHITE);
                        b.setForeground(Theme.INK);
                        b.setBorder(BorderFactory.createLineBorder(Theme.LINE));
                    } else {
                        if (selectedSeats.size() >= travellers) {
                            JOptionPane.showMessageDialog(this, "You can pick " + travellers + " seat(s) only.");
                            return;
                        }
                        selectedSeats.add(seat);
                        b.setBackground(Theme.PRIMARY);
                        b.setForeground(Color.WHITE);
                        b.setBorder(BorderFactory.createLineBorder(Theme.PRIMARY));
                    }
                    updateTotal();
                });
                seatPanel.add(b);
            }
        }
        seatPanel.revalidate();
        seatPanel.repaint();
    }

    private JPanel buildPassengerRows() {
        JPanel rows = new JPanel(new GridBagLayout());
        rows.setOpaque(false);
        GridBagConstraints g = new GridBagConstraints();
        g.insets = new Insets(3, 4, 3, 4);
        g.fill = GridBagConstraints.HORIZONTAL;
        g.anchor = GridBagConstraints.WEST;

        // column headers so every box is labelled
        g.gridx = 0; g.gridy = 0; g.weightx = 0;
        rows.add(Theme.muted("#", 11), g);
        g.gridx = 1; g.weightx = 1;
        rows.add(Theme.label("Full name"), g);
        g.gridx = 2; g.weightx = 0;
        rows.add(Theme.label("Age"), g);
        g.gridx = 3; g.weightx = 0;
        rows.add(Theme.label("Gender"), g);

        for (int i = 0; i < travellers; i++) {
            int y = i + 1;
            g.gridx = 0; g.gridy = y; g.weightx = 0;
            rows.add(Theme.label("P" + (i + 1)), g);

            JTextField name = new PlaceholderField("Passenger " + (i + 1) + " name", 16);
            g.gridx = 1; g.weightx = 1;
            rows.add(name, g);

            JTextField age = new PlaceholderField("Age", 4);
            age.setHorizontalAlignment(SwingConstants.CENTER);
            g.gridx = 2; g.weightx = 0;
            rows.add(age, g);

            JComboBox<String> gender = Theme.combo(new String[]{"Male", "Female", "Other"});
            g.gridx = 3; g.weightx = 0;
            rows.add(gender, g);

            nameFields.put(i, name);
            ageFields.put(i, age);
```

#	src/main/java/com/airline/ui/BookingDialog.java · Java · 314 lines

```
            genderFields.put(i, gender);
        }

        g.gridx = 0; g.gridy = travellers + 1; g.gridwidth = 4; g.weightx = 1;
        rows.add(Theme.muted("Number of people in this booking: " + travellers +
                (travellers == 1 ? " person" : " people"), 11), g);
        return rows;
    }

    private void updateTotal() {
        double total = flight.price * travellers;
        totalLabel.setText("Total: ₹" + String.format("%,.0f", total) + "  (" + travellers +
                (travellers == 1 ? " person × " : " people × ") + "₹" + String.format("%,.0f", fligh
t.price) + ")");
        if (selectedSeats.size() == travellers) {
            seatInfo.setText("Seats selected: " + String.join(", ", selectedSeats));
            seatInfo.setForeground(Theme.OK);
        } else {
            seatInfo.setText("Pick " + (travellers - selectedSeats.size()) + " more seat(s) for " +
travellers + (travellers == 1 ? " person" : " people") + ".");
            seatInfo.setForeground(Theme.MUTED);
        }
    }

    private void confirm(JButton pay) {
        if (selectedSeats.size() != travellers) {
            JOptionPane.showMessageDialog(this, "Please select " + travellers + " seat(s).");
            return;
        }
        List<Map<String, Object>> pax = new ArrayList<>();
        for (int i = 0; i < travellers; i++) {
            String name = nameFields.get(i).getText().trim();
            if (name.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Enter passenger name for P" + (i + 1) + ".");
                return;
            }
            String ageText = ageFields.get(i).getText().trim();
            if (ageText.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Enter age for P" + (i + 1) + ".");
                ageFields.get(i).requestFocusInWindow();
                return;
            }
            int age;
            try { age = Integer.parseInt(ageText); }
            catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(this, "Age for P" + (i + 1) + " must be a number.");
                ageFields.get(i).requestFocusInWindow();
                return;
            }
            if (age < 1 || age > 120) {
                JOptionPane.showMessageDialog(this, "Age for P" + (i + 1) + " must be between 1 and
120.");
                ageFields.get(i).requestFocusInWindow();
                return;
            }
            Map<String, Object> m = new LinkedHashMap<>();
            m.put("name", name);
            m.put("age", ageText);
            m.put("gender", (String) genderFields.get(i).getSelectedItem());
            pax.add(m);
        }
        String card = cardField.getText().replaceAll("\\s", "");
        if (card.length() < 12) {
            JOptionPane.showMessageDialog(this, "Enter a valid test card number (12+ digits).");
            return;
        }
        double total = flight.price * travellers;
        Map<String, Object> payment = bookings.mockPay(total, card.substring(card.length() - 4));
        BookingService.Result r = bookings.create(userId, flight.id, pax, new ArrayList<>(selectedSe
ats),
```

src/main/java/com/airline/ui/BookingDialog.java · Java · 314 lines

```
                String.valueOf(payment.get("paymentId")), total);
        if (r.error != null) {
            JOptionPane.showMessageDialog(this, r.error);
            renderSeats();
            return;
        }
        dispose();
        JOptionPane.showMessageDialog(getOwner(),
                "Booking submitted.\n\nPNR: " + r.booking.pnr + "\nStatus: PENDING — waiting for admin confirmation.\n" +
                        "Your e-ticket is issued as soon as an admin approves this booking.",
                "Booking pending approval", JOptionPane.INFORMATION_MESSAGE);
        new TicketDialog(getOwner(), r.booking, flight).setVisible(true);
    }
}

```

src/main/java/com/airline/ui/BookingsPanel.java · Java · 138 lines

```
package com.airline.ui;

import com.airline.model.Booking;
import com.airline.model.Flight;
import com.airline.service.BookingService;
import com.airline.service.FlightService;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.List;

/** "My bookings" tab: list of bookings with view-ticket and cancel actions. */
public class BookingsPanel extends JPanel {
    public interface OnChanged { void changed(); }

    private final BookingService bookings;
    private final FlightService flights;
    private final String userId;
    private final boolean admin;
    private final OnChanged onChanged;
    private final JPanel list = new JPanel();
    private final JLabel count = Theme.muted("", 12);

    public BookingsPanel(BookingService bookings, FlightService flights, String userId, boolean admin, OnChanged onChanged) {
        this.bookings = bookings;
        this.flights = flights;
        this.userId = userId;
        this.admin = admin;
        this.onChanged = onChanged;

        setLayout(new BorderLayout(0, 10));
        setBackground(Theme.BG);
        setBorder(Theme.pad(14, 14, 14, 14));

        list.setLayout(new BoxLayout(list, BoxLayout.Y_AXIS));
        list.setOpaque(false);

        JPanel head = new JPanel(new BorderLayout());
        head.setOpaque(false);
        head.add(Theme.title(admin ? "All bookings" : "My bookings", 17), BorderLayout.WEST);
        JPanel right = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
        right.setOpaque(false);
        right.add(count);
        JButton refresh = Theme.button("↻ Refresh");
        refresh.setToolTipText("Check whether an admin has confirmed a pending booking");
        refresh.addActionListener(e -> { refresh(); onChanged.changed(); });
        right.add(refresh);
        head.add(right, BorderLayout.EAST);

        JScrollPane sc = new JScrollPane(list);
```

src/main/java/com/airline/ui/BookingsPanel.java · Java · 138 lines

```
        sc.setBorder(BorderFactory.createEmptyBorder());
        sc.setOpaque(false);
        sc.getViewport().setOpaque(false);
        sc.getVerticalScrollBar().setUnitIncrement(16);

        JPanel center = new JPanel(new BorderLayout(0, 8));
        center.setOpaque(false);
        center.add(head, BorderLayout.NORTH);
        center.add(sc, BorderLayout.CENTER);
        add(center, BorderLayout.CENTER);
    }

    public void refresh() {
        list.removeAll();
        List<Booking> all = admin ? bookings.all() : bookings.forUser(userId);
        count.setText(all.size() + " booking" + (all.size() == 1 ? "" : "s"));
        if (all.isEmpty()) {
            JPanel empty = Theme.card(new BorderLayout());
            empty.add(Theme.muted("No bookings yet. Search a flight and book a seat.", 13), BorderLa
yout.CENTER);
            empty.setAlignmentX(Component.LEFT_ALIGNMENT);
            list.add(empty);
        } else {
            for (Booking b : all) list.add(card(b));
        }
        list.revalidate();
        list.repaint();
    }

    private JPanel card(Booking b) {
        Flight f = flights.get(b.flightId);
        boolean pending = "PENDING".equals(b.status);
        JPanel p = new JPanel(new BorderLayout(14, 6));
        p.setBackground(Theme.CARD);
        p.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createMatteBorder(0, 3, 0, 0,
                        pending ? new Color(0xD97706) : b.status.equals("CONFIRMED") ? Theme.OK : Th
eme.BAD),
                Theme.pad(12, 14, 12, 14)));
        p.setAlignmentX(Component.LEFT_ALIGNMENT);
        p.setMaximumSize(new Dimension(Integer.MAX_VALUE, 96));

        JPanel left = new JPanel(new GridLayout(0, 1));
        left.setOpaque(false);
        JLabel route = Theme.title((f == null ? b.flightId : f.from + "  →  " + f.to), 17);
        String who = b.passengers.isEmpty() ? "" : String.valueOf(b.passengers.get(0).getOrDefault
("name", ""));
        JLabel meta = Theme.muted("PNR " + b.pnr + (who.isEmpty() ? "" : " · " + who) +
                " · Seats " + String.join(", ", b.seats) +
                " · " + (f == null ? "" : f.date + " " + f.departure) +
                " · ₹" + String.format("%,.0f", b.totalPrice), 12);
        left.add(route);
        left.add(meta);
        if (pending) {
            JLabel note = Theme.muted("Waiting for admin confirmation — the e-ticket is issued once
approved.", 11);
            note.setForeground(new Color(0x92400E));
            left.add(note);
        } else if ("CONFIRMED".equals(b.status) && b.confirmedAt != null && !b.confirmedAt.isEmpty
()) {
            left.add(Theme.muted("Confirmed on " + b.confirmedAt.replace('T', ' ').substring(0, 16),
11));
        }

        JPanel right = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
        right.setOpaque(false);
        right.add(Theme.statusPill(b.status));
        JButton view = Theme.button(pending ? "View status" : "View ticket");
        view.addActionListener(e -> {
```

src/main/java/com/airline/ui/BookingsPanel.java · Java · 138 lines

```
            if (f != null) new TicketDialog(SwingUtilities.getWindowAncestor(this), b, f).setVisible
(true);
            else JOptionPane.showMessageDialog(this, "Flight details unavailable.");
        });
        right.add(view);
        if (!b.status.equals("CANCELLED")) {
            JButton cancel = Theme.danger("Cancel");
            cancel.addActionListener(e -> {
                int ok = JOptionPane.showConfirmDialog(this, "Cancel booking " + b.pnr + "? Seats ar
e released.",
                        "Cancel booking", JOptionPane.YES_NO_OPTION);
                if (ok != JOptionPane.YES_OPTION) return;
                BookingService.Result r = bookings.cancel(b.pnr, userId, admin);
                if (r.error != null) JOptionPane.showMessageDialog(this, r.error);
                else JOptionPane.showMessageDialog(this, "Booking " + b.pnr + " cancelled. Refund mo
cked.");
                refresh();
                onChanged.changed();
            });
            right.add(cancel);
        }

        p.add(left, BorderLayout.CENTER);
        p.add(right, BorderLayout.EAST);
        return p;
    }
}

```

src/main/java/com/airline/ui/TicketDialog.java · Java · 118 lines

```
package com.airline.ui;

import com.airline.model.Booking;
import com.airline.model.Flight;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.print.PrinterException;

/** E-ticket window with PNR, passenger table and print support. */
public class TicketDialog extends JDialog {
    public TicketDialog(Window owner, Booking booking, Flight flight) {
        super(owner, "E-Ticket", ModalityType.APPLICATION_MODAL);
        setLayout(new BorderLayout(10, 10));
        getContentPane().setBackground(Theme.BG);
        setSize(620, 540);
        setLocationRelativeTo(owner);

        JPanel card = new JPanel(new BorderLayout(0, 10));
        card.setBackground(new Color(0xF0F9FF));
        card.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createLineBorder(Theme.PRIMARY, 2, true), Theme.pad(16, 16, 16, 16)));

        JPanel head = new JPanel(new BorderLayout());
        head.setOpaque(false);
        JLabel pnr = Theme.title("PNR  " + booking.pnr, 24);
        pnr.setForeground(Theme.PRIMARY_DARK);
        head.add(pnr, BorderLayout.NORTH);
        JLabel sub = Theme.muted(flight.airline + " · " + flight.flightNumber + " · " + flight.from
+ " → " + flight.to +
                " · " + flight.date + " · " + flight.departure + "–" + flight.arrival, 12);
        head.add(sub, BorderLayout.SOUTH);

        boolean confirmed = "CONFIRMED".equals(booking.status);
        JPanel banner = new JPanel(new BorderLayout(8, 0));
        banner.setOpaque(true);
        banner.setBackground(Theme.statusBg(booking.status));
        banner.setBorder(BorderFactory.createCompoundBorder(
```

```
38                 BorderFactory.createMatteBorder(0, 0, 1, 0, Theme.statusFg(booking.status)), Theme.p
ad(8, 10, 8, 10)));
39         String bannerText = confirmed
40                 ? "Confirmed by the airline — this e-ticket is valid for travel."
41                 : "PENDING — this booking is waiting for admin confirmation. The e-ticket is issued
only after approval.";
42         JLabel bannerTextLabel = Theme.label(bannerText);
43         bannerTextLabel.setForeground(Theme.statusFg(booking.status));
44         banner.add(bannerTextLabel, BorderLayout.CENTER);
45
46         String[] cols = {"Passenger", "Age", "Gender", "Seat"};
47         DefaultTableModel model = new DefaultTableModel(cols, 0);
48         for (int i = 0; i < booking.passengers.size(); i++) {
49             java.util.Map<String, Object> p = booking.passengers.get(i);
50             String seat = i < booking.seats.size() ? booking.seats.get(i) : "";
51             model.addRow(new Object[]{p.getOrDefault("name", ""), p.getOrDefault("age", ""),
52                     p.getOrDefault("gender", ""), seat});
53         }
54         JTable table = new JTable(model);
55         table.setFont(Theme.font(13));
56         table.setRowHeight(24);
57         table.setBackground(Color.WHITE);
58         table.setGridColor(Theme.LINE);
59         table.setShowGrid(true);
60         JScrollPane sc = new JScrollPane(table);
61         sc.setBorder(BorderFactory.createLineBorder(Theme.LINE));
62
63         JPanel foot = new JPanel(new BorderLayout());
64         foot.setOpaque(false);
65         JLabel total = Theme.title("Total paid: ₹" + String.format("%,.0f", booking.totalPrice), 1
5);
66         total.setForeground(Theme.PRIMARY_DARK);
67         JLabel st = Theme.statusPill(booking.status);
68         foot.add(total, BorderLayout.WEST);
69         foot.add(st, BorderLayout.EAST);
70
71         JPanel top = new JPanel(new BorderLayout(0, 8));
72         top.setOpaque(false);
73         top.add(head, BorderLayout.NORTH);
74         top.add(banner, BorderLayout.SOUTH);
75         card.add(top, BorderLayout.NORTH);
76         card.add(sc, BorderLayout.CENTER);
77         card.add(foot, BorderLayout.SOUTH);
78         add(card, BorderLayout.CENTER);
79
80         JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
81         buttons.setOpaque(false);
82         JButton print = Theme.button(confirmed ? "🖨  Print ticket" : "🖨  Print (after approval)");
83         print.setEnabled(confirmed);
84         if (!confirmed) print.setToolTipText("Printing unlocks once the admin confirms this bookin
g.");
85         print.addActionListener(e -> printTicket(booking, flight));
86         JButton close = Theme.primary("Close");
87         close.addActionListener(e -> dispose());
88         buttons.add(print);
89         buttons.add(close);
90         add(buttons, BorderLayout.SOUTH);
91     }
92
93     private void printTicket(Booking b, Flight f) {
94         JTextArea area = new JTextArea();
95         area.setFont(Theme.font(12));
96         area.setText("AIROPLANE TICKET SERVICE\n" +
97                 "--------------------------------------\n" +
98                 "PNR: " + b.pnr + "        Status: " + b.status + "\n" +
99                 "Flight: " + f.airline + " " + f.flightNumber + "\n" +
100                "Route: " + f.from + " -> " + f.to + "\n" +
101                "Date: " + f.date + "   " + f.departure + " - " + f.arrival + "\n" +
102                "Payment: " + b.paymentId + "\n" +
103                "--------------------------------------\n");
```

src/main/java/com/airline/ui/TicketDialog.java · Java · 118 lines

```
        for (int i = 0; i < b.passengers.size(); i++) {
            java.util.Map<String, Object> p = b.passengers.get(i);
            area.append(String.format("%-20s %-4s %-7s Seat %s%n", p.getOrDefault("name", ""),
                    p.getOrDefault("age", ""), p.getOrDefault("gender", ""),
                    i < b.seats.size() ? b.seats.get(i) : ""));
        }
        area.append("-----------------------------------\nTotal: Rs. " + String.format("%,.0f",
b.totalPrice) + "\n");
        try {
            area.print();
        } catch (PrinterException ex) {
            JOptionPane.showMessageDialog(this, "Could not print: " + ex.getMessage());
        }
    }
}

```

src/main/java/com/airline/ui/AdminPanel.java · Java · 343 lines

```
package com.airline.ui;

import com.airline.model.Booking;
import com.airline.model.Flight;
import com.airline.model.User;
import com.airline.service.BookingService;
import com.airline.service.FlightService;
import com.airline.store.DataStore;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** Admin tab: stats, flight CRUD and all-bookings management. */
public class AdminPanel extends JPanel {
    public interface OnChanged { void changed(); }

    private final DataStore db;
    private final FlightService flights;
    private final BookingService bookings;
    private final OnChanged onChanged;

    private final JPanel statsRow = new JPanel(new GridLayout(1, 6, 10, 0));
    private final JTable flightTable;
    private final DefaultTableModel flightModel;
    private final JTable bookingTable;
    private final DefaultTableModel bookingModel;
    private final JTextField searchField = Theme.field(20);

    public AdminPanel(DataStore db, FlightService flights, BookingService bookings, OnChanged onChan
ged) {
        this.db = db;
        this.flights = flights;
        this.bookings = bookings;
        this.onChanged = onChanged;

        setLayout(new BorderLayout(0, 10));
        setBackground(Theme.BG);
        setBorder(Theme.pad(14, 14, 14, 14));

        statsRow.setOpaque(false);
        add(statsRow, BorderLayout.NORTH);

        flightModel = new DefaultTableModel(new Object[]{"Flight", "Airline", "Route", "Date", "Tim
e", "Price", "Seats"}, 0) {
            @Override public boolean isCellEditable(int r, int c) { return false; }
        };
        flightTable = new JTable(flightModel);
        style(flightTable);
```

#	src/main/java/com/airline/ui/AdminPanel.java · Java · 343 lines

```
50
51          bookingModel = new DefaultTableModel(new Object[]{"PNR", "Flight", "Seats", "Total", "Statu
    s", "Booked at"}, 0) {
52              @Override public boolean isCellEditable(int r, int c) { return false; }
53          };
54          bookingTable = new JTable(bookingModel);
55          style(bookingTable);
56
57          JTabbedPane tabs = new JTabbedPane();
58          tabs.setFont(Theme.bold(13));
59          tabs.addTab("✈ Flights", flightsTab());
60          tabs.addTab("🎫 Bookings", bookingsTab());
61          add(tabs, BorderLayout.CENTER);
62      }
63
64      private void style(JTable t) {
65          t.setFont(Theme.font(13));
66          t.setRowHeight(26);
67          t.setBackground(Color.WHITE);
68          t.setGridColor(Theme.LINE);
69          t.setSelectionBackground(new Color(0xE0F2FE));
70          t.setSelectionForeground(Theme.INK);
71          t.setAutoResizeMode(JTable.AUTO_RESIZE_SUBSEQUENT_COLUMNS);
72      }
73
74      private JPanel flightsTab() {
75          JPanel p = new JPanel(new BorderLayout(0, 10));
76          p.setBackground(Theme.BG);
77          p.setBorder(Theme.pad(12, 12, 12, 12));
78
79          JPanel toolbar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
80          toolbar.setOpaque(false);
81          toolbar.add(Theme.label("Search"));
82          searchField.setPreferredSize(new Dimension(240, 34));
83          toolbar.add(searchField);
84          JButton doSearch = Theme.button("Search");
85          doSearch.addActionListener(e -> refreshFlights());
86          JButton clear = Theme.button("Reset");
87          clear.addActionListener(e -> { searchField.setText(""); refreshFlights(); });
88          toolbar.add(doSearch);
89          toolbar.add(clear);
90          toolbar.add(Box.createHorizontalStrut(20));
91          JButton add = Theme.primary("+ Add flight");
92          add.addActionListener(e -> new FlightDialog(SwingUtilities.getWindowAncestor(this), flights,
    null).setVisible(true));
93          toolbar.add(add);
94          JButton edit = Theme.button("Edit selected");
95          edit.addActionListener(e -> editSelected());
96          toolbar.add(edit);
97          JButton del = Theme.danger("Delete");
98          del.addActionListener(e -> deleteSelected());
99          toolbar.add(del);
100
101         JScrollPane sc = new JScrollPane(flightTable);
102         sc.setBorder(BorderFactory.createLineBorder(Theme.LINE));
103         p.add(toolbar, BorderLayout.NORTH);
104         p.add(sc, BorderLayout.CENTER);
105         return p;
106     }
107
108     private JPanel bookingsTab() {
109         JPanel p = new JPanel(new BorderLayout(0, 10));
110         p.setBackground(Theme.BG);
111         p.setBorder(Theme.pad(12, 12, 12, 12));
112         JPanel toolbar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
113         toolbar.setOpaque(false);
114         JButton refresh = Theme.button("↻ Refresh");
115         refresh.addActionListener(e -> refreshBookings());
116         JButton confirm = Theme.primary("✓ Confirm booking");
117         confirm.setToolTipText("Approve the selected pending booking and issue its e-ticket");
```

```
        confirm.addActionListener(e -> confirmSelectedBooking());
        JButton cancel = Theme.danger("Cancel selected booking");
        cancel.addActionListener(e -> cancelSelectedBooking());
        toolbar.add(refresh);
        toolbar.add(confirm);
        toolbar.add(cancel);
        JScrollPane sc = new JScrollPane(bookingTable);
        sc.setBorder(BorderFactory.createLineBorder(Theme.LINE));
        p.add(toolbar, BorderLayout.NORTH);
        p.add(sc, BorderLayout.CENTER);
        return p;
    }

    public void refresh() {
        int active = 0;
        int pending = 0;
        double revenue = 0;
        for (Booking b : db.bookings.values()) {
            if (b.status.equals("CONFIRMED")) { revenue += b.totalPrice; active++; }
            else if (b.status.equals("PENDING")) pending++;
        }
        statsRow.removeAll();
        statsRow.add(stat("Flights", String.valueOf(db.flights.size())));
        statsRow.add(stat("Users", String.valueOf(db.users.size())));
        statsRow.add(stat("Bookings", String.valueOf(db.bookings.size())));
        statsRow.add(stat("Active", String.valueOf(active)));
        statsRow.add(stat("Pending", String.valueOf(pending)));
        statsRow.add(stat("Revenue", "₹" + String.format("%,.0f", revenue)));
        statsRow.revalidate();
        statsRow.repaint();
        refreshFlights();
        refreshBookings();
    }

    private JPanel stat(String label, String value) {
        JPanel p = Theme.card(new GridLayout(0, 1));
        JLabel l = Theme.muted(label, 12);
        JLabel v = Theme.title(value, 20);
        v.setForeground(Theme.PRIMARY_DARK);
        p.add(l);
        p.add(v);
        return p;
    }

    private void refreshFlights() {
        String q = searchField.getText().trim().toLowerCase();
        flightModel.setRowCount(0);
        for (Flight f : flights.search(null, null, null)) {
            String hay = (f.flightNumber + " " + f.airline + " " + f.from + " " + f.to + " " + f.dat
e).toLowerCase();
            if (!q.isEmpty() && !hay.contains(q)) continue;
            flightModel.addRow(new Object[]{f.flightNumber, f.airline, f.from + " → " + f.to, f.dat
e,
                    f.departure + "–" + f.arrival, "₹" + String.format("%,.0f", f.price),
                    f.availableSeats() + "/" + f.totalSeats});
        }
    }

    private void refreshBookings() {
        bookingModel.setRowCount(0);
        for (Booking b : bookings.all()) {
            bookingModel.addRow(new Object[]{b.pnr, b.flightId, String.join(", ", b.seats),
                    "₹" + String.format("%,.0f", b.totalPrice), b.status, b.createdAt});
        }
    }

    /** Approves the selected pending booking — the e-ticket is issued only after this. */
    private void confirmSelectedBooking() {
        int r = bookingTable.getSelectedRow();
        if (r < 0) { JOptionPane.showMessageDialog(this, "Select a booking first."); return; }
```

src/main/java/com/airline/ui/AdminPanel.java · Java · 343 lines

```
        String pnr = (String) bookingTable.getValueAt(r, 0);
        Booking b = db.bookings.get(pnr);
        if (b == null) { JOptionPane.showMessageDialog(this, "Booking not found."); return; }
        if (!"PENDING".equals(b.status)) {
            JOptionPane.showMessageDialog(this, "Booking " + pnr + " is " + b.status + " — only pending bookings can be confirmed.");
            return;
        }
        int ok = JOptionPane.showConfirmDialog(this,
                "Confirm booking " + pnr + " for ₹" + String.format("%,.0f", b.totalPrice) + "?\nThe e-ticket is issued now.",
                "Confirm booking", JOptionPane.YES_NO_OPTION);
        if (ok != JOptionPane.YES_OPTION) return;
        BookingService.Result res = bookings.confirm(pnr, true);
        if (res.error != null) JOptionPane.showMessageDialog(this, res.error);
        else JOptionPane.showMessageDialog(this, "Booking " + pnr + " confirmed. E-ticket issued.");
        refresh();
        onChanged.changed();
    }

    private void editSelected() {
        int r = flightTable.getSelectedRow();
        if (r < 0) { JOptionPane.showMessageDialog(this, "Select a flight first."); return; }
        String id = (String) flightTable.getValueAt(r, 0);
        for (Flight f : flights.search(null, null, null)) {
            if (f.flightNumber.equals(id)) { new FlightDialog(SwingUtilities.getWindowAncestor(this), flights, f).setVisible(true); return; }
        }
    }

    private void deleteSelected() {
        int r = flightTable.getSelectedRow();
        if (r < 0) { JOptionPane.showMessageDialog(this, "Select a flight first."); return; }
        String id = (String) flightTable.getValueAt(r, 0);
        for (Flight f : flights.search(null, null, null)) {
            if (!f.flightNumber.equals(id)) continue;
            int ok = JOptionPane.showConfirmDialog(this, "Delete flight " + f.flightNumber + "?",
                    "Delete flight", JOptionPane.YES_NO_OPTION);
            if (ok != JOptionPane.YES_OPTION) return;
            if (!flights.delete(f.id)) {
                JOptionPane.showMessageDialog(this, "Cannot delete: flight has active bookings.");
                return;
            }
            refresh();
            onChanged.changed();
            return;
        }
    }

    private void cancelSelectedBooking() {
        int r = bookingTable.getSelectedRow();
        if (r < 0) { JOptionPane.showMessageDialog(this, "Select a booking first."); return; }
        String pnr = (String) bookingTable.getValueAt(r, 0);
        int ok = JOptionPane.showConfirmDialog(this, "Cancel booking " + pnr + "?", "Cancel booking",
                JOptionPane.YES_NO_OPTION);
        if (ok != JOptionPane.YES_OPTION) return;
        BookingService.Result res = bookings.cancel(pnr, "", true);
        if (res.error != null) JOptionPane.showMessageDialog(this, res.error);
        refresh();
        onChanged.changed();
    }

    /** Add / edit flight form. */
    static class FlightDialog extends JDialog {
        private final FlightService flights;
        private final Flight editing;
        private final JTextField airline = Theme.field(16);
        private final JTextField number = Theme.field(10);
        private final JTextField from = Theme.field(5);
```

```
252         private final JTextField to = Theme.field(5);
253         private final JTextField date = Theme.field(10);
254         private final JTextField dep = Theme.field(6);
255         private final JTextField arr = Theme.field(6);
256         private final JTextField price = Theme.field(8);
257 
258         FlightDialog(java.awt.Window owner, FlightService flights, Flight editing) {
259             super(owner, editing == null ? "Add flight" : "Edit flight " + editing.flightNumber, Mod
    alityType.APPLICATION_MODAL);
260             this.flights = flights;
261             this.editing = editing;
262             setLayout(new BorderLayout(10, 10));
263             getContentPane().setBackground(Theme.BG);
264             setSize(460, 420);
265             setLocationRelativeTo(owner);
266 
267             JPanel form = Theme.card(new GridBagLayout());
268             GridBagConstraints g = new GridBagConstraints();
269             g.insets = new Insets(6, 4, 6, 4);
270             g.anchor = GridBagConstraints.WEST;
271             g.fill = GridBagConstraints.HORIZONTAL;
272 
273             int y = 0;
274             y = addRow(form, g, y, "Airline", airline);
275             y = addRow(form, g, y, "Flight number", number);
276             y = addRow(form, g, y, "From (IATA)", from);
277             y = addRow(form, g, y, "To (IATA)", to);
278             y = addRow(form, g, y, "Date (yyyy-mm-dd)", date);
279             y = addRow(form, g, y, "Departure (HH:mm)", dep);
280             y = addRow(form, g, y, "Arrival (HH:mm)", arr);
281             y = addRow(form, g, y, "Price (₹)", price);
282 
283             JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
284             buttons.setOpaque(false);
285             JButton cancel = Theme.button("Cancel");
286             cancel.addActionListener(e -> dispose());
287             JButton save = Theme.primary("Save flight");
288             save.addActionListener(e -> save());
289             buttons.add(cancel);
290             buttons.add(save);
291 
292             add(form, BorderLayout.CENTER);
293             add(buttons, BorderLayout.SOUTH);
294 
295             if (editing != null) {
296                 airline.setText(editing.airline);
297                 number.setText(editing.flightNumber);
298                 from.setText(editing.from);
299                 to.setText(editing.to);
300                 date.setText(editing.date);
301                 dep.setText(editing.departure);
302                 arr.setText(editing.arrival);
303                 price.setText(String.valueOf((long) editing.price));
304             } else {
305                 airline.setText("Airoplane Airways");
306                 date.setText(java.time.LocalDate.now().plusDays(1).toString());
307                 dep.setText("09:00");
308                 arr.setText("11:00");
309             }
310         }
311 
312         private int addRow(JPanel form, GridBagConstraints g, int y, String label, JTextField field)
    {
313             g.gridx = 0; g.gridy = y; g.weightx = 0.35;
314             form.add(Theme.label(label), g);
315             g.gridx = 1; g.weightx = 0.65;
316             form.add(field, g);
317             return y + 1;
318         }
319 
```

src/main/java/com/airline/ui/AdminPanel.java · Java · 343 lines

```java
        private void save() {
            if (airline.getText().isBlank() || number.getText().isBlank() || from.getText().isBlank()
                    || to.getText().isBlank() || date.getText().isBlank() || price.getText().isBlank()) {
                JOptionPane.showMessageDialog(this, "Fill all required fields.");
                return;
            }
            Map<String, Object> m = new LinkedHashMap<>();
            m.put("airline", airline.getText().trim());
            m.put("flightNumber", number.getText().trim().toUpperCase());
            m.put("from", from.getText().trim().toUpperCase());
            m.put("to", to.getText().trim().toUpperCase());
            m.put("date", date.getText().trim());
            m.put("departure", dep.getText().trim());
            m.put("arrival", arr.getText().trim());
            m.put("durationMin", 120);
            m.put("price", Double.parseDouble(price.getText().trim()));
            m.put("totalSeats", 180);
            if (editing == null) flights.create(m);
            else flights.update(editing.id, m);
            dispose();
        }
    }
}

```

Part 2 — Web and Progressive Web App Front End

src/main/resources/static/index.html · HTML · 200 lines

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0, viewport-fit=cover" />
<meta name="theme-color" content="#0ea5e9" />
<meta name="mobile-web-app-capable" content="yes" />
<meta name="apple-mobile-web-app-capable" content="yes" />
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent" />
<meta name="apple-mobile-web-app-title" content="Airoplane Ticket Service" />
<link rel="manifest" href="/manifest.webmanifest" />
<link rel="icon" type="image/png" sizes="192x192" href="/icon-192.png" />
<link rel="apple-touch-icon" href="/apple-touch-icon.png" />
<title>Airoplane Ticket Service</title>
<link rel="stylesheet" href="/styles.css" />
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
</head>
<body>
<header class="nav">
  <div class="nav-inner">
    <div class="brand"><span class="plane">✈</span> Airoplane Ticket Service</div>
    <nav class="links">
      <button data-nav="search" class="active">Search</button>
      <button data-nav="airports">Airports</button>
      <button data-nav="bookings">My Bookings</button>
      <button data-nav="admin" id="adminNav" style="display:none">Admin</button>
    </nav>
    <div class="auth-area">
      <span id="userChip" class="user-chip" style="display:none"></span>
      <button id="serverBtn" class="btn btn-sm" title="Server">⚙</button>
      <button id="authBtn" class="btn btn-sm">Login</button>
    </div>
  </div>
</header>

<main class="wrap">
```

#	src/main/resources/static/index.html · HTML · 200 lines

```
  <!-- SEARCH -->
  <section id="view-search" class="view">
    <div class="hero">
      <div class="sky-anim" aria-hidden="true">
        <span class="cloud c1"></span>
        <span class="cloud c2"></span>
        <span class="cloud c3"></span>
        <span class="jet">✈</span>
      </div>
      <h1>Book domestic flights in minutes</h1>
      <p>Search → pick seats → pay (mock) → get instant e-ticket with PNR.</p>
    </div>
    <form id="searchForm" class="card search-card">
      <div class="grid-4">
        <label>From
          <select id="fromSel"></select>
        </label>
        <button type="button" id="swapBtn" class="swap" title="Swap">⇄</button>
        <label>To
          <select id="toSel"></select>
        </label>
        <label>Date
          <input type="date" id="dateInp" required />
        </label>
        <label>Travellers
          <select id="paxSel"><option>1</option><option>2</option><option>3</option><option>4</optio
n><option>5</option><option>6</option></select>
        </label>
      </div>
      <button class="btn btn-primary" type="submit">🔍 Search flights</button>
    </form>
    <div id="results" class="results"></div>
    </section>

  <!-- AIRPORTS + MAP -->
  <section id="view-airports" class="view" style="display:none">
    <div class="hero hero-sm">
      <h1>Airport locations</h1>
      <p>All airports served by Airoplane Ticket Service, plotted on the map.</p>
    </div>
    <div class="map-grid">
      <div class="card map-card">
        <div id="indiaMap"></div>
      </div>
      <div id="airportList" class="results" style="margin-top:0"></div>
    </div>
  </section>

  <!-- BOOKINGS -->
  <section id="view-bookings" class="view" style="display:none">
    <h2>My bookings</h2>
    <p class="muted">Login to see your e-tickets. Cancel anytime for instant refund (mock).</p>
    <div id="myBookings" class="results"></div>
  </section>

  <!-- ADMIN -->
  <section id="view-admin" class="view" style="display:none">
    <h2>Admin dashboard</h2>
    <div id="adminGate"></div>
    <div id="adminBody">
    <div id="statsRow" class="stats"></div>
    <div class="tabs admin-tabs">
      <button id="tabFlights" class="active">✈ Flights</button>
      <button id="tabBookings">🎫 Bookings</button>
    </div>
    <div id="adminFlightsPane">
    <div class="card admin-toolbar">
      <input id="adminSearch" placeholder="Search flight no., airline, route..." />
      <button class="btn btn-sm" id="adminSearchBtn">Search</button>
      <button class="btn btn-sm" id="adminResetBtn" type="button">Reset</button>
```

#	src/main/resources/static/index.html · HTML · 200 lines

```
    </div>
    <div class="card">
      <h3 id="flightFormTitle">Add flight</h3>
      <form id="flightForm" class="grid-4">
        <input type="hidden" id="fId" />
        <label>Airline<input id="fAirline" required placeholder="Airoplane Airways" /></label>
        <label>Flight no.<input id="fNum" required placeholder="SJ-101" /></label>
        <label>From<input id="fFrom" required placeholder="DEL" maxlength="3" /></label>
        <label>To<input id="fTo" required placeholder="BOM" maxlength="3" /></label>
        <label>Date<input id="fDate" type="date" required /></label>
        <label>Departure<input id="fDep" type="time" required /></label>
        <label>Arrival<input id="fArr" type="time" required /></label>
        <label>Price (₹)<input id="fPrice" type="number" min="500" required /></label>
        <div class="form-actions">
          <button class="btn btn-primary" type="submit">Save flight</button>
          <button class="btn" type="button" id="fReset">Reset</button>
        </div>
      </form>
    </div>
    <div id="adminFlights" class="results"></div>
    </div>
    <div id="adminBookingsPane" style="display:none">
      <div class="card admin-toolbar">
        <input id="bookingSearch" placeholder="Search PNR, flight, status..." />
        <button class="btn btn-sm" id="bookingSearchBtn">Search</button>
        <button class="btn btn-sm" id="bookingResetBtn" type="button">Reset</button>
        <button class="btn btn-sm" id="bookingRefreshBtn" type="button">↻ Refresh</button>
      </div>
      <div id="adminBookings" class="results"></div>
    </div>
    </div>
  </section>
</main>

<!-- BOOKING MODAL -->
<div id="bookModal" class="modal" style="display:none">
  <div class="modal-box wide">
    <div class="modal-head"><h3 id="bookTitle">Complete booking</h3><button class="x" data-close="bo
okModal">x</button></div>
    <div class="book-grid">
      <div>
        <h4>1. Select seats (<span id="seatNeed"></span> needed)</h4>
        <div id="seatMap" class="seatmap"></div>
        <div class="legend"><span class="seat"></span> free <span class="seat taken"></span> taken <
span class="seat sel"></span> selected</div>
      </div>
      <div>
        <h4>2. Passengers</h4>
        <div id="paxForms"></div>
        <h4>3. Payment (mock)</h4>
        <input id="cardInp" inputmode="numeric" placeholder="Test card 4111 1111 1111 1111" value="4
111 1111 1111 1111" />
        <div id="priceLine" class="price"></div>
        <button id="payBtn" class="btn btn-primary btn-block">Pay & confirm booking</button>
      </div>
    </div>
  </div>
</div>

<!-- TICKET MODAL -->
<div id="ticketModal" class="modal" style="display:none">
  <div class="modal-box">
    <div class="modal-head"><h3>🎫 E-Ticket</h3><button class="x" data-close="ticketModal">x</button
></div>
    <div id="ticketBody"></div>
    <div class="row">
      <button class="btn" onclick="window.print()">🖨️ Print</button>
      <button class="btn" data-close="ticketModal">Close</button>
    </div>
  </div>
```

src/main/resources/static/index.html · HTML · 200 lines

```
</div>

<!-- AUTH MODAL -->
<div id="authModal" class="modal" style="display:none">
  <div class="modal-box">
    <div class="modal-head"><h3 id="authTitle">Login</h3><button class="x" data-close="authModal">x</button></div>
    <div class="tabs"><button id="tabLogin" class="active">Login</button><button id="tabReg">Register</button></div>
    <form id="authForm" class="stack">
      <input id="aName" placeholder="Full name (register only)" style="display:none" />
      <input id="aEmail" type="email" required placeholder="Enter your email" value="" />
      <input id="aPass" type="password" required placeholder="Enter your password" value="" />
      <button class="btn btn-primary btn-block" type="submit">Continue</button>
      <p class="muted small">New here? Switch to Register to create an account.</p>
    </form>
  </div>
</div>

<div id="toast" class="toast"></div>
<footer class="foot">Airoplane Ticket Service · Java (JDK HttpServer) + HTML/CSS/JS · No external dependencies · Data stored in <code>data/*.json</code></footer>
<script>
if ('serviceWorker' in navigator && location.protocol.indexOf('http') === 0 && !window.Capacitor) {
  window.addEventListener('load', () => navigator.serviceWorker.register('/sw.js').catch(() => {}));
}
</script>
<script src="/app.js"></script>
</body>
</html>

```

src/main/resources/static/styles.css · CSS · 86 lines

```
:root{--bg:#eef2f7;--card:#fff;--ink:#0f172a;--mut:#64748b;--pri:#0ea5e9;--pri2:#0369a1;--ok:#16a34a;--bad:#dc2626;--line:#e2e8f0}
*{box-sizing:border-box}body{margin:0;font-family:Inter,system-ui,Segoe UI,Roboto,Arial;color:var(--ink);
background:var(--bg) url("/plane-bg.svg") no-repeat top center;background-size:1600px auto;background-attachment:fixed}
.nav{background:#0f172a;color:#fff;position:sticky;top:0;z-index:10}
.nav-inner{max-width:1080px;margin:auto;display:flex;align-items:center;gap:16px;padding:12px 16px}
.brand{font-weight:800;font-size:20px}.plane{color:#38bdf8}.dot{color:#38bdf8}
.links{display:flex;gap:8px;margin-left:8px}.links button{background:transparent;border:0;color:#cbd5e1;padding:8px 12px;border-radius:8px;cursor:pointer;font-weight:600}
.links button.active,.links button:hover{background:#1e293b;color:#fff}
.auth-area{margin-left:auto;display:flex;gap:8px;align-items:center}
.user-chip{background:#1e293b;padding:6px 10px;border-radius:20px;font-size:13px}
.wrap{max-width:1080px;margin:auto;padding:20px 16px 60px}
.hero{position:relative;overflow:hidden;border-radius:16px;padding:46px 22px 30px;margin:6px 0 14px;color:#fff;
background:#0c4a6e url("/plane-bg.svg") no-repeat center center;background-size:cover;
border:1px solid #0c4a6e;box-shadow:0 6px 24px rgba(2,6,23,.25)}
.hero::after{content:"";position:absolute;inset:0;background:linear-gradient(90deg,rgba(2,6,23,.62),rgba(2,6,23,.15) 60%,rgba(2,6,23,0));pointer-events:none}
.hero h1,.hero p{position:relative;z-index:1;text-shadow:0 1px 8px rgba(0,0,0,.4)}
.hero h1{margin:8px 0 4px;font-size:30px}.hero p{color:#e0f2fe;margin:0 0 4px}
.sky-anim{position:absolute;inset:0;overflow:hidden;pointer-events:none}
.sky-anim .jet{position:absolute;top:22%;left:-12%;font-size:44px;color:#fff;filter:drop-shadow(0 4px 8px rgba(0,0,0,.45));animation:flyAcross 9s linear infinite}
.sky-anim .cloud{position:absolute;background:#fff;opacity:.7;border-radius:999px;animation:drift linear infinite}
.sky-anim .cloud::before,.sky-anim .cloud::after{content:"";position:absolute;background:#fff;border-radius:50%}
.sky-anim .cloud::before{width:55%;height:160%;left:12%;top:-80%}
.sky-anim .cloud::after{width:38%;height:110%;right:14%;top:-45%}
.sky-anim .c1{width:120px;height:34px;top:18%;animation-duration:26s}
.sky-anim .c2{width:80px;height:24px;top:58%;animation-duration:38s;animation-delay:-12s;opacity:.5}
.sky-anim .c3{width:150px;height:40px;top:74%;animation-duration:48s;animation-delay:-25s;opacity:.35}
```

#	src/main/resources/static/styles.css · CSS · 86 lines
27	.brand .plane{display:inline-block;animation:bob 2.6s ease-in-out infinite}
28	@keyframes flyAcross{0%{transform:translate(0,26px) rotate(-4deg)}15%{transform:translate(18vw,0) rotate(0deg)}50%{transform:translate(55vw,-26px) rotate(2deg)}85%{transform:translate(95vw,-8px) rotate(0deg)}100%{transform:translate(125vw,14px) rotate(-4deg)}}
29	@keyframes drift{from{transform:translateX(110vw)}to{transform:translateX(-30vw)}}
30	@keyframes bob{0%,100%{transform:translateY(0) rotate(-6deg)}50%{transform:translateY(-3px) rotate(6deg)}}
31	@media(prefers-reduced-motion:reduce){.sky-anim .jet,.sky-anim .cloud,.brand .plane{animation:none}}
32	.card{background:var(--card);border:1px solid var(--line);border-radius:14px;padding:16px;box-shadow:0 2px 10px rgba(2,6,23,.05)}
33	.search-card{display:flex;flex-direction:column;gap:12px}
34	.grid-4{display:grid;grid-template-columns:1fr auto 1fr 1fr 1fr;gap:10px;align-items:end}
35	label{display:flex;flex-direction:column;gap:6px;font-size:13px;font-weight:600;color:#334155}
36	input,select{padding:10px 12px;border:1px solid var(--line);border-radius:10px;font-size:14px;background:#fff;color:var(--ink)}
37	.swap{border:1px solid var(--line);background:#fff;border-radius:10px;font-size:18px;cursor:pointer;padding:8px 12px}
38	.btn{border:1px solid var(--line);background:#fff;padding:10px 14px;border-radius:10px;cursor:pointer;font-weight:700}
39	.btn-primary{background:var(--pri);border-color:var(--pri);color:#fff}.btn-primary:hover{background:var(--pri2)}
40	.btn-sm{padding:7px 12px}.btn-block{width:100%;margin-top:10px}
41	.results{display:flex;flex-direction:column;gap:12px;margin-top:14px}
42	.flight{display:flex;gap:14px;align-items:center;flex-wrap:wrap}
43	.flight .route{font-size:20px;font-weight:800}.flight .meta{color:var(--mut);font-size:13px}
44	.flight .fare{margin-left:auto;text-align:right}.fare b{font-size:20px;color:var(--pri2)}
45	.pill{background:#f1f5f9;border:1px solid var(--line);border-radius:20px;padding:3px 10px;font-size:12px;font-weight:700}
46	.pill.green{background:#dcfce7;border-color:#bbf7d0;color:#166534}.pill.red{background:#fee2e2;border-color:#fecaca;color:#991b1b}
47	.pill.amber{background:#fef3c7;border-color:#fde68a;color:#92400e}
48	.pending-note{background:#fef3c7;border:1px solid #fde68a;color:#92400e;border-radius:10px;padding:10px 12px;margin:8px 0;font-size:13px}
49	.pending-note.small{font-size:12px;padding:6px 9px;margin:4px 0}
50	.muted{color:var(--mut)}.small{font-size:12px}
51	.stats{display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:10px;margin:12px 0}
52	.stat{background:#fff;border:1px solid var(--line);border-radius:12px;padding:12px}.stat b{font-size:22px;display:block}
53	.modal{position:fixed;inset:0;background:rgba(2,6,23,.55);display:flex;align-items:flex-start;justify-content:center;padding:24px;z-index:50;overflow:auto}
54	.modal-box{background:#fff;border-radius:14px;padding:18px;max-width:520px;width:100%}
55	.modal-box.wide{max-width:880px}
56	.modal-head{display:flex;justify-content:space-between;align-items:center}
57	.x{border:0;background:#f1f5f9;border-radius:8px;padding:6px 10px;cursor:pointer}
58	.book-grid{display:grid;grid-template-columns:1.2fr 1fr;gap:16px}
59	@media(max-width:800px){.book-grid{grid-template-columns:1fr}.grid-4{grid-template-columns:1fr 1fr}}
60	.seatmap{display:grid;grid-template-columns:repeat(7,1fr);gap:5px;max-height:300px;overflow:auto;background:#f8fafc;padding:10px;border-radius:10px;border:1px solid var(--line)}
61	.seat{border:1px solid #cbd5e1;background:#fff;border-radius:6px;font-size:11px;padding:6px 0;text-align:center;cursor:pointer}
62	.seat.taken{background:#e2e8f0;color:#94a3b8;cursor:not-allowed;text-decoration:line-through}
63	.seat.sel{background:var(--pri);color:#fff;border-color:var(--pri)}
64	.seat.aisle{border:0;background:transparent;cursor:default}
65	.legend{display:flex;gap:8px;align-items:center;font-size:12px;color:var(--mut);margin-top:8px}.legend .seat{width:26px;cursor:default}
66	.stack{display:flex;flex-direction:column;gap:10px}
67	.tabs{display:flex;gap:8px;margin-bottom:12px}.tabs button{flex:1;padding:9px;border-radius:10px;border:1px solid var(--line);background:#f8fafc;cursor:pointer;font-weight:700}.tabs button.active{background:#0f172a;color:#fff}
68	.price{font-size:18px;font-weight:800;margin-top:10px}
69	.row{display:flex;gap:8px;margin-top:12px}
70	.ticket{border:2px dashed #0ea5e9;border-radius:12px;padding:14px;background:#f0f9ff}
71	.ticket h2{margin:0}.pnr{font-size:26px;letter-spacing:3px;font-weight:900}
72	.toast{position:fixed;bottom:20px;left:50%;transform:translateX(-50%);background:#0f172a;color:#fff;padding:10px 16px;border-radius:10px;display:none;z-index:99}
73	.form-actions{display:flex;gap:8px;align-items:end}
74	.admin-tabs{margin:12px 0 4px}
75	.admin-toolbar{display:flex;gap:8px;margin:12px 0;align-items:center}
76	.admin-toolbar input{flex:1}
77	#adminGate:empty{display:none}

src/main/resources/static/styles.css · CSS · 86 lines

```
78 .hero-sm{padding:30px 22px 22px}
79 .map-grid{display:grid;grid-template-columns:1fr 1.2fr;gap:14px;margin-top:14px;align-items:start}
80 @media(max-width:800px){.map-grid{grid-template-columns:1fr}}
81 .map-card{padding:8px}
82 #indiaMap{width:100%;height:440px;border-radius:10px;z-index:0}
83 table{width:100%;border-collapse:collapse;font-size:13px}th,td{border-bottom:1px solid var(--line);p
   adding:8px;text-align:left}
84 .foot{text-align:center;color:var(--mut);font-size:12px;padding:20px}
85 @media print{.nav,.search-card,.hero,.foot,.row,.links,.auth-area{display:none!important}.modal{posi
   tion:static;background:none;padding:0}.modal-box{max-width:100%}}
86
```

src/main/resources/static/app.js · JavaScript · 364 lines

```
1  /* Airoplane Ticket Service frontend — vanilla JS */
2  const $ = (s) => document.querySelector(s);
3  const state = { token: localStorage.getItem('skyjet_token') || '', user: null, flights: [], bookingF
   light: null, seats: [], airports: {} };
4  // API base: same-origin on desktop; configurable for the mobile app (phone talks to laptop over LA
   N).
5  const isNativeApp = !!window.Capacitor;
6  let API = localStorage.getItem('skyjet_api') || '';
7  function setApiBase(url) { API = (url || '').trim().replace(/\/$/, ''); if (API) localStorage.setIte
   m('skyjet_api', API); else localStorage.removeItem('skyjet_api'); refreshServerBtn(); }
8
9  function toast(msg) { const t = $('#toast'); t.textContent = msg; t.style.display = 'block'; setTime
   out(() => t.style.display = 'none', 3200); }
10 function headers(json = true) { const h = {}; if (json) h['Content-Type'] = 'application/json'; if
   (state.token) h['Authorization'] = 'Bearer ' + state.token; return h; }
11 async function api(path, opts = {}) { const r = await fetch(API + path, opts); const j = await r.jso
   n().catch(() => ({ ok: false, error: 'Bad response' })); if (!r.ok || !j.ok) throw new Error(j.error
   || ('HTTP ' + r.status)); return j; }
12 const inr = (n) => '₹' + Number(n || 0).toLocaleString('en-IN');
13
14 // ---------- nav ----------
15 document.querySelectorAll('[data-nav]').forEach(b => b.onclick = () => showView(b.dataset.nav));
16 function showView(v) {
17   document.querySelectorAll('[data-nav]').forEach(b => b.classList.toggle('active', b.dataset.nav ==
   = v));
18   ['search', 'airports', 'bookings', 'admin'].forEach(k => $('#view-' + k).style.display = k === v ?
   'block' : 'none');
19   if (v === 'airports') renderAirports();
20   if (v === 'bookings') loadMyBookings();
21   if (v === 'admin') loadAdmin();
22 }
23
24 // ---------- airports + map (offline SVG, zero dependencies) ----------
25 const AIRPORT_INFO = {
26   DEL: { city: 'New Delhi', name: 'Indira Gandhi Intl', lat: 28.556, lng: 77.100 },
27   BOM: { city: 'Mumbai', name: 'Chhatrapati Shivaji Intl', lat: 19.090, lng: 72.865 },
28   BLR: { city: 'Bengaluru', name: 'Kempegowda Intl', lat: 13.198, lng: 77.706 },
29   HYD: { city: 'Hyderabad', name: 'Rajiv Gandhi Intl', lat: 17.231, lng: 78.429 },
30   MAA: { city: 'Chennai', name: 'Chennai Intl', lat: 12.994, lng: 80.171 },
31 };
32 let _airportMap = null;
33 function renderAirports() {
34   const codes = Object.keys(AIRPORT_INFO);
35   $('#airportList').innerHTML = codes.map(c => {
36     const a = AIRPORT_INFO[c];
37     return `<div class="card flight" id="airport-card-${c}">
38       <div><div class="route">${c} <span class="muted" style="font-size:14px;font-weight:600">· ${a.
   city}</span></div>
39       <div class="meta">${a.name} · ${a.lat.toFixed(3)}°N, ${a.lng.toFixed(3)}°E</div></div>
40       <div class="fare"><button class="btn btn-sm" onclick="flyFrom('${c}')">Fly from here</button>
   </div>
41     </div>`;
42   }).join('');
43   const el = $('#indiaMap');
44   if (!el) return;
```

#	src/main/resources/static/app.js · JavaScript · 364 lines

```
  if (typeof L === 'undefined') { el.innerHTML = '<div class="card">Real map needs an internet connection.</div>'; return; }
  if (_airportMap) { _airportMap.remove(); _airportMap = null; }
  const map = L.map('indiaMap').setView([22.5, 79.5], 5);
  _airportMap = map;
  L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', { maxZoom: 18, attribution: '&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>' }).addTo(map);
  const latlngs = codes.map(c => {
    const a = AIRPORT_INFO[c];
    const ll = [a.lat, a.lng];
    L.marker(ll).addTo(map).bindPopup(`<b>${c} — ${a.city}</b><br>${a.name}`);
    return ll;
  });
  for (let i = 0; i < latlngs.length; i++) for (let j = i + 1; j < latlngs.length; j++) {
    L.polyline([latlngs[i], latlngs[j]], { color: '#0ea5e9', weight: 1.5, dashArray: '5 4', opacity: 0.7 }).addTo(map);
  }
  map.fitBounds(L.latLngBounds(latlngs).pad(0.3));
  setTimeout(() => map.invalidateSize(), 100);
}
window.flyFrom = (c) => {
  showView('search');
  if ($('#fromSel')) $('#fromSel').value = c;
};
document.querySelectorAll('[data-close]').forEach(b => b.onclick = () => $('#' + b.dataset.close).style.display = 'none');

// ---------- auth ----------
let mode = 'login';
$('#authBtn').onclick = () => $('#authModal').style.display = 'flex';
$('#tabLogin').onclick = () => setMode('login');
$('#tabReg').onclick = () => setMode('register');
function setMode(m) { mode = m; $('#tabLogin').classList.toggle('active', m === 'login'); $('#tabReg').classList.toggle('active', m === 'register'); $('#aName').style.display = m === 'register' ? 'block' : 'none'; $('#authTitle').textContent = m === 'login' ? 'Login' : 'Create account'; }
$('#authForm').onsubmit = async (e) => {
  e.preventDefault();
  try {
    const body = { email: $('#aEmail').value.trim(), password: $('#aPass').value };
    let j;
    if (mode === 'register') { body.name = $('#aName').value.trim(); j = await api('/api/auth/register', { method: 'POST', headers: headers(), body: JSON.stringify(body) }); }
    else j = await api('/api/auth/login', { method: 'POST', headers: headers(), body: JSON.stringify(body) });
    state.token = j.token; state.user = j.user; localStorage.setItem('skyjet_token', state.token);
    $('#authModal').style.display = 'none'; refreshAuth(); toast('Welcome, ' + state.user.name + '!');
    if (state.user.role === 'admin') showView('admin');
  } catch (err) { toast(err.message); }
};
function refreshAuth() {
  const logged = !!state.user;
  $('#authBtn').textContent = logged ? 'Logout' : 'Login';
  $('#userChip').style.display = logged ? 'inline' : 'none';
  if (logged) $('#userChip').textContent = state.user.name + (state.user.role === 'admin' ? ' (admin)' : '');
  $('#adminNav').style.display = state.user && state.user.role === 'admin' ? 'block' : 'none';
}
$('#authBtn').onclick = () => {
  if (state.user) { state.user = null; state.token = ''; localStorage.removeItem('skyjet_token'); refreshAuth(); toast('Logged out.'); }
  else $('#authModal').style.display = 'flex';
};
// ---------- server switch (used by mobile app on LAN) ----------
function refreshServerBtn() { const b = $('#serverBtn'); if (b) b.title = 'Server: ' + (API || location.origin + ' (this device)'); }
$('#serverBtn').onclick = () => {
  const cur = API || '';
  const v = prompt('Airoplane Ticket Service server URL (empty = this device).\nOn your phone use your laptop IP, e.g. http://192.168.1.5:8080', cur);
```

#	src/main/resources/static/app.js · JavaScript · 364 lines

```
  if (v === null) return;
  setApiBase(v);
  state.flights = [];
  init(true);
  toast(API ? 'Server set to ' + API : 'Using built-in server.');
};
async function restoreSession() {
  if (!state.token) return;
  try { const j = await api('/api/auth/me', { headers: headers(false) }); state.user = j.user; refreshAuth(); } catch { state.token = ''; localStorage.removeItem('skyjet_token'); }
}

// ---------- search ----------
function fillAirports() {
  const a = state.airports;
  const opts = Object.entries(a).map(([c, n]) => `<option value="${c}">${c} — ${n.split('(')[0].trim()}</option>`).join('');
  $('#fromSel').innerHTML = opts; $('#toSel').innerHTML = opts;
  $('#fromSel').value = 'DEL'; $('#toSel').value = 'BOM';
  if (!$('#dateInp').value) {
    const t = new Date(Date.now() + 86400000).toISOString().slice(0, 10);
    $('#dateInp').value = t;
  }
  $('#dateInp').min = new Date().toISOString().slice(0, 10);
}
$('#swapBtn').onclick = () => { const v = $('#fromSel').value; $('#fromSel').value = $('#toSel').value; $('#toSel').value = v; };
$('#searchForm').onsubmit = async (e) => {
  e.preventDefault();
  const from = $('#fromSel').value, to = $('#toSel').value, date = $('#dateInp').value;
  if (from === to) return toast('From and To cannot be the same.');
  try {
    const j = await api(`/api/flights?from=${from}&to=${to}&date=${date}`, { headers: headers(false) });
    state.airports = j.airports || state.airports; fillKeep();
    state.flights = j.flights;
    renderResults();
  } catch (err) { toast(err.message); }
};
function fillKeep() { const f = $('#fromSel').value, t = $('#toSel').value; fillAirports(); $('#fromSel').value = f; $('#toSel').value = t; }
function renderResults() {
  const box = $('#results');
  if (!state.flights.length) { box.innerHTML = `<div class="card">No flights found. Try another date or route.</div>`; return; }
  box.innerHTML = state.flights.map(f => `
    <div class="card flight">
      <div><div class="route">${f.from} → ${f.to}</div>
      <div class="meta">${f.airline} · ${f.flightNumber} · ${f.date} · ${f.departure}–${f.arrival} · ${Math.floor(f.durationMin / 60)}h ${f.durationMin % 60}m</div>
      <span class="pill ${f.availableSeats > 20 ? 'green' : 'red'}">${f.availableSeats} seats left</span></div>
      <div class="fare"><b>${inr(f.price)}</b><div class="muted small">per adult</div>
      <button class="btn btn-primary" onclick="startBooking('${f.id}')">Book now</button></div>
    </div>`).join('');
}

// ---------- booking ----------
window.startBooking = async (id) => {
  const f = state.flights.find(x => x.id === id) || (await api('/api/flights/' + id, { headers: headers(false) })).flight;
  const need = parseInt($('#paxSel').value || '1', 10);
  state.bookingFlight = f; state.seats = [];
  $('#bookTitle').textContent = `${f.airline} ${f.flightNumber} · ${f.from}→${f.to}`;
  $('#seatNeed').textContent = need;
  renderSeatMap(f); renderPax(need); updatePrice();
  $('#bookModal').style.display = 'flex';
};
function renderSeatMap(f) {
  const taken = new Set(f.bookedSeats || []);
```

#	src/main/resources/static/app.js · JavaScript · 364 lines

```
  let html = '';
  for (let row = 1; row <= 30; row++) {
    for (const col of ['A', 'B', 'C', '', 'D', 'E', 'F']) {
      if (!col) { html += `<div class="seat aisle">${row}</div>`; continue; }
      const s = row + col;
      html += `<div class="seat ${taken.has(s) ? 'taken' : ''}" data-seat="${s}" onclick="toggleSeat('${s}',this)">${s}</div>`;
    }
  }
  $('#seatMap').innerHTML = html;
}
window.toggleSeat = (s, el) => {
  if (el.classList.contains('taken')) return;
  const need = parseInt($('#paxSel').value || '1', 10);
  if (el.classList.contains('sel')) { el.classList.remove('sel'); state.seats = state.seats.filter(x => x !== s); }
  else {
    if (state.seats.length >= need) return toast(`Select only ${need} seat(s).`);
    el.classList.add('sel'); state.seats.push(s);
  }
  updatePrice();
};
function renderPax(n) {
  $('#paxForms').innerHTML = Array.from({ length: n }, (_, i) => `
    <div class="stack" style="margin-bottom:8px"><b>Passenger ${i + 1}</b>
    <input data-pax="${i}" data-k="name" placeholder="Full name" required />
    <div class="row" style="margin:0"><input data-pax="${i}" data-k="age" type="number" min="1" max="120" placeholder="Age" style="flex:1" required />
    <select data-pax="${i}" data-k="gender" style="flex:1"><option>Male</option><option>Female</option><option>Other</option></select></div></div>`).join('');
}
function updatePrice() {
  const n = parseInt($('#paxSel').value || '1', 10);
  $('#priceLine').textContent = `Total: ${inr((state.bookingFlight?.price || 0) * n)} · ${n} ${n === 1 ? 'person' : 'people'} · Seats: ${state.seats.join(', ') || '—'}`;
}
$('#payBtn').onclick = async () => {
  try {
    if (!state.user) { $('#bookModal').style.display = 'none'; $('#authModal').style.display = 'flex'; return toast('Login first to book.'); }
    const n = parseInt($('#paxSel').value || '1', 10);
    if (state.seats.length !== n) return toast(`Please select ${n} seat(s).`);
    const pax = [];
    for (let i = 0; i < n; i++) {
      const name = document.querySelector(`[data-pax="${i}"][data-k="name"]`).value.trim();
      const age = document.querySelector(`[data-pax="${i}"][data-k="age"]`).value;
      const gender = document.querySelector(`[data-pax="${i}"][data-k="gender"]`).value;
      if (!name || !age) return toast('Fill all passenger details.');
      pax.push({ name, age: parseInt(age, 10), gender });
    }
    const card = $('#cardInp').value.replace(/\s/g, '');
    const amount = state.bookingFlight.price * n;
    const pay = await api('/api/payments/mock', { method: 'POST', headers: headers(), body: JSON.stringify({ amount, cardNumber: card }) });
    const j = await api('/api/bookings', { method: 'POST', headers: headers(), body: JSON.stringify({ flightId: state.bookingFlight.id, passengers: pax, seats: state.seats, paymentId: pay.payment.paymentId, paidAmount: amount }) });
    $('#bookModal').style.display = 'none';
    showTicket(j.booking);
    toast('Booking submitted — PNR ' + j.booking.pnr + ' is PENDING until an admin confirms it.');
  } catch (err) { toast(err.message); }
};

// ---------- ticket + my bookings ----------
function showTicket(b) {
  const f = b.flight || {};
  const confirmed = b.status === 'CONFIRMED';
  const note = confirmed ? '' : `<div class="pending-note">⏳ <b>PENDING</b> — this booking is waiting for admin confirmation. The e-ticket is issued only after approval.</div>`;
  $('#ticketBody').innerHTML = `<div class="ticket">
```

#	src/main/resources/static/app.js · JavaScript · 364 lines

```
    <div class="muted small">AIROPLANE E-TICKET · ${b.status}</div><div class="pnr">${b.pnr}</div>
    ${note}
    <h2>${f.from || ''} → ${f.to || ''}</h2>
    <p>${f.airline || ''} ${f.flightNumber || ''} · ${f.date || ''} · ${f.departure || ''}–${f.arrival || ''}</p>
    <p><b>Passengers:</b> ${(b.passengers || []).map(p => `${p.name} (${p.age})`).join(', ')}</p>
    <p><b>Seats:</b> ${(b.seats || []).join(', ')} · <b>Total paid:</b> ${inr(b.totalPrice)}</p>
    <p class="muted small">Payment ${b.paymentId || ''} · Booked ${String(b.createdAt || '').slice(0, 16).replace('T', ' ')}</p></div>`;
  const pb = $('#ticketPrintBtn');
  if (pb) { pb.disabled = !confirmed; pb.textContent = confirmed ? '🖨 Print' : '🖨 Print (after approval)'; }
  $('#ticketModal').style.display = 'flex';
}
async function loadMyBookings() {
  const box = $('#myBookings');
  if (!state.user) { box.innerHTML = `<div class="card">Please <b>Login</b> to view bookings.</div>`; return; }
  try {
    const j = await api('/api/bookings', { headers: headers(false) });
    if (!j.bookings.length) { box.innerHTML = `<div class="card">No bookings yet. Search and book your first flight!</div>`; return; }
    box.innerHTML = j.bookings.map(b => { const f = b.flight || {}; return `
      <div class="card flight"><div><div class="route">${f.from || '?'} → ${f.to || '?'} <span class="pill ${b.status === 'CONFIRMED' ? 'green' : b.status === 'PENDING' ? 'amber' : 'red'}">${b.status}</span></div>
      <div class="meta">PNR <b>${b.pnr}</b> · ${f.airline || ''} ${f.flightNumber || ''} · ${f.date || ''} · Seats ${(b.seats || []).join(', ')} · ${inr(b.totalPrice)}</div>
      ${b.status === 'PENDING' ? `<div class="pending-note small">⏳ Waiting for admin confirmation — the e-ticket is issued once approved.</div>` : ''}</div>
      <div class="fare"><button class="btn btn-sm" onclick='showTicket(${JSON.stringify(b).replace(/'/g, "&#39;")})'>${b.status === 'PENDING' ? 'View status' : 'View ticket'}</button>
      ${b.status !== 'CANCELLED' ? `<button class="btn btn-sm" onclick="cancelBooking('${b.pnr}')">Cancel</button>` : ''}</div></div>`; }).join('');
    window._cache = j.bookings;
    window.showTicket = (b) => { if (typeof b === 'string') b = window._cache.find(x => x.pnr === b) || b; if (typeof b !== 'object') return; showTicket(b); };
  } catch (err) { box.innerHTML = `<div class="card">${err.message}</div>`; }
}
window.cancelBooking = async (pnr) => {
  if (!confirm('Cancel booking ' + pnr + '? Refund will be mocked instantly.')) return;
  try { await api('/api/bookings/' + pnr + '/cancel', { method: 'POST', headers: headers() }); toast('Booking ' + pnr + ' cancelled.'); loadMyBookings(); }
  catch (err) { toast(err.message); }
};

// ---------- admin ----------
let adminTab = 'flights';
let adminFlightFilter = '';
let adminBookingFilter = '';
let adminAllBookings = [];
function setAdminTab(t) {
  adminTab = t;
  const tf = $('#tabFlights'), tb = $('#tabBookings');
  if (tf) tf.classList.toggle('active', t === 'flights');
  if (tb) tb.classList.toggle('active', t === 'bookings');
  if ($('#adminFlightsPane')) $('#adminFlightsPane').style.display = t === 'flights' ? 'block' : 'none';
  if ($('#adminBookingsPane')) $('#adminBookingsPane').style.display = t === 'bookings' ? 'block' : 'none';
  if (t === 'bookings') loadAdminBookings();
}
async function loadAdmin() {
  if (!state.user || state.user.role !== 'admin') {
    if ($('#adminGate')) $('#adminGate').innerHTML = '<div class="card">Admin login required. Please log in with an admin account.</div>';
    if ($('#adminBody')) $('#adminBody').style.display = 'none';
    return;
  }
  if ($('#adminGate')) $('#adminGate').innerHTML = '';
```

#	src/main/resources/static/app.js · JavaScript · 364 lines

```
  if ($('#adminBody')) $('#adminBody').style.display = 'block';
  try {
    const s = await api('/api/admin/stats', { headers: headers(false) });
    $('#statsRow').innerHTML = [['Flights', s.stats.flights], ['Users', s.stats.users], ['Bookings', s.stats.bookings], ['Active', s.stats.activeBookings], ['Pending', s.stats.pendingBookings || 0], ['Revenue', inr(s.stats.revenue)]].map(([k, v]) => `<div class="stat"><span class="muted">${k}</span><b>${v}</b></div>`).join('');
    const f = await api('/api/flights', { headers: headers(false) });
    state.flights = f.flights;
    renderAdminFlights();
    if (adminTab === 'bookings') loadAdminBookings();
  } catch (err) { toast(err.message); }
}
function renderAdminFlights() {
  const q = (adminFlightFilter || '').toLowerCase();
  const list = (state.flights || []).filter(x => !q || [x.flightNumber, x.airline, x.from, x.to, x.date].join(' ').toLowerCase().includes(q));
  $('#adminFlights').innerHTML = list.length ? `<div class="card"><table><tr><th>Flight</th><th>Route</th><th>Date</th><th>Time</th><th>Price</th><th>Seats</th><th></th></tr>${list.map(x => `<tr><td>${x.airline} <b>${x.flightNumber}</b></td><td>${x.from}→${x.to}</td><td>${x.date}</td><td>${x.departure}–${x.arrival}</td><td>${inr(x.price)}</td><td>${x.availableSeats}/${x.totalSeats}</td><td><button class="btn btn-sm" onclick="editFlight('${x.id}')">Edit</button> <button class="btn btn-sm" onclick="delFlight('${x.id}')">Del</button></td></tr>`).join('')}</table></div>`
    : '<div class="card">No flights match.</div>';
}
async function loadAdminBookings() {
  try {
    const j = await api('/api/bookings', { headers: headers() });
    adminAllBookings = j.bookings || [];
    renderAdminBookings();
  } catch (err) { if ($('#adminBookings')) $('#adminBookings').innerHTML = `<div class="card">${err.message}</div>`; }
}
function renderAdminBookings() {
  const q = (adminBookingFilter || '').toLowerCase();
  const list = adminAllBookings.filter(b => !q || [b.pnr, b.flightId, b.status, b.userId].join(' ').toLowerCase().includes(q));
  if (!$('#adminBookings')) return;
  $('#adminBookings').innerHTML = list.length ? `<div class="card"><table><tr><th>PNR</th><th>Flight</th><th>Seats</th><th>Total</th><th>Status</th><th></th></tr>${list.map(b => `<tr><td><b>${b.pnr}</b></td><td>${b.flightId}</td><td>${(b.seats || []).join(', ')}</td><td>${inr(b.totalPrice)}</td><td><span class="pill ${b.status === 'CONFIRMED' ? 'green' : b.status === 'PENDING' ? 'amber' : 'red'}">${b.status}</span></td><td>${b.status === 'PENDING' ? `<button class="btn btn-sm btn-primary" onclick="adminConfirmBooking('${b.pnr}')">✓ Confirm</button> ` : ''}${b.status !== 'CANCELLED' ? `<button class="btn btn-sm" onclick="adminCancelBooking('${b.pnr}')">Cancel</button>` : ''}</td></tr>`).join('')}</table></div>`
    : '<div class="card">No bookings found.</div>';
}
window.adminConfirmBooking = async (pnr) => {
  if (!confirm('Confirm booking ' + pnr + ' and issue the e-ticket?')) return;
  try { await api('/api/bookings/' + pnr + '/confirm', { method: 'POST', headers: headers() }); toast('Booking ' + pnr + ' confirmed — e-ticket issued.'); loadAdminBookings(); loadAdmin(); }
  catch (err) { toast(err.message); }
};
window.adminCancelBooking = async (pnr) => {
  if (!confirm('Cancel booking ' + pnr + ' (admin)?')) return;
  try { await api('/api/bookings/' + pnr + '/cancel', { method: 'POST', headers: headers() }); toast('Booking ' + pnr + ' cancelled.'); loadAdminBookings(); loadAdmin(); } catch (err) { toast(err.message); }
};
window.editFlight = (id) => {
  const f = state.flights.find(x => x.id === id); if (!f) return;
  $('#flightFormTitle').textContent = 'Edit flight ' + f.flightNumber;
  $('#fId').value = f.id; $('#fAirline').value = f.airline; $('#fNum').value = f.flightNumber;
  $('#fFrom').value = f.from; $('#fTo').value = f.to; $('#fDate').value = f.date;
  $('#fDep').value = f.departure; $('#fArr').value = f.arrival; $('#fPrice').value = f.price;
  window.scrollTo({ top: 0, behavior: 'smooth' });
};
window.delFlight = async (id) => {
  if (!confirm('Delete this flight?')) return;
```

src/main/resources/static/app.js · JavaScript · 364 lines

```
  try { await api('/api/flights/' + id, { method: 'DELETE', headers: headers() }); toast('Flight deleted.'); loadAdmin(); } catch (err) { toast(err.message); }
};
$('#fReset').onclick = () => { $('#flightForm').reset(); $('#fId').value = ''; $('#flightFormTitle').textContent = 'Add flight'; };
if ($('#tabFlights')) $('#tabFlights').onclick = () => setAdminTab('flights');
if ($('#tabBookings')) $('#tabBookings').onclick = () => setAdminTab('bookings');
if ($('#adminSearchBtn')) $('#adminSearchBtn').onclick = () => { adminFlightFilter = $('#adminSearch').value.trim(); renderAdminFlights(); };
if ($('#adminResetBtn')) $('#adminResetBtn').onclick = () => { adminFlightFilter = ''; $('#adminSearch').value = ''; renderAdminFlights(); };
if ($('#adminSearch')) $('#adminSearch').onkeydown = (e) => { if (e.key === 'Enter') { e.preventDefault(); adminFlightFilter = e.target.value.trim(); renderAdminFlights(); } };
if ($('#bookingSearchBtn')) $('#bookingSearchBtn').onclick = () => { adminBookingFilter = $('#bookingSearch').value.trim(); renderAdminBookings(); };
if ($('#bookingResetBtn')) $('#bookingResetBtn').onclick = () => { adminBookingFilter = ''; $('#bookingSearch').value = ''; renderAdminBookings(); };
if ($('#bookingRefreshBtn')) $('#bookingRefreshBtn').onclick = () => loadAdminBookings();
$('#flightForm').onsubmit = async (e) => {
  e.preventDefault();
  const body = { airline: $('#fAirline').value, flightNumber: $('#fNum').value, from: $('#fFrom').value.toUpperCase(), to: $('#fTo').value.toUpperCase(), date: $('#fDate').value, departure: $('#fDep').value, arrival: $('#fArr').value, durationMin: 120, price: parseFloat($('#fPrice').value), totalSeats: 180 };
  try {
    const id = $('#fId').value;
    if (id) await api('/api/flights/' + id, { method: 'PUT', headers: headers(), body: JSON.stringify(body) });
    else await api('/api/flights', { method: 'POST', headers: headers(), body: JSON.stringify(body) });
    toast('Flight saved.'); $('#flightForm').reset(); $('#fId').value = ''; loadAdmin();
  } catch (err) { toast(err.message); }
};

// ---------- init ----------
async function init() {
  refreshServerBtn();
  if (isNativeApp && !API) {
    const v = prompt('Enter your Airoplane Ticket Service server URL\n(same Wi-Fi as your laptop, e.g. http://192.168.1.5:8080)', '');
    if (v) setApiBase(v);
  }
  try {
    const j = await api('/api/flights', { headers: headers(false) });
    state.airports = j.airports; fillAirports();
    const t = new Date(Date.now() + 86400000).toISOString().slice(0, 10);
    const j2 = await api(`/api/flights?from=DEL&to=BOM&date=${t}`, { headers: headers(false) });
    state.flights = j2.flights; renderResults();
  } catch (err) { $('#results').innerHTML = `<div class="card">Backend not reachable: ${err.message}. ${isNativeApp ? 'Tap ⚙ to set the server URL.' : 'Start it with <code>./run.sh</code>.'}</div>`; }
  await restoreSession();
}
init();
```

src/main/resources/static/manifest.webmanifest · JSON · 17 lines

```
{
  "name": "Airoplane Ticket Service",
  "short_name": "Airoplane",
  "description": "Search, book and manage domestic flight tickets.",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "orientation": "portrait",
  "background_color": "#0f172a",
  "theme_color": "#0ea5e9",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
```

src/main/resources/static/manifest.webmanifest · JSON · 17 lines

```
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" },
    { "src": "/maskable-512.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable" }
  ]
}

```

src/main/resources/static/sw.js · JavaScript · 30 lines

```
/* SkyJet service worker — offline shell, network-first API. */
const CACHE = 'skyjet-v1';
const SHELL = ['/', '/index.html', '/styles.css', '/app.js', '/manifest.webmanifest', '/icon-192.png', '/icon-512.png'];

self.addEventListener('install', (e) => {
  e.waitUntil(caches.open(CACHE).then((c) => c.addAll(SHELL)).then(() => self.skipWaiting()));
});

self.addEventListener('activate', (e) => {
  e.waitUntil(caches.keys().then((ks) => Promise.all(ks.filter((k) => k !== CACHE).map((k) => caches.delete(k)))).then(() => self.clients.claim()));
});

self.addEventListener('fetch', (e) => {
  const url = new URL(e.request.url);
  if (e.request.method !== 'GET' || url.origin !== self.location.origin) return;
  if (url.pathname.startsWith('/api/')) return; // always live for bookings
  e.respondWith(
    caches.match(e.request).then((hit) => {
      const net = fetch(e.request).then((res) => {
        if (res && res.status === 200) {
          const copy = res.clone();
          caches.open(CACHE).then((c) => c.put(e.request, copy));
        }
        return res;
      }).catch(() => hit);
      return hit || net;
    })
  );
});

```

Part 3 — Build, Packaging and Run Scripts

pom.xml · XML · 47 lines

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.airline</groupId>
  <artifactId>airplane-ticket-service</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>
  <name>Airoplane Ticket Service</name>
  <description>Flight search, booking, e-ticket, auth and admin dashboard. Zero-dependency Java (JDK HttpServer).</description>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <mainClass>com.airline.App</mainClass>
  </properties>

  <build>
    <plugins>
```

pom.xml · XML · 47 lines

```xml
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.11.0</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.3.0</version>
        <configuration>
          <archive>
            <manifest>
              <mainClass>com.airline.App</mainClass>
            </manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
    <resources>
      <resource>
        <directory>src/main/resources</directory>
      </resource>
    </resources>
  </build>
</project>

```

run.sh · Shell · 25 lines

```sh
#!/bin/bash
# Run without Maven (uses only the JDK). Works with Java 17+.
set -e
cd "$(dirname "$0")"
# Auto-pick a compatible JDK (17+): prefer JAVA_HOME, else Homebrew openjdk@17, else default java.
pick_java() {
  if [ -n "$JAVA_HOME" ] && [ -x "$JAVA_HOME/bin/javac" ]; then return 0; fi
  for c in "/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home" "/usr/local/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home"; do
    if [ -x "$c/bin/javac" ]; then export JAVA_HOME="$c"; export PATH="$JAVA_HOME/bin:$PATH"; return 0; fi
  done
}
pick_java
command -v javac >/dev/null || { echo "ERROR: No Java JDK found. Install JDK 17+: brew install openjdk@17"; exit 1; }
PORT="${PORT:-8080}"
echo "→ Compiling (javac)..."
rm -rf out
mkdir -p out
find src/main/java -name "*.java" > /tmp/skyjet_sources.txt
javac -encoding UTF-8 -d out @/tmp/skyjet_sources.txt
echo "→ Copying static UI..."
mkdir -p out/static
cp -r src/main/resources/static/* out/static/
echo "→ Starting on http://localhost:$PORT ..."
java -cp out com.airline.App --port "$PORT"

```

AiroplaneTicketService.command · Shell · 16 lines

```sh
#!/bin/bash
# Double-click to launch Airoplane Ticket Service (macOS) — native Java desktop window.
cd "$(dirname "$0")"
PORT="${PORT:-8080}"
if [ -x "/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home/bin/java" ]; then
  export JAVA_HOME="/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home"
  export PATH="$JAVA_HOME/bin:$PATH"
fi
# free the port if a previous run is still alive
lsof -ti tcp:$PORT | xargs kill -9 2>/dev/null
```

AiroplaneTicketService.command · Shell · 16 lines

```
java -jar AiroplaneTicketService.jar --port $PORT --ui > skyjet.log 2>&1 &
sleep 2
echo "Airoplane Ticket Service is running at http://localhost:$PORT"
echo "Close this window to keep it running, or run: lsof -ti tcp:$PORT | xargs kill"
wait

```

mobile/package.json · JSON · 20 lines

```
{
  "name": "skyjet-mobile",
  "version": "1.0.0",
  "private": true,
  "description": "Airoplane Ticket Service — native Android wrapper (Capacitor)",
  "scripts": {
    "sync-www": "rm -rf www && mkdir -p www && cp -r ../src/main/resources/static/* www/ && rm -f www/sw.js",
    "sync": "npm run sync-www && npx cap sync android",
    "build-apk": "npm run sync && bash build-apk.sh"
  },
  "dependencies": {
    "@capacitor/android": "^7.4.3",
    "@capacitor/app": "^7.1.0",
    "@capacitor/core": "^7.4.3"
  },
  "devDependencies": {
    "@capacitor/cli": "^7.4.3"
  }
}

```

mobile/capacitor.config.js · JavaScript · 12 lines

```
/** @type {import('@capacitor/cli').CapacitorConfig} */
const config = {
  appId: 'in.skyjet.app',
  appName: 'Airoplane Ticket Service',
  webDir: 'www',
  android: {
    allowMixedContent: false,
  },
};

module.exports = config;

```

mobile/build-apk.sh · Shell · 29 lines

```
#!/bin/bash
# Build SkyJet debug APK.
# One-time setup (macOS):
#   brew install --cask android-commandlinetools
#   sdkmanager "platform-tools" "platforms;android-34" "build-tools;34.0.0"
#   brew install openjdk@21            (Capacitor 7.4 needs Java 21)
#   curl -L -o /tmp/gradle.zip https://services.gradle.org/distributions/gradle-8.11.1-bin.zip
#   unzip -q /tmp/gradle.zip -d $HOME  (gives ~/gradle-8.11.1)
set -e
cd "$(dirname "$0")"
if [ -d /opt/homebrew/opt/openjdk@21 ]; then
  export JAVA_HOME=/opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk/Contents/Home
fi
if [ -z "$ANDROID_HOME" ]; then
  if [ -d "$HOME/Library/Android/sdk" ]; then export ANDROID_HOME="$HOME/Library/Android/sdk";
  elif [ -d /opt/homebrew/share/android-commandlinetools ]; then export ANDROID_HOME=/opt/homebrew/share/android-commandlinetools; fi
fi
echo "sdk.dir=$ANDROID_HOME" > android/local.properties
npm run sync
if [ -x "$HOME/gradle-8.11.1/bin/gradle" ]; then
  (cd android && "$HOME/gradle-8.11.1/bin/gradle" assembleDebug)
```

mobile/build-apk.sh · Shell · 29 lines

```
else
  (cd android && ./gradlew assembleDebug)
fi
APK="$PWD/android/app/build/outputs/apk/debug/app-debug.apk"
cp "$APK" ../SkyJet.apk 2>/dev/null || cp "$APK" ./SkyJet.apk
echo "APK built: $APK (also copied to SkyJet.apk)"
ls -la "$APK"

```

mobile/tools/IconGen.java · Java · 66 lines

```
import java.awt.*;
import java.awt.geom.RoundRectangle2D;
import java.awt.image.BufferedImage;
import java.io.File;
import javax.imageio.ImageIO;

/** Generates Airoplane Ticket Service app icons (paper-plane on navy gradient). Usage: java IconGen <outDir> */
public class IconGen {
    public static void main(String[] a) throws Exception {
        File out = new File(a.length > 0 ? a[0] : ".");
        out.mkdirs();
        draw(new File(out, "icon-192.png"), 192, false);
        draw(new File(out, "icon-512.png"), 512, false);
        draw(new File(out, "maskable-512.png"), 512, true);
        draw(new File(out, "apple-touch-icon.png"), 180, false);
        // macOS .iconset source sizes
        int[] set = {16, 32, 64, 128, 256, 512, 1024};
        for (int s : set) {
            draw(new File(out, "icon_" + s + ".png"), s, false);
            if (s < 1024) {
                BufferedImage img = read(new File(out, "icon_" + s + ".png"));
                BufferedImage two = new BufferedImage(s * 2, s * 2, BufferedImage.TYPE_INT_ARGB);
                Graphics2D g = two.createGraphics();
                g.setRenderingHint(RenderingHints.KEY_INTERPOLATION, RenderingHints.VALUE_INTERPOLATION_BICUBIC);
                g.drawImage(img, 0, 0, s * 2, s * 2, null);
                g.dispose();
                ImageIO.write(two, "png", new File(out, "icon_" + (s * 2) + "@2x.png"));
            }
        }
        System.out.println("icons written to " + out.getAbsolutePath());
    }

    static BufferedImage read(File f) throws Exception { return ImageIO.read(f); }

    static void draw(File f, int s, boolean maskable) throws Exception {
        BufferedImage img = new BufferedImage(s, s, BufferedImage.TYPE_INT_ARGB);
        Graphics2D g = img.createGraphics();
        g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        int inset = maskable ? s / 8 : 0; // safe zone for maskable
        // navy gradient background
        GradientPaint gp = new GradientPaint(0, 0, new Color(0x0f172a), s, s, new Color(0x0369a1));
        g.setPaint(gp);
        if (maskable) g.fillRect(0, 0, s, s);
        else { g.setClip(new RoundRectangle2D.Float(0, 0, s, s, s * 0.24f, s * 0.24f)); g.fillRect(0, 0, s, s); g.setClip(null); }
        // sky circle
        g.setColor(new Color(0x38bdf8));
        int r = s * 3 / 8;
        g.fillOval((s - r) / 2, (s - r) / 2 - s / 32, r, r);
        // paper plane (white polygon)
        double cx = s / 2.0, cy = s / 2.0 - s / 64.0, w = s * 0.21, h = s * 0.17;
        Polygon plane = new Polygon();
        plane.addPoint((int) (cx - w), (int) (cy - h * 0.5));
        plane.addPoint((int) (cx + w * 1.1), (int) cy);
        plane.addPoint((int) (cx - w), (int) (cy + h * 0.5));
        plane.addPoint((int) (cx - w * 0.45), (int) cy);
        g.setColor(Color.WHITE);
```

mobile/tools/IconGen.java · Java · 66 lines

```
        g.fillPolygon(plane);
        // wordmark bar
        g.setColor(Color.WHITE);
        int bw = (int) (s * 0.44), bh = Math.max(2, s / 42);
        g.fillRoundRect((s - bw) / 2, (int) (s * 0.74), bw, bh, bh, bh);
        g.dispose();
        ImageIO.write(img, "png", f);
    }
}

```

Part 4 — Android Client (Capacitor + Web Assets)

mobile/www/index.html · HTML · 200 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0, viewport-fit=cover" />
<meta name="theme-color" content="#0ea5e9" />
<meta name="mobile-web-app-capable" content="yes" />
<meta name="apple-mobile-web-app-capable" content="yes" />
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent" />
<meta name="apple-mobile-web-app-title" content="Airoplane Ticket Service" />
<link rel="manifest" href="/manifest.webmanifest" />
<link rel="icon" type="image/png" sizes="192x192" href="/icon-192.png" />
<link rel="apple-touch-icon" href="/apple-touch-icon.png" />
<title>Airoplane Ticket Service</title>
<link rel="stylesheet" href="/styles.css" />
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
</head>
<body>
<header class="nav">
  <div class="nav-inner">
    <div class="brand"><span class="plane">✈</span> Airoplane Ticket Service</div>
    <nav class="links">
      <button data-nav="search" class="active">Search</button>
      <button data-nav="airports">Airports</button>
      <button data-nav="bookings">My Bookings</button>
      <button data-nav="admin" id="adminNav" style="display:none">Admin</button>
    </nav>
    <div class="auth-area">
      <span id="userChip" class="user-chip" style="display:none"></span>
      <button id="serverBtn" class="btn btn-sm" title="Server">⚙</button>
      <button id="authBtn" class="btn btn-sm">Login</button>
    </div>
  </div>
</header>

<main class="wrap">
  <!-- SEARCH -->
  <section id="view-search" class="view">
    <div class="hero">
      <div class="sky-anim" aria-hidden="true">
        <span class="cloud c1"></span>
        <span class="cloud c2"></span>
        <span class="cloud c3"></span>
        <span class="jet">✈</span>
      </div>
      <h1>Book domestic flights in minutes</h1>
      <p>Search → pick seats → pay (mock) → get instant e-ticket with PNR.</p>
    </div>
    <form id="searchForm" class="card search-card">
      <div class="grid-4">
        <label>From
          <select id="fromSel"></select>
```

mobile/www/index.html · HTML · 200 lines

```
        </label>
        <button type="button" id="swapBtn" class="swap" title="Swap">⇄</button>
        <label>To
          <select id="toSel"></select>
        </label>
        <label>Date
          <input type="date" id="dateInp" required />
        </label>
        <label>Travellers
          <select id="paxSel"><option>1</option><option>2</option><option>3</option><option>4</option><option>5</option><option>6</option></select>
        </label>
      </div>
      <button class="btn btn-primary" type="submit">🔍 Search flights</button>
    </form>
    <div id="results" class="results"></div>
    </section>

  <!-- AIRPORTS + MAP -->
  <section id="view-airports" class="view" style="display:none">
    <div class="hero hero-sm">
      <h1>Airport locations</h1>
      <p>All airports served by Airoplane Ticket Service, plotted on the map.</p>
    </div>
    <div class="map-grid">
      <div class="card map-card">
        <div id="indiaMap"></div>
      </div>
      <div id="airportList" class="results" style="margin-top:0"></div>
    </div>
  </section>

  <!-- BOOKINGS -->
  <section id="view-bookings" class="view" style="display:none">
    <h2>My bookings</h2>
    <p class="muted">Login to see your e-tickets. Cancel anytime for instant refund (mock).</p>
    <div id="myBookings" class="results"></div>
  </section>

  <!-- ADMIN -->
  <section id="view-admin" class="view" style="display:none">
    <h2>Admin dashboard</h2>
    <div id="adminGate"></div>
    <div id="adminBody">
    <div id="statsRow" class="stats"></div>
    <div class="tabs admin-tabs">
      <button id="tabFlights" class="active">✈ Flights</button>
      <button id="tabBookings">🎫 Bookings</button>
    </div>
    <div id="adminFlightsPane">
    <div class="card admin-toolbar">
      <input id="adminSearch" placeholder="Search flight no., airline, route..." />
      <button class="btn btn-sm" id="adminSearchBtn">Search</button>
      <button class="btn btn-sm" id="adminResetBtn" type="button">Reset</button>
    </div>
    <div class="card">
      <h3 id="flightFormTitle">Add flight</h3>
      <form id="flightForm" class="grid-4">
        <input type="hidden" id="fId" />
        <label>Airline<input id="fAirline" required placeholder="Airoplane Airways" /></label>
        <label>Flight no.<input id="fNum" required placeholder="SJ-101" /></label>
        <label>From<input id="fFrom" required placeholder="DEL" maxlength="3" /></label>
        <label>To<input id="fTo" required placeholder="BOM" maxlength="3" /></label>
        <label>Date<input id="fDate" type="date" required /></label>
        <label>Departure<input id="fDep" type="time" required /></label>
        <label>Arrival<input id="fArr" type="time" required /></label>
        <label>Price (₹)<input id="fPrice" type="number" min="500" required /></label>
        <div class="form-actions">
          <button class="btn btn-primary" type="submit">Save flight</button>
          <button class="btn" type="button" id="fReset">Reset</button>
```

mobile/www/index.html · HTML · 200 lines

```
        </div>
      </form>
    </div>
    <div id="adminFlights" class="results"></div>
    </div>
    <div id="adminBookingsPane" style="display:none">
      <div class="card admin-toolbar">
        <input id="bookingSearch" placeholder="Search PNR, flight, status..." />
        <button class="btn btn-sm" id="bookingSearchBtn">Search</button>
        <button class="btn btn-sm" id="bookingResetBtn" type="button">Reset</button>
        <button class="btn btn-sm" id="bookingRefreshBtn" type="button">↻ Refresh</button>
      </div>
      <div id="adminBookings" class="results"></div>
    </div>
    </div>
  </section>
</main>

<!-- BOOKING MODAL -->
<div id="bookModal" class="modal" style="display:none">
  <div class="modal-box wide">
    <div class="modal-head"><h3 id="bookTitle">Complete booking</h3><button class="x" data-close="bookModal">x</button></div>
    <div class="book-grid">
      <div>
        <h4>1. Select seats (<span id="seatNeed"></span> needed)</h4>
        <div id="seatMap" class="seatmap"></div>
        <div class="legend"><span class="seat"></span> free <span class="seat taken"></span> taken <span class="seat sel"></span> selected</div>
      </div>
      <div>
        <h4>2. Passengers</h4>
        <div id="paxForms"></div>
        <h4>3. Payment (mock)</h4>
        <input id="cardInp" inputmode="numeric" placeholder="Test card 4111 1111 1111 1111" value="4111 1111 1111 1111" />
        <div id="priceLine" class="price"></div>
        <button id="payBtn" class="btn btn-primary btn-block">Pay & confirm booking</button>
      </div>
    </div>
  </div>
</div>

<!-- TICKET MODAL -->
<div id="ticketModal" class="modal" style="display:none">
  <div class="modal-box">
    <div class="modal-head"><h3>🎫 E-Ticket</h3><button class="x" data-close="ticketModal">x</button></div>
    <div id="ticketBody"></div>
    <div class="row">
      <button class="btn" onclick="window.print()">🖨 Print</button>
      <button class="btn" data-close="ticketModal">Close</button>
    </div>
  </div>
</div>

<!-- AUTH MODAL -->
<div id="authModal" class="modal" style="display:none">
  <div class="modal-box">
    <div class="modal-head"><h3 id="authTitle">Login</h3><button class="x" data-close="authModal">x</button></div>
    <div class="tabs"><button id="tabLogin" class="active">Login</button><button id="tabReg">Register</button></div>
    <form id="authForm" class="stack">
      <input id="aName" placeholder="Full name (register only)" style="display:none" />
      <input id="aEmail" type="email" required placeholder="Enter your email" value="" />
      <input id="aPass" type="password" required placeholder="Enter your password" value="" />
      <button class="btn btn-primary btn-block" type="submit">Continue</button>
      <p class="muted small">New here? Switch to Register to create an account.</p>
    </form>
```

mobile/www/index.html · HTML · 200 lines

```
  </div>
</div>

<div id="toast" class="toast"></div>
<footer class="foot">Airoplane Ticket Service · Java (JDK HttpServer) + HTML/CSS/JS · No external dependencies · Data stored in <code>data/*.json</code></footer>
<script>
if ('serviceWorker' in navigator && location.protocol.indexOf('http') === 0 && !window.Capacitor) {
  window.addEventListener('load', () => navigator.serviceWorker.register('/sw.js').catch(() => {}));
}
</script>
<script src="/app.js"></script>
</body>
</html>

```

mobile/www/styles.css · CSS · 83 lines

```
:root{--bg:#eef2f7;--card:#fff;--ink:#0f172a;--mut:#64748b;--pri:#0ea5e9;--pri2:#0369a1;--ok:#16a34a;--bad:#dc2626;--line:#e2e8f0}
*{box-sizing:border-box}body{margin:0;font-family:Inter,system-ui,Segoe UI,Roboto,Arial;color:var(--ink);
background:var(--bg) url("/plane-bg.svg") no-repeat top center;background-size:1600px auto;background-attachment:fixed}
.nav{background:#0f172a;color:#fff;position:sticky;top:0;z-index:10}
.nav-inner{max-width:1080px;margin:auto;display:flex;align-items:center;gap:16px;padding:12px 16px}
.brand{font-weight:800;font-size:20px}.plane{color:#38bdf8}.dot{color:#38bdf8}
.links{display:flex;gap:8px;margin-left:8px}.links button{background:transparent;border:0;color:#cbd5e1;padding:8px 12px;border-radius:8px;cursor:pointer;font-weight:600}
.links button.active,.links button:hover{background:#1e293b;color:#fff}
.auth-area{margin-left:auto;display:flex;gap:8px;align-items:center}
.user-chip{background:#1e293b;padding:6px 10px;border-radius:20px;font-size:13px}
.wrap{max-width:1080px;margin:auto;padding:20px 16px 60px}
.hero{position:relative;overflow:hidden;border-radius:16px;padding:46px 22px 30px;margin:6px 0 14px;color:#fff;
background:#0c4a6e url("/plane-bg.svg") no-repeat center center;background-size:cover;
border:1px solid #0c4a6e;box-shadow:0 6px 24px rgba(2,6,23,.25)}
.hero::after{content:"";position:absolute;inset:0;background:linear-gradient(90deg,rgba(2,6,23,.62),rgba(2,6,23,.15) 60%,rgba(2,6,23,0));pointer-events:none}
.hero h1,.hero p{position:relative;z-index:1;text-shadow:0 1px 8px rgba(0,0,0,.4)}
.hero h1{margin:8px 0 4px;font-size:30px}.hero p{color:#e0f2fe;margin:0 0 4px}
.sky-anim{position:absolute;inset:0;overflow:hidden;pointer-events:none}
.sky-anim .jet{position:absolute;top:22%;left:-12%;font-size:44px;color:#fff;filter:drop-shadow(0 4px 8px rgba(0,0,0,.45));animation:flyAcross 9s linear infinite}
.sky-anim .cloud{position:absolute;background:#fff;opacity:.7;border-radius:999px;animation:drift linear infinite}
.sky-anim .cloud::before,.sky-anim .cloud::after{content:"";position:absolute;background:#fff;border-radius:50%}
.sky-anim .cloud::before{width:55%;height:160%;left:12%;top:-80%}
.sky-anim .cloud::after{width:38%;height:110%;right:14%;top:-45%}
.sky-anim .c1{width:120px;height:34px;top:18%;animation-duration:26s}
.sky-anim .c2{width:80px;height:24px;top:58%;animation-duration:38s;animation-delay:-12s;opacity:.5}
.sky-anim .c3{width:150px;height:40px;top:74%;animation-duration:48s;animation-delay:-25s;opacity:.35}
.brand .plane{display:inline-block;animation:bob 2.6s ease-in-out infinite}
@keyframes flyAcross{0%{transform:translate(0,26px) rotate(-4deg)}15%{transform:translate(18vw,0) rotate(0deg)}50%{transform:translate(55vw,-26px) rotate(2deg)}85%{transform:translate(95vw,-8px) rotate(0deg)}100%{transform:translate(125vw,14px) rotate(-4deg)}}
@keyframes drift{from{transform:translateX(110vw)}to{transform:translateX(-30vw)}}
@keyframes bob{0%,100%{transform:translateY(0) rotate(-6deg)}50%{transform:translateY(-3px) rotate(6deg)}}
@media(prefers-reduced-motion:reduce){.sky-anim .jet,.sky-anim .cloud,.brand .plane{animation:none}}
.card{background:var(--card);border:1px solid var(--line);border-radius:14px;padding:16px;box-shadow:0 2px 10px rgba(2,6,23,.05)}
.search-card{display:flex;flex-direction:column;gap:12px}
.grid-4{display:grid;grid-template-columns:1fr auto 1fr 1fr 1fr;gap:10px;align-items:end}
label{display:flex;flex-direction:column;gap:6px;font-size:13px;font-weight:600;color:#334155}
input,select{padding:10px 12px;border:1px solid var(--line);border-radius:10px;font-size:14px;background:#fff;color:var(--ink)}
```

mobile/www/styles.css · CSS · 83 lines

```
.swap{border:1px solid var(--line);background:#fff;border-radius:10px;font-size:18px;cursor:pointer;padding:8px 12px}
.btn{border:1px solid var(--line);background:#fff;padding:10px 14px;border-radius:10px;cursor:pointer;font-weight:700}
.btn-primary{background:var(--pri);border-color:var(--pri);color:#fff}.btn-primary:hover{background:var(--pri2)}
.btn-sm{padding:7px 12px}.btn-block{width:100%;margin-top:10px}
.results{display:flex;flex-direction:column;gap:12px;margin-top:14px}
.flight{display:flex;gap:14px;align-items:center;flex-wrap:wrap}
.flight .route{font-size:20px;font-weight:800}.flight .meta{color:var(--mut);font-size:13px}
.flight .fare{margin-left:auto;text-align:right}.fare b{font-size:20px;color:var(--pri2)}
.pill{background:#f1f5f9;border:1px solid var(--line);border-radius:20px;padding:3px 10px;font-size:12px;font-weight:700}
.pill.green{background:#dcfce7;border-color:#bbf7d0;color:#166534}.pill.red{background:#fee2e2;border-color:#fecaca;color:#991b1b}
.muted{color:var(--mut)}.small{font-size:12px}
.stats{display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:10px;margin:12px 0}
.stat{background:#fff;border:1px solid var(--line);border-radius:12px;padding:12px}.stat b{font-size:22px;display:block}
.modal{position:fixed;inset:0;background:rgba(2,6,23,.55);display:flex;align-items:flex-start;justify-content:center;padding:24px;z-index:50;overflow:auto}
.modal-box{background:#fff;border-radius:14px;padding:18px;max-width:520px;width:100%}
.modal-box.wide{max-width:880px}
.modal-head{display:flex;justify-content:space-between;align-items:center}
.x{border:0;background:#f1f5f9;border-radius:8px;padding:6px 10px;cursor:pointer}
.book-grid{display:grid;grid-template-columns:1.2fr 1fr;gap:16px}
@media(max-width:800px){.book-grid{grid-template-columns:1fr}.grid-4{grid-template-columns:1fr 1fr}}
.seatmap{display:grid;grid-template-columns:repeat(7,1fr);gap:5px;max-height:300px;overflow:auto;background:#f8fafc;padding:10px;border-radius:10px;border:1px solid var(--line)}
.seat{border:1px solid #cbd5e1;background:#fff;border-radius:6px;font-size:11px;padding:6px 0;text-align:center;cursor:pointer}
.seat.taken{background:#e2e8f0;color:#94a3b8;cursor:not-allowed;text-decoration:line-through}
.seat.sel{background:var(--pri);color:#fff;border-color:var(--pri)}
.seat.aisle{border:0;background:transparent;cursor:default}
.legend{display:flex;gap:8px;align-items:center;font-size:12px;color:var(--mut);margin-top:8px}.legend .seat{width:26px;cursor:default}
.stack{display:flex;flex-direction:column;gap:10px}
.tabs{display:flex;gap:8px;margin-bottom:12px}.tabs button{flex:1;padding:9px;border-radius:10px;border:1px solid var(--line);background:#f8fafc;cursor:pointer;font-weight:700}.tabs button.active{background:#0f172a;color:#fff}
.price{font-size:18px;font-weight:800;margin-top:10px}
.row{display:flex;gap:8px;margin-top:12px}
.ticket{border:2px dashed #0ea5e9;border-radius:12px;padding:14px;background:#f0f9ff}
.ticket h2{margin:0}.pnr{font-size:26px;letter-spacing:3px;font-weight:900}
.toast{position:fixed;bottom:20px;left:50%;transform:translateX(-50%);background:#0f172a;color:#fff;padding:10px 16px;border-radius:10px;display:none;z-index:99}
.form-actions{display:flex;gap:8px;align-items:end}
.admin-tabs{margin:12px 0 4px}
.admin-toolbar{display:flex;gap:8px;margin:12px 0;align-items:center}
.admin-toolbar input{flex:1}
#adminGate:empty{display:none}
.hero-sm{padding:30px 22px 22px}
.map-grid{display:grid;grid-template-columns:1fr 1.2fr;gap:14px;margin-top:14px;align-items:start}
@media(max-width:800px){.map-grid{grid-template-columns:1fr}}
.map-card{padding:8px}
#indiaMap{width:100%;height:440px;border-radius:10px;z-index:0}
table{width:100%;border-collapse:collapse;font-size:13px}th,td{border-bottom:1px solid var(--line);padding:8px;text-align:left}
.foot{text-align:center;color:var(--mut);font-size:12px;padding:20px}
@media print{.nav,.search-card,.hero,.foot,.row,.links,.auth-area{display:none!important}.modal{position:static;background:none;padding:0}.modal-box{max-width:100%}}

```

mobile/www/app.js · JavaScript · 353 lines

```
/* Airoplane Ticket Service frontend — vanilla JS */
const $ = (s) => document.querySelector(s);
const state = { token: localStorage.getItem('skyjet_token') || '', user: null, flights: [], bookingFlight: null, seats: [], airports: {} };
```

mobile/www/app.js · **JavaScript** · **353 lines**

```
// API base: same-origin on desktop; configurable for the mobile app (phone talks to laptop over LAN).
const isNativeApp = !!window.Capacitor;
let API = localStorage.getItem('skyjet_api') || '';
function setApiBase(url) { API = (url || '').trim().replace(/\/$/, ''); if (API) localStorage.setItem('skyjet_api', API); else localStorage.removeItem('skyjet_api'); refreshServerBtn(); }

function toast(msg) { const t = $('#toast'); t.textContent = msg; t.style.display = 'block'; setTimeout(() => t.style.display = 'none', 3200); }
function headers(json = true) { const h = {}; if (json) h['Content-Type'] = 'application/json'; if (state.token) h['Authorization'] = 'Bearer ' + state.token; return h; }
async function api(path, opts = {}) { const r = await fetch(API + path, opts); const j = await r.json().catch(() => ({ ok: false, error: 'Bad response' })); if (!r.ok || !j.ok) throw new Error(j.error || ('HTTP ' + r.status)); return j; }
const inr = (n) => '₹' + Number(n || 0).toLocaleString('en-IN');

// ---------- nav ----------
document.querySelectorAll('[data-nav]').forEach(b => b.onclick = () => showView(b.dataset.nav));
function showView(v) {
  document.querySelectorAll('[data-nav]').forEach(b => b.classList.toggle('active', b.dataset.nav === v));
  ['search', 'airports', 'bookings', 'admin'].forEach(k => $('#view-' + k).style.display = k === v ? 'block' : 'none');
  if (v === 'airports') renderAirports();
  if (v === 'bookings') loadMyBookings();
  if (v === 'admin') loadAdmin();
}

// ---------- airports + map (offline SVG, zero dependencies) ----------
const AIRPORT_INFO = {
  DEL: { city: 'New Delhi', name: 'Indira Gandhi Intl', lat: 28.556, lng: 77.100 },
  BOM: { city: 'Mumbai', name: 'Chhatrapati Shivaji Intl', lat: 19.090, lng: 72.865 },
  BLR: { city: 'Bengaluru', name: 'Kempegowda Intl', lat: 13.198, lng: 77.706 },
  HYD: { city: 'Hyderabad', name: 'Rajiv Gandhi Intl', lat: 17.231, lng: 78.429 },
  MAA: { city: 'Chennai', name: 'Chennai Intl', lat: 12.994, lng: 80.171 },
};
let _airportMap = null;
function renderAirports() {
  const codes = Object.keys(AIRPORT_INFO);
  $('#airportList').innerHTML = codes.map(c => {
    const a = AIRPORT_INFO[c];
    return `<div class="card flight" id="airport-card-${c}">
      <div><div class="route">${c} <span class="muted" style="font-size:14px;font-weight:600">· ${a.city}</span></div>
      <div class="meta">${a.name} · ${a.lat.toFixed(3)}°N, ${a.lng.toFixed(3)}°E</div></div>
      <div class="fare"><button class="btn btn-sm" onclick="flyFrom('${c}')">Fly from here</button></div>
    </div>`;
  }).join('');
  const el = $('#indiaMap');
  if (!el) return;
  if (typeof L === 'undefined') { el.innerHTML = '<div class="card">Real map needs an internet connection.</div>'; return; }
  if (_airportMap) { _airportMap.remove(); _airportMap = null; }
  const map = L.map('indiaMap').setView([22.5, 79.5], 5);
  _airportMap = map;
  L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', { maxZoom: 18, attribution: '&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>' }).addTo(map);
  const latlngs = codes.map(c => {
    const a = AIRPORT_INFO[c];
    const ll = [a.lat, a.lng];
    L.marker(ll).addTo(map).bindPopup(`<b>${c} — ${a.city}</b><br>${a.name}`);
    return ll;
  });
  for (let i = 0; i < latlngs.length; i++) for (let j = i + 1; j < latlngs.length; j++) {
    L.polyline([latlngs[i], latlngs[j]], { color: '#0ea5e9', weight: 1.5, dashArray: '5 4', opacity: 0.7 }).addTo(map);
  }
  map.fitBounds(L.latLngBounds(latlngs).pad(0.3));
  setTimeout(() => map.invalidateSize(), 100);
```

mobile/www/app.js · JavaScript · 353 lines

```
}
window.flyFrom = (c) => {
  showView('search');
  if ($('#fromSel')) $('#fromSel').value = c;
};
document.querySelectorAll('[data-close]').forEach(b => b.onclick = () => $('#' + b.dataset.close).style.display = 'none');

// ---------- auth ----------
let mode = 'login';
$('#authBtn').onclick = () => $('#authModal').style.display = 'flex';
$('#tabLogin').onclick = () => setMode('login');
$('#tabReg').onclick = () => setMode('register');
function setMode(m) { mode = m; $('#tabLogin').classList.toggle('active', m === 'login'); $('#tabReg').classList.toggle('active', m === 'register'); $('#aName').style.display = m === 'register' ? 'block' : 'none'; $('#authTitle').textContent = m === 'login' ? 'Login' : 'Create account'; }
$('#authForm').onsubmit = async (e) => {
  e.preventDefault();
  try {
    const body = { email: $('#aEmail').value.trim(), password: $('#aPass').value };
    let j;
    if (mode === 'register') { body.name = $('#aName').value.trim(); j = await api('/api/auth/register', { method: 'POST', headers: headers(), body: JSON.stringify(body) }); }
    else j = await api('/api/auth/login', { method: 'POST', headers: headers(), body: JSON.stringify(body) });
    state.token = j.token; state.user = j.user; localStorage.setItem('skyjet_token', state.token);
    $('#authModal').style.display = 'none'; refreshAuth(); toast('Welcome, ' + state.user.name + '!');
    if (state.user.role === 'admin') showView('admin');
  } catch (err) { toast(err.message); }
};
function refreshAuth() {
  const logged = !!state.user;
  $('#authBtn').textContent = logged ? 'Logout' : 'Login';
  $('#userChip').style.display = logged ? 'inline' : 'none';
  if (logged) $('#userChip').textContent = state.user.name + (state.user.role === 'admin' ? ' (admin)' : '');
  $('#adminNav').style.display = state.user && state.user.role === 'admin' ? 'block' : 'none';
}
$('#authBtn').onclick = () => {
  if (state.user) { state.user = null; state.token = ''; localStorage.removeItem('skyjet_token'); refreshAuth(); toast('Logged out.'); }
  else $('#authModal').style.display = 'flex';
};
// ---------- server switch (used by mobile app on LAN) ----------
function refreshServerBtn() { const b = $('#serverBtn'); if (b) b.title = 'Server: ' + (API || location.origin + ' (this device)'); }
$('#serverBtn').onclick = () => {
  const cur = API || '';
  const v = prompt('Airoplane Ticket Service server URL (empty = this device).\nOn your phone use your laptop IP, e.g. http://192.168.1.5:8080', cur);
  if (v === null) return;
  setApiBase(v);
  state.flights = [];
  init(true);
  toast(API ? 'Server set to ' + API : 'Using built-in server.');
};
async function restoreSession() {
  if (!state.token) return;
  try { const j = await api('/api/auth/me', { headers: headers(false) }); state.user = j.user; refreshAuth(); } catch { state.token = ''; localStorage.removeItem('skyjet_token'); }
}

// ---------- search ----------
function fillAirports() {
  const a = state.airports;
  const opts = Object.entries(a).map(([c, n]) => `<option value="${c}">${c} — ${n.split('(')[0].trim()}</option>`).join('');
  $('#fromSel').innerHTML = opts; $('#toSel').innerHTML = opts;
  $('#fromSel').value = 'DEL'; $('#toSel').value = 'BOM';
```

mobile/www/app.js · JavaScript · 353 lines

```
  if (!$('#dateInp').value) {
    const t = new Date(Date.now() + 86400000).toISOString().slice(0, 10);
    $('#dateInp').value = t;
  }
  $('#dateInp').min = new Date().toISOString().slice(0, 10);
}
$('#swapBtn').onclick = () => { const v = $('#fromSel').value; $('#fromSel').value = $('#toSel').value; $('#toSel').value = v; };
$('#searchForm').onsubmit = async (e) => {
  e.preventDefault();
  const from = $('#fromSel').value, to = $('#toSel').value, date = $('#dateInp').value;
  if (from === to) return toast('From and To cannot be the same.');
  try {
    const j = await api(`/api/flights?from=${from}&to=${to}&date=${date}`, { headers: headers(false) });
    state.airports = j.airports || state.airports; fillKeep();
    state.flights = j.flights;
    renderResults();
  } catch (err) { toast(err.message); }
};
function fillKeep() { const f = $('#fromSel').value, t = $('#toSel').value; fillAirports(); $('#fromSel').value = f; $('#toSel').value = t; }
function renderResults() {
  const box = $('#results');
  if (!state.flights.length) { box.innerHTML = `<div class="card">No flights found. Try another date or route.</div>`; return; }
  box.innerHTML = state.flights.map(f => `
    <div class="card flight">
      <div><div class="route">${f.from} → ${f.to}</div>
      <div class="meta">${f.airline} · ${f.flightNumber} · ${f.date} · ${f.departure}–${f.arrival} · ${Math.floor(f.durationMin / 60)}h ${f.durationMin % 60}m</div>
      <span class="pill ${f.availableSeats > 20 ? 'green' : 'red'}">${f.availableSeats} seats left</span></div>
      <div class="fare"><b>${inr(f.price)}</b><div class="muted small">per adult</div>
      <button class="btn btn-primary" onclick="startBooking('${f.id}')">Book now</button></div>
    </div>`).join('');
}

// ---------- booking ----------
window.startBooking = async (id) => {
  const f = state.flights.find(x => x.id === id) || (await api('/api/flights/' + id, { headers: headers(false) })).flight;
  const need = parseInt($('#paxSel').value || '1', 10);
  state.bookingFlight = f; state.seats = [];
  $('#bookTitle').textContent = `${f.airline} ${f.flightNumber} · ${f.from}→${f.to}`;
  $('#seatNeed').textContent = need;
  renderSeatMap(f); renderPax(need); updatePrice();
  $('#bookModal').style.display = 'flex';
};
function renderSeatMap(f) {
  const taken = new Set(f.bookedSeats || []);
  let html = '';
  for (let row = 1; row <= 30; row++) {
    for (const col of ['A', 'B', 'C', '', 'D', 'E', 'F']) {
      if (!col) { html += `<div class="seat aisle">${row}</div>`; continue; }
      const s = row + col;
      html += `<div class="seat ${taken.has(s) ? 'taken' : ''}" data-seat="${s}" onclick="toggleSeat('${s}',this)">${s}</div>`;
    }
  }
  $('#seatMap').innerHTML = html;
}
window.toggleSeat = (s, el) => {
  if (el.classList.contains('taken')) return;
  const need = parseInt($('#paxSel').value || '1', 10);
  if (el.classList.contains('sel')) { el.classList.remove('sel'); state.seats = state.seats.filter(x => x !== s); }
  else {
    if (state.seats.length >= need) return toast(`Select only ${need} seat(s).`);
    el.classList.add('sel'); state.seats.push(s);
```

#	mobile/www/app.js · JavaScript · 353 lines

```
  }
  updatePrice();
};
function renderPax(n) {
  $('#paxForms').innerHTML = Array.from({ length: n }, (_, i) => `
    <div class="stack" style="margin-bottom:8px"><b>Passenger ${i + 1}</b>
    <input data-pax="${i}" data-k="name" placeholder="Full name" required />
    <div class="row" style="margin:0"><input data-pax="${i}" data-k="age" type="number" min="1" max="120" placeholder="Age" style="flex:1" required />
    <select data-pax="${i}" data-k="gender" style="flex:1"><option>Male</option><option>Female</option><option>Other</option></select></div></div>`).join('');
}
function updatePrice() {
  const n = parseInt($('#paxSel').value || '1', 10);
  $('#priceLine').textContent = `Total: ${inr((state.bookingFlight?.price || 0) * n)} · Seats: ${state.seats.join(', ') || '—'}`;
}
$('#payBtn').onclick = async () => {
  try {
    if (!state.user) { $('#bookModal').style.display = 'none'; $('#authModal').style.display = 'flex'; return toast('Login first to book.'); }
    const n = parseInt($('#paxSel').value || '1', 10);
    if (state.seats.length !== n) return toast(`Please select ${n} seat(s).`);
    const pax = [];
    for (let i = 0; i < n; i++) {
      const name = document.querySelector(`[data-pax="${i}"][data-k="name"]`).value.trim();
      const age = document.querySelector(`[data-pax="${i}"][data-k="age"]`).value;
      const gender = document.querySelector(`[data-pax="${i}"][data-k="gender"]`).value;
      if (!name || !age) return toast('Fill all passenger details.');
      pax.push({ name, age: parseInt(age, 10), gender });
    }
    const card = $('#cardInp').value.replace(/\s/g, '');
    const amount = state.bookingFlight.price * n;
    const pay = await api('/api/payments/mock', { method: 'POST', headers: headers(), body: JSON.stringify({ amount, cardNumber: card }) });
    const j = await api('/api/bookings', { method: 'POST', headers: headers(), body: JSON.stringify({ flightId: state.bookingFlight.id, passengers: pax, seats: state.seats, paymentId: pay.payment.paymentId, paidAmount: amount }) });
    $('#bookModal').style.display = 'none';
    showTicket(j.booking);
    toast('Booking confirmed! PNR ' + j.booking.pnr);
  } catch (err) { toast(err.message); }
};

// ---------- ticket + my bookings ----------
function showTicket(b) {
  const f = b.flight || {};
  $('#ticketBody').innerHTML = `<div class="ticket">
    <div class="muted small">SKYJET E-TICKET · ${b.status}</div><div class="pnr">${b.pnr}</div>
    <h2>${f.from || ''} → ${f.to || ''}</h2>
    <p>${f.airline || ''} ${f.flightNumber || ''} · ${f.date || ''} · ${f.departure || ''}–${f.arrival || ''}</p>
    <p><b>Passengers:</b> ${(b.passengers || []).map(p => `${p.name} (${p.age})`).join(', ')}</p>
    <p><b>Seats:</b> ${(b.seats || []).join(', ')} · <b>Total paid:</b> ${inr(b.totalPrice)}</p>
    <p class="muted small">Payment ${b.paymentId || ''} · Booked ${String(b.createdAt || '').slice(0, 16).replace('T', ' ')}</p></div>`;
  $('#ticketModal').style.display = 'flex';
}
async function loadMyBookings() {
  const box = $('#myBookings');
  if (!state.user) { box.innerHTML = `<div class="card">Please <b>Login</b> to view bookings.</div>`; return; }
  try {
    const j = await api('/api/bookings', { headers: headers(false) });
    if (!j.bookings.length) { box.innerHTML = `<div class="card">No bookings yet. Search and book your first flight!</div>`; return; }
    box.innerHTML = j.bookings.map(b => { const f = b.flight || {}; return `
      <div class="card flight"><div><div class="route">${f.from || '?'} → ${f.to || '?'} <span class="pill ${b.status === 'CONFIRMED' ? 'green' : 'red'}">${b.status}</span></div>
```

mobile/www/app.js · JavaScript · 353 lines

```
      <div class="meta">PNR <b>${b.pnr}</b> · ${f.airline || ''} ${f.flightNumber || ''} · ${f.date || ''} · Seats ${(b.seats || []).join(', ')} · ${inr(b.totalPrice)}</div></div>
      <div class="fare"><button class="btn btn-sm" onclick='showTicket(${JSON.stringify(b).replace(/'/g, "&#39;")})'>View ticket</button>
      ${b.status === 'CONFIRMED' ? `<button class="btn btn-sm" onclick="cancelBooking('${b.pnr}')">Cancel</button>` : ''}</div></div>`; }).join('');
    window._cache = j.bookings;
    window.showTicket = (b) => { if (typeof b === 'string') b = window._cache.find(x => x.pnr === b) || b; if (typeof b !== 'object') return; showTicket(b); };
  } catch (err) { box.innerHTML = `<div class="card">${err.message}</div>`; }
}
window.cancelBooking = async (pnr) => {
  if (!confirm('Cancel booking ' + pnr + '? Refund will be mocked instantly.')) return;
  try { await api('/api/bookings/' + pnr + '/cancel', { method: 'POST', headers: headers() }); toast('Booking ' + pnr + ' cancelled.'); loadMyBookings(); }
  catch (err) { toast(err.message); }
};

// ---------- admin ----------
let adminTab = 'flights';
let adminFlightFilter = '';
let adminBookingFilter = '';
let adminAllBookings = [];
function setAdminTab(t) {
  adminTab = t;
  const tf = $('#tabFlights'), tb = $('#tabBookings');
  if (tf) tf.classList.toggle('active', t === 'flights');
  if (tb) tb.classList.toggle('active', t === 'bookings');
  if ($('#adminFlightsPane')) $('#adminFlightsPane').style.display = t === 'flights' ? 'block' : 'none';
  if ($('#adminBookingsPane')) $('#adminBookingsPane').style.display = t === 'bookings' ? 'block' : 'none';
  if (t === 'bookings') loadAdminBookings();
}
async function loadAdmin() {
  if (!state.user || state.user.role !== 'admin') {
    if ($('#adminGate')) $('#adminGate').innerHTML = '<div class="card">Admin login required. Please log in with an admin account.</div>';
    if ($('#adminBody')) $('#adminBody').style.display = 'none';
    return;
  }
  if ($('#adminGate')) $('#adminGate').innerHTML = '';
  if ($('#adminBody')) $('#adminBody').style.display = 'block';
  try {
    const s = await api('/api/admin/stats', { headers: headers(false) });
    $('#statsRow').innerHTML = [['Flights', s.stats.flights], ['Users', s.stats.users], ['Bookings', s.stats.bookings], ['Active', s.stats.activeBookings], ['Revenue', inr(s.stats.revenue)]].map(([k, v]) => `<div class="stat"><span class="muted">${k}</span><b>${v}</b></div>`).join('');
    const f = await api('/api/flights', { headers: headers(false) });
    state.flights = f.flights;
    renderAdminFlights();
    if (adminTab === 'bookings') loadAdminBookings();
  } catch (err) { toast(err.message); }
}
function renderAdminFlights() {
  const q = (adminFlightFilter || '').toLowerCase();
  const list = (state.flights || []).filter(x => !q || [x.flightNumber, x.airline, x.from, x.to, x.date].join(' ').toLowerCase().includes(q));
  $('#adminFlights').innerHTML = list.length ? `<div class="card"><table><tr><th>Flight</th><th>Route</th><th>Date</th><th>Time</th><th>Price</th><th>Seats</th><th></th></tr>${list.map(x => `<tr><td>${x.airline} <b>${x.flightNumber}</b></td><td>${x.from}→${x.to}</td><td>${x.date}</td><td>${x.departure}–${x.arrival}</td><td>${inr(x.price)}</td><td>${x.availableSeats}/${x.totalSeats}</td><td><button class="btn btn-sm" onclick="editFlight('${x.id}')">Edit</button> <button class="btn btn-sm" onclick="delFlight('${x.id}')">Del</button></td></tr>`).join('')}</table></div>`
    : '<div class="card">No flights match.</div>';
}
async function loadAdminBookings() {
  try {
    const j = await api('/api/bookings', { headers: headers() });
    adminAllBookings = j.bookings || [];
```

mobile/www/app.js · **JavaScript** · **353 lines**

```
    renderAdminBookings();
  } catch (err) { if ($('#adminBookings')) $('#adminBookings').innerHTML = `<div class="card">${err.message}</div>`; }
}
function renderAdminBookings() {
  const q = (adminBookingFilter || '').toLowerCase();
  const list = adminAllBookings.filter(b => !q || [b.pnr, b.flightId, b.status, b.userId].join(' ').toLowerCase().includes(q));
  if (!$('#adminBookings')) return;
  $('#adminBookings').innerHTML = list.length ? `<div class="card"><table><tr><th>PNR</th><th>Flight</th><th>Seats</th><th>Total</th><th>Status</th><th></th></tr>${list.map(b => `<tr><td><b>${b.pnr}</b></td><td>${b.flightId}</td><td>${(b.seats || []).join(', ')}</td><td>${inr(b.totalPrice)}</td><td><span class="pill ${b.status === 'CONFIRMED' ? 'green' : 'red'}">${b.status}</span></td><td>${b.status === 'CONFIRMED' ? `<button class="btn btn-sm" onclick="adminCancelBooking('${b.pnr}')">Cancel</button>` : ''}</td></tr>`).join('')}</table></div>`
    : '<div class="card">No bookings found.</div>';
}
window.adminCancelBooking = async (pnr) => {
  if (!confirm('Cancel booking ' + pnr + ' (admin)?')) return;
  try { await api('/api/bookings/' + pnr + '/cancel', { method: 'POST', headers: headers() }); toast('Booking ' + pnr + ' cancelled.'); loadAdminBookings(); loadAdmin(); } catch (err) { toast(err.message); }
};
window.editFlight = (id) => {
  const f = state.flights.find(x => x.id === id); if (!f) return;
  $('#flightFormTitle').textContent = 'Edit flight ' + f.flightNumber;
  $('#fId').value = f.id; $('#fAirline').value = f.airline; $('#fNum').value = f.flightNumber;
  $('#fFrom').value = f.from; $('#fTo').value = f.to; $('#fDate').value = f.date;
  $('#fDep').value = f.departure; $('#fArr').value = f.arrival; $('#fPrice').value = f.price;
  window.scrollTo({ top: 0, behavior: 'smooth' });
};
window.delFlight = async (id) => {
  if (!confirm('Delete this flight?')) return;
  try { await api('/api/flights/' + id, { method: 'DELETE', headers: headers() }); toast('Flight deleted.'); loadAdmin(); } catch (err) { toast(err.message); }
};
$('#fReset').onclick = () => { $('#flightForm').reset(); $('#fId').value = ''; $('#flightFormTitle').textContent = 'Add flight'; };
if ($('#tabFlights')) $('#tabFlights').onclick = () => setAdminTab('flights');
if ($('#tabBookings')) $('#tabBookings').onclick = () => setAdminTab('bookings');
if ($('#adminSearchBtn')) $('#adminSearchBtn').onclick = () => { adminFlightFilter = $('#adminSearch').value.trim(); renderAdminFlights(); };
if ($('#adminResetBtn')) $('#adminResetBtn').onclick = () => { adminFlightFilter = ''; $('#adminSearch').value = ''; renderAdminFlights(); };
if ($('#adminSearch')) $('#adminSearch').onkeydown = (e) => { if (e.key === 'Enter') { e.preventDefault(); adminFlightFilter = e.target.value.trim(); renderAdminFlights(); } };
if ($('#bookingSearchBtn')) $('#bookingSearchBtn').onclick = () => { adminBookingFilter = $('#bookingSearch').value.trim(); renderAdminBookings(); };
if ($('#bookingResetBtn')) $('#bookingResetBtn').onclick = () => { adminBookingFilter = ''; $('#bookingSearch').value = ''; renderAdminBookings(); };
if ($('#bookingRefreshBtn')) $('#bookingRefreshBtn').onclick = () => loadAdminBookings();
$('#flightForm').onsubmit = async (e) => {
  e.preventDefault();
  const body = { airline: $('#fAirline').value, flightNumber: $('#fNum').value, from: $('#fFrom').value.toUpperCase(), to: $('#fTo').value.toUpperCase(), date: $('#fDate').value, departure: $('#fDep').value, arrival: $('#fArr').value, durationMin: 120, price: parseFloat($('#fPrice').value), totalSeats: 180 };
  try {
    const id = $('#fId').value;
    if (id) await api('/api/flights/' + id, { method: 'PUT', headers: headers(), body: JSON.stringify(body) });
    else await api('/api/flights', { method: 'POST', headers: headers(), body: JSON.stringify(body) });
    toast('Flight saved.'); $('#flightForm').reset(); $('#fId').value = ''; loadAdmin();
  } catch (err) { toast(err.message); }
};

// ---------- init ----------
async function init() {
  refreshServerBtn();
```

mobile/www/app.js · JavaScript · 353 lines

```
  if (isNativeApp && !API) {
    const v = prompt('Enter your Airoplane Ticket Service server URL\n(same Wi-Fi as your laptop, e.g. http://192.168.1.5:8080)', '');
    if (v) setApiBase(v);
  }
  try {
    const j = await api('/api/flights', { headers: headers(false) });
    state.airports = j.airports; fillAirports();
    const t = new Date(Date.now() + 86400000).toISOString().slice(0, 10);
    const j2 = await api(`/api/flights?from=DEL&to=BOM&date=${t}`, { headers: headers(false) });
    state.flights = j2.flights; renderResults();
  } catch (err) { $('#results').innerHTML = `<div class="card">Backend not reachable: ${err.message}. ${isNativeApp ? 'Tap ⚙ to set the server URL.' : 'Start it with <code>./run.sh</code>.'}</div>`; }
  await restoreSession();
}
init();
```

mobile/www/manifest.webmanifest · JSON · 17 lines

```
{
  "name": "Airoplane Ticket Service",
  "short_name": "Airoplane",
  "description": "Search, book and manage domestic flight tickets.",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "orientation": "portrait",
  "background_color": "#0f172a",
  "theme_color": "#0ea5e9",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" },
    { "src": "/maskable-512.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable" }
  ]
}
```

mobile/android/app/src/main/java/in/skyjet/app/MainActivity.java · Java · 6 lines

```
package in.skyjet.app;

import com.getcapacitor.BridgeActivity;

public class MainActivity extends BridgeActivity {}
```

mobile/android/app/src/main/AndroidManifest.xml · XML · 42 lines

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">

        <activity
            android:configChanges="orientation|keyboardHidden|keyboard|screenSize|locale|smallestScreenSize|screenLayout|uiMode|navigation"
            android:name=".MainActivity"
            android:label="@string/title_activity_main"
            android:theme="@style/AppTheme.NoActionBarLaunch"
            android:launchMode="singleTask"
            android:exported="true">

```

mobile/android/app/src/main/AndroidManifest.xml · XML · 42 lines

```xml
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>

        <provider
            android:name="androidx.core.content.FileProvider"
            android:authorities="${applicationId}.fileprovider"
            android:exported="false"
            android:grantUriPermissions="true">
            <meta-data
                android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/file_paths"></meta-data>
        </provider>
    </application>

    <!-- Permissions -->

    <uses-permission android:name="android.permission.INTERNET" />
</manifest>

```

mobile/android/app/src/main/assets/capacitor.config.json · JSON · 9 lines

```json
{
        "appId": "in.skyjet.app",
        "appName": "SkyJet",
        "webDir": "www",
        "android": {
                "allowMixedContent": false
        }
}

```

mobile/android/app/src/main/res/layout/activity_main.xml · XML · 13 lines

```xml
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <WebView
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

mobile/android/app/src/main/res/values/strings.xml · XML · 8 lines

```xml
<?xml version='1.0' encoding='utf-8'?>
<resources>
    <string name="app_name">Airoplane Ticket Service</string>
    <string name="title_activity_main">Airoplane Ticket Service</string>
    <string name="package_name">in.skyjet.app</string>
    <string name="custom_url_scheme">in.skyjet.app</string>
</resources>

```

mobile/android/app/src/main/res/values/styles.xml · XML · 22 lines

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>

```

mobile/android/app/src/main/res/values/styles.xml · XML · 22 lines

```xml
    <!-- Base application theme. -->
    <style name="AppTheme" parent="Theme.AppCompat.Light.DarkActionBar">
        <!-- Customize your theme here. -->
        <item name="colorPrimary">@color/colorPrimary</item>
        <item name="colorPrimaryDark">@color/colorPrimaryDark</item>
        <item name="colorAccent">@color/colorAccent</item>
    </style>

    <style name="AppTheme.NoActionBar" parent="Theme.AppCompat.DayNight.NoActionBar">
        <item name="windowActionBar">false</item>
        <item name="windowNoTitle">true</item>
        <item name="android:background">@null</item>
    </style>


    <style name="AppTheme.NoActionBarLaunch" parent="Theme.SplashScreen">
        <item name="android:background">@drawable/splash</item>
    </style>
</resources>
```

mobile/android/app/src/main/res/xml/config.xml · XML · 6 lines

```xml
<?xml version='1.0' encoding='utf-8'?>
<widget version="1.0.0" xmlns="http://www.w3.org/ns/widgets" xmlns:cdv="http://cordova.apache.org/ns/1.0">
  <access origin="*" />


</widget>
```

mobile/android/app/build.gradle · Gradle · 55 lines

```
apply plugin: 'com.android.application'

android {
    namespace "in.skyjet.app"
    compileSdk rootProject.ext.compileSdkVersion
    defaultConfig {
        applicationId "in.skyjet.app"
        minSdkVersion rootProject.ext.minSdkVersion
        targetSdkVersion rootProject.ext.targetSdkVersion
        versionCode 1
        versionName "1.0"
        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
        aaptOptions {
             // Files and dirs to omit from the packaged assets dir, modified to accommodate modern web apps.
             // Default: https://android.googlesource.com/platform/frameworks/base/+/282e181b58cf72b6ca770dc7ca5f91f135444502/tools/aapt/AaptAssets.cpp#61
            ignoreAssetsPattern '!.svn:!.git:!.ds_store:!*.scc:.*:!CVS:!thumbs.db:!picasa.ini:!*~'
        }
    }
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
        }
    }
}

repositories {
    flatDir{
        dirs '../capacitor-cordova-android-plugins/src/main/libs', 'libs'
    }
}

dependencies {
    implementation fileTree(include: ['*.jar'], dir: 'libs')
    implementation "androidx.appcompat:appcompat:$androidxAppCompatVersion"
    implementation "androidx.coordinatorlayout:coordinatorlayout:$androidxCoordinatorLayoutVersion"
```

mobile/android/app/build.gradle · Gradle · 55 lines

```
    implementation "androidx.core:core-splashscreen:$coreSplashScreenVersion"
    implementation project(':capacitor-android')
    testImplementation "junit:junit:$junitVersion"
    androidTestImplementation "androidx.test.ext:junit:$androidxJunitVersion"
    androidTestImplementation "androidx.test.espresso:espresso-core:$androidxEspressoCoreVersion"
    implementation project(':capacitor-cordova-android-plugins')
}

apply from: 'capacitor.build.gradle'

try {
    def servicesJSON = file('google-services.json')
    if (servicesJSON.text) {
        apply plugin: 'com.google.gms.google-services'
    }
} catch(Exception e) {
    logger.info("google-services.json not found, google-services plugin not applied. Push Notificati
ons won't work")
}

```

mobile/android/build.gradle · Gradle · 30 lines

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.

buildscript {

    repositories {
        google()
        mavenCentral()
    }
    dependencies {
        classpath 'com.android.tools.build:gradle:8.7.2'
        classpath 'com.google.gms:google-services:4.4.2'

        // NOTE: Do not place your application dependencies here; they belong
        // in the individual module build.gradle files
    }
}

apply from: "variables.gradle"

allprojects {
    repositories {
        google()
        mavenCentral()
    }
}

task clean(type: Delete) {
    delete rootProject.buildDir
}

```

mobile/android/variables.gradle · Gradle · 16 lines

```
ext {
    minSdkVersion = 23
    compileSdkVersion = 35
    targetSdkVersion = 35
    androidxActivityVersion = '1.9.2'
    androidxAppCompatVersion = '1.7.0'
    androidxCoordinatorLayoutVersion = '1.2.0'
    androidxCoreVersion = '1.15.0'
    androidxFragmentVersion = '1.8.4'
    coreSplashScreenVersion = '1.0.1'
    androidxWebkitVersion = '1.12.1'
    junitVersion = '4.13.2'
    androidxJunitVersion = '1.2.1'
    androidxEspressoCoreVersion = '3.6.1'
```

mobile/android/variables.gradle · Gradle · 16 lines

```
    cordovaAndroidVersion = '10.1.1'
}
```

mobile/android/settings.gradle · Gradle · 5 lines

```
include ':app'
include ':capacitor-cordova-android-plugins'
project(':capacitor-cordova-android-plugins').projectDir = new File('./capacitor-cordova-android-plugins/')

apply from: 'capacitor.settings.gradle'
```

mobile/android/app/src/main/res/drawable/ic_launcher_background.xml · XML · 171 lines

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="108dp"
    android:height="108dp"
    android:viewportHeight="108"
    android:viewportWidth="108">
    <path
        android:fillColor="#26A69A"
        android:pathData="M0,0h108v108h-108z" />
    <path
        android:fillColor="#00000000"
        android:pathData="M9,0L9,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M19,0L19,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M29,0L29,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M39,0L39,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M49,0L49,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M59,0L59,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M69,0L69,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M79,0L79,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M89,0L89,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
```

mobile/android/app/src/main/res/drawable/ic_launcher_background.xml · XML · 171 lines

```
        android:pathData="M99,0L99,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,9L108,9"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,19L108,19"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,29L108,29"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,39L108,39"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,49L108,49"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,59L108,59"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,69L108,69"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,79L108,79"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,89L108,89"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,99L108,99"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M19,29L89,29"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M19,39L89,39"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M19,49L89,49"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
```

mobile/android/app/src/main/res/drawable/ic_launcher_background.xml · XML · 171 lines

```
        android:pathData="M19,59L89,59"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M19,69L89,69"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M19,79L89,79"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M29,19L29,89"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M39,19L39,89"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M49,19L49,89"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M59,19L59,89"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M69,19L69,89"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M79,19L79,89"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
</vector>

```

mobile/android/app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml · XML · 5 lines

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/ic_launcher_background"/>
    <foreground android:drawable="@mipmap/ic_launcher_foreground"/>
</adaptive-icon>
```

mobile/android/app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml · XML · 5 lines

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/ic_launcher_background"/>
    <foreground android:drawable="@mipmap/ic_launcher_foreground"/>
</adaptive-icon>
```

mobile/android/app/src/main/res/drawable-v24/ic_launcher_foreground.xml · XML · 35 lines

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:aapt="http://schemas.android.com/aapt"
    android:width="108dp"
    android:height="108dp"
    android:viewportHeight="108"
    android:viewportWidth="108">
```

mobile/android/app/src/main/res/drawable-v24/ic_launcher_foreground.xml · XML · 35 lines

```xml
    <path
        android:fillType="evenOdd"
        android:pathData="M32,64C32,64 38.39,52.99 44.13,50.95C51.37,48.37 70.14,49.57 70.14,49.57L108.26,87.69L108,109.01L75.97,107.97L32,64Z"
        android:strokeColor="#00000000"
        android:strokeWidth="1">
        <aapt:attr name="android:fillColor">
            <gradient
                android:endX="78.5885"
                android:endY="90.9159"
                android:startX="48.7653"
                android:startY="61.0927"
                android:type="linear">
                <item
                    android:color="#44000000"
                    android:offset="0.0" />
                <item
                    android:color="#00000000"
                    android:offset="1.0" />
            </gradient>
        </aapt:attr>
    </path>
    <path
        android:fillColor="#FFFFFF"
        android:fillType="nonZero"
        android:pathData="M66.94,46.02L66.94,46.02C72.44,50.07 76,56.61 76,64L32,64C32,56.61 35.56,50.11 40.98,46.06L36.18,41.19C35.45,40.45 35.45,39.3 36.18,38.56C36.91,37.81 38.05,37.81 38.78,38.56L44.25,44.05C47.18,42.57 50.48,41.71 54,41.71C57.48,41.71 60.78,42.57 63.68,44.05L69.11,38.56C69.84,37.81 70.98,37.81 71.71,38.56C72.44,39.3 72.44,40.45 71.71,41.19L66.94,46.02ZM62.94,56.92C64.08,56.9265,56.01 65,54.88C65,53.76 64.08,52.85 62.94,52.85C61.8,52.85 60.88,53.76 60.88,54.88C60.88,56.01 61.8,56.92 62.94,56.92ZM45.06,56.92C46.2,56.92 47.13,56.01 47.13,54.88C47.13,53.76 46.2,52.85 45.06,52.85C43.92,52.85 43,53.76 43,54.88C43,56.01 43.92,56.92 45.06,56.92Z"
        android:strokeColor="#00000000"
        android:strokeWidth="1" />
</vector>

```

Part 5 — Seed Data and Project Documentation

data/flights.json · JSON · 1 lines


```
[{"id":"f6","airline":"Air Himalaya","flightNumber":"AH-611","from":"BLR","to":"DEL","date":"2026-10-23","departure":"18:00","arrival":"20:45","durationMin":165,"price":7199.0,"totalSeats":180,"availableSeats":163,"bookedSeats":["4A","15A","26F","9B","24C","3A","13C","1B","30C","14C","4F","22E","29B","7E","7C","6F","18A"]},{"id":"f7","airline":"Deccan Wings","flightNumber":"DW-702","from":"DEL","to":"MAA","date":"2026-10-24","departure":"08:30","arrival":"11:15","durationMin":165,"price":6499.0,"totalSeats":150,"availableSeats":131,"bookedSeats":["14A","30B","24B","6A","8F","8B","14C","9E","21E","14B","22D","3B","15C","9B","16D","11F","14F","27E","6D"]},{"id":"f8","airline":"SkyJet Airways","flightNumber":"SJ-808","from":"BOM","to":"DEL","date":"2026-10-24","departure":"19:20","arrival":"21:30","durationMin":130,"price":5899.0,"totalSeats":180,"availableSeats":173,"bookedSeats":["5F","3B","4D","25C","20D","7D","16F"]},{"id":"f9","airline":"Air Himalaya","flightNumber":"AH-909","from":"HYD","to":"BOM","date":"2026-10-26","departure":"10:05","arrival":"11:25","durationMin":120,"price":4299.0,"totalSeats":180,"availableSeats":157,"bookedSeats":["27E","1E","13B","24B","10C","26C","28B","22A","17B","27D","7A","9B","3A","23D","14E","23C","29E","13C","16C","18B","24F","2F","15F"]},{"id":"f10","airline":"Deccan Wings","flightNumber":"DW-515","from":"BLR","to":"HYD","date":"2026-10-22","departure":"16:40","arrival":"17:50","durationMin":70,"price":3599.0,"totalSeats":120,"availableSeats":96,"bookedSeats":["24B","12E","6F","16B","20C","12C","22E","7C","21B","1C","10F","2C","15D","19E","22A","7B","23C","20A","6A","16A","22C","4C","29A","30F"]},{"id":"f1","airline":"SkyJet Airways","flightNumber":"SJ-101","from":"DEL","to":"BOM","date":"2026-10-22","departure":"06:00","arrival":"08:10","durationMin":130,"price":5499.0,"totalSeats":180,"availableSeats":158,"bookedSeats":["10C","19C","30B","13B","5C","1B","9E","5B","30F","24B","6F","2F","4F","17B","16B","29D","7D","8A","24A","7A","12E","21F"]},{"id":"f2","airline":"SkyJet Airways","flightNumber":"SJ-204","from":"DEL","to":"BOM","date":"2026-10-22","departure":"14:30","arrival":"16:45","durationMin":135,"price":6299.0,"totalSeats":180,"availableSeats":167,"bookedSeats":["22B","12A","26D","16F","16D","10D","2C","10A","17E","8F","18D","7C","1A"]},{"id":"f3","airline":"Air Himalaya","flightNumber":"AH-312","from":"DEL","to":"BLR","date":"2026-10-22","departure":"09:15","arrival":"12:00","durationMin":165,"price":6999.0,"totalSeats":180,"availableSeats":167,"bookedSeats":["4E","8F","13D","11A","20C","16C","1A","18F","4B","12B","18B","25A","1D"]},{"id":"f4","airline":"Deccan Wings","flightNumber":"DW-418","from":"BOM","to":"BLR","date":"2026-10-23","departure":"07:45","arrival":"09:20","durationMin":95,"price":4999.0,"totalSeats":150,"availableSeats":130,"bookedSeats":["14E","3F","9E","15F","15B","14D","21D","26A","20C","19F","12A","13C","24F","8C","27B","14B","27D","25D","7B","4D"]},{"id":"f5","airline":"SkyJet Airways","flightNumber":"SJ-520","from":"DEL","to":"HYD","date":"2026-10-23","departure":"11:00","arrival":"13:10","durationMin":130,"price":5799.0,"totalSeats":180,"availableSeats":155,"bookedSeats":["25B","22A","1B","17C","8E","24F","1E","17A","6A","27C","16D","29D","3E","20C","6E","13C","13B","13F","3B","6D","9F","26C","28C","13E","14B"]}]
```


data/users.json · JSON · 1 lines


```
[{"id":"u-1bf60ce0","name":"karthik123@gmail.com","email":"karthik123@gmail.com","role":"customer","passwordHash":"7f1d2d940a22ce0d9f84f4af7af310c6e50cf84141c9d9b71d02beef2639c521"},{"id":"u-demo","name":"Demo Flyer","email":"demo@skyjet.in","role":"customer","passwordHash":"d3ad9315b7be5dd53b31a273b3b3aba5defe700808305aa16a3062b76658a791"},{"id":"u-admin","name":"Admin","email":"admin@skyjet.in","role":"admin","passwordHash":"240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9"}]
```


README.md · Markdown · 140 lines

```
# ✈ Airoplane Ticket Service (Java)

> 📕 **[Airoplane-Ticket-Service-Abstract.pdf](Airoplane-Ticket-Service-Abstract.pdf)** — the finished,
> print-ready PDF (9 pages, A4, with page numbers) — submit this one.
>
> 📄 **[ABSTRACT.md](ABSTRACT.md)** — the project abstract, ready to export as PDF: fill-in title block,
> abstract and keywords, objectives, languages used, methodology, modules, main code, testing, conclusion
> and future scope.

Full flight-ticket booking app in **pure Java (no external dependencies)** + HTML/CSS/JS frontend.

- **Search & booking flow** — from/to/date, flight list, seat map (30×6), passengers, mock payment
- **Admin approval flow** — a new booking is **PENDING**; the e-ticket is issued only after an admin confirms it
- **Auth + My bookings** — register/login (token), booking history, view/print ticket, cancel with refund
- **Payments mock** — test card validation, fake `PAY-XXXX` transactions
```

README.md · Markdown · 140 lines

```
- **Admin dashboard** — revenue/booking stats, pending-approval queue, add/edit/delete flights, all-bookings tab with search
- **Airport map** — real interactive map (Leaflet + OpenStreetMap) with airport locations and route network
- **Persistence** — `data/*.json` files (flights, users, bookings)

## ⏳ Booking approval flow

1. The customer books a flight and pays (mock) → the booking is created with status **PENDING**.
   The seat is held so nobody else can take it, but **no e-ticket is issued yet**.
2. The customer sees the booking as `PENDING · awaiting admin` with the note
   *"Waiting for admin confirmation"* — printing is locked.
3. The admin opens **Admin → 🛂 Bookings**, selects the pending row and clicks **✓ Confirm booking**.
4. The booking becomes **CONFIRMED**, the e-ticket is issued and printing unlocks for the customer.
   A pending booking can also be cancelled (which releases the seat).

Status values: `PENDING` → `CONFIRMED` | `CANCELLED`. Revenue counts confirmed bookings only.

## 🖥️ Desktop app (macOS) — native Java window

The app is a **native Java/Swing desktop application** (no browser needed):

- **`AiroplaneTicketService.app`** — double-clickable app (bundled Java runtime, custom plane icon).
  Opens a native window: login → **Search · Airports · My bookings · Admin** tabs, animated sky-and-plane
  background, 30×6 seat map, passenger forms, mock payment, printable e-ticket.
  Already installed in `/Applications` as **Airoplane Ticket Service**.
- **`AiroplaneTicketService.command`** — double-click launcher for the same native window (needs Java 17+).
- **`AiroplaneTicketService.jar`** — portable JAR (any OS with Java 17+):
  ```bash
  java -jar AiroplaneTicketService.jar --ui
  ```
- The HTTP server keeps running in the background on port 8080, so phones on the same Wi-Fi can still
  reach the app at `http://<laptop-ip>:8080`.
- Stop the server: `lsof -ti tcp:8080 | xargs kill` (or close the app window).
- The app stores its data in `AiroplaneTicketService.app/Contents/app/data/`.
- Copying the `.app` to another Mac may require a right-click → **Open** (unsigned local build).

To rebuild the app files after code changes:
```bash
./run.sh                          # compiles to out/ and starts the server
jar --create --file AiroplaneTicketService.jar --main-class com.airline.App -C out .
jpackage --name AiroplaneTicketService --input <dir-with-jar> \
  --main-jar AiroplaneTicketService.jar --type app-image --dest . --app-version 1.0.0
```

📱 Mobile app (Android APK + PWA)

- **`SkyJet.apk`** — native Android app. Copy to your phone, install, enter
  the server URL (`http://<laptop-ip>:8080`) on first launch. Same Wi-Fi as the laptop.
- **PWA** — on your phone browser open `http://<laptop-ip>:8080` → Add to Home Screen.
  Works on Android + iPhone, runs fullscreen, shell works offline.
- Details + rebuild steps: [`mobile/README.md`](mobile/README.md).

Run from source (no Maven needed — just Java 17+)

```bash
cd airplane-ticket-service
chmod +x run.sh
./run.sh
# open http://localhost:8080
```

Custom port: `PORT=9090 ./run.sh`

Run with Maven (optional)
```

README.md · Markdown · 140 lines

```bash
mvn package
java -cp target/classes com.airline.App --port 8080
```

Demo accounts

Role	Email	Password
Customer	`demo@skyjet.in`	`demo123`
Admin	`admin@skyjet.in`	`admin123`

API

Method	Endpoint	Auth	Description
POST	`/api/auth/register`	–	`{name,email,password}`
POST	`/api/auth/login`	–	`{email,password}` → `{user,token}`
GET	`/api/auth/me`	Bearer	current user
GET	`/api/flights?from=DEL&to=BOM&date=2026-09-19`	–	search
GET	`/api/flights/{id}`	–	one flight + seat map
POST/PUT/DELETE	`/api/flights…`	Admin	manage flights
POST	`/api/payments/mock`	–	`{amount,cardNumber}` → fake success
POST	`/api/bookings`	Bearer	`{flightId,passengers[],seats[],paymentId}`
GET	`/api/bookings`	Bearer	my bookings (all if admin)
GET	`/api/bookings/{pnr}`	Bearer	e-ticket
POST	`/api/bookings/{pnr}/cancel`	Bearer	cancel + free seats
POST	`/api/bookings/{pnr}/confirm`	Admin	approve a pending booking, issue the e-ticket
GET	`/api/admin/stats`	Admin	flights/users/revenue/pending

Project layout

```
src/main/java/com/airline/
  App.java                ← HTTP server + routing (port 8080)
  model/Flight.java, User.java, Booking.java
  store/DataStore.java    ← JSON file DB + seed data
  service/AuthService.java, FlightService.java, BookingService.java
  util/Json.java, HttpUtil.java, Passwords.java
  ui/                     ← native Swing desktop app
    MainFrame.java        ← header + Search/Airports/Bookings/Admin tabs
    LoginDialog.java      ← login + register
    SearchPanel.java      ← search form + flight cards
    AirportsPanel.java    ← painted map + airport list
    BookingDialog.java    ← seat map, passengers, mock payment
    TicketDialog.java     ← e-ticket with print
    BookingsPanel.java    ← my bookings list
    AdminPanel.java       ← stats, flight CRUD, all bookings
    SkyPanel.java         ← animated plane background
    Theme.java            ← colors, fonts, widget factories
src/main/resources/static/ ← index.html, styles.css, app.js (web/PWA UI for phones)
data/                     ← auto-created flights.json, users.json, bookings.json
```

Notes

- Passwords are SHA-256 hashed (demo-grade; use bcrypt in production).
- Seats are allocated per-flight and freed on cancellation; double-booking is prevented server-side.
- To reset demo data, stop the server and delete `data/*.json`.
- The desktop app stores its data in `AiroplaneTicketService.app/Contents/app/data/`.